

逻辑验证漫游指南

The Hitchhiker's Guide to Logical Verification 2026

目录

前言	i
第一章 LoVe 序言	1
1.1 证明助理	1
1.2 成功案例	1
1.3 Lean	2
1.4 我们的目标	2
第二章 LoVe 演示 1: 类型与项	3
2.1 Lean 的一种图景	3
2.2 类型	3
2.3 项	3
2.4 类型检查与类型推断	5
2.5 类型栖居问题	5
第三章 LoVe 演示 2: 程序与定理	7
3.1 类型定义	7
3.2 函数定义	8
3.3 定理陈述	11
第四章 LoVe 演示 3: 反向证明	13
4.1 策略模式	13
4.2 基本策略	14
4.3 关于逻辑联结词和量词的推理	16
4.4 关于等式的推理	18
4.5 数学归纳证明	19
4.6 清理策略	21
第五章 LoVe 演示 4: 正向证明	22
5.1 结构化构造	22
5.2 关于联结词和量词的正向推理	24

5.3	计算式证明	26
5.4	用策略进行正向推理	27
5.5	依赖类型	28
5.6	PAT 原则	29
5.7	通过模式匹配和递归进行归纳	30
第六章	LoVe 演示 5: 函数式程序设计	32
6.1	归纳类型	32
6.2	结构归纳	32
6.3	结构递归	33
6.4	模式匹配表达式	34
6.5	结构体	35
6.6	类型类	36
6.7	列表	37
6.8	二叉树	41
6.9	依赖归纳类型 (可选)	43
第七章	LoVe 演示 6: 归纳谓词	44
7.1	引导性例子	44
7.2	逻辑符号	47
7.3	规则归纳	48
7.4	消去	50
7.5	更多例子	52
第八章	LoVe 演示 7: 带效应程序设计	57
8.1	引导性例子	57
8.2	两个运算和三条定律	59
8.3	Monad 的类型类	61
8.4	无效应	62
8.5	基本异常	63
8.6	可变状态	64
8.7	非确定性	65
8.8	一个泛型算法: 遍历列表	66
第九章	LoVe 演示 8: 元编程	68
9.1	策略组合子	69
9.2	宏	70
9.3	元编程 Monad	71
9.4	第一个例子: 一个 assumption 策略	72
9.5	表达式	73
9.6	第二个例子: 析取合取的策略	75

9.7 第三个例子：直接证明查找器	77
第十章 LoVe 演示 9：操作语义	80
10.1 首先：形式化项目	80
10.2 形式语义	81
10.3 一个极简命令式语言	82
10.4 大步语义	83
10.5 大步语义的性质	85
10.6 小步语义	89
10.7 小步语义的性质	92
第十一章 LoVe 演示 10：Hoare 逻辑	99
11.1 Hoare 三元组	99
11.2 Hoare 规则	100
11.3 Hoare 逻辑的语义方法	101
11.4 第一个程序：交换两个变量	105
11.5 第二个程序：两个数相加	105
11.6 一个验证条件生成器	106
11.7 重新考察第二个程序：两个数相加	108
11.8 总正确性的 Hoare 三元组	108
11.9 组合性	109
11.10 关系式指称语义	110
11.11 不动点	110
11.12 单调函数	111
11.13 完全格	113
11.14 最小不动点	114
11.15 关系式指称语义，续	115
11.16 程序等价性的应用	116
11.17 指称语义与大步语义的等价性	117
11.18 （可选）	117
11.19 基于归纳谓词的更简单方法（可选）	120
第十二章 LoVe 演示 12：数学的逻辑基础	121
12.1 宇宙	121
12.2 Prop 的特殊性	122
12.3 选择公理	125
12.4 子类型	126
12.5 商类型	129
12.6 第一个例子：整数	129
第十三章 LoVe 演示 13：基本数学结构	135

13.1 单个二元运算上的类型类	135
13.2 具有两个二元运算的类型类	138
13.3 强制转换	141
13.4 序类型类	143
13.5 有理数	144

前言

本 PDF 由仓库中的独立中文译稿自动生成。译稿以英文原书和英文 **Lean** 注释为依据；**Lean** 代码保持原样，以便与形式化文件对应阅读。

第一章 LoVe 序言

1.1 证明助理

证明助理（也称为交互式定理证明器）

- 检查并帮助开发形式化证明；
- 可用于证明大型定理，而不只是用于求解逻辑谜题；
- 使用起来可能相当繁琐；
- 极易令人着迷（不妨想想电子游戏）。

按照逻辑基础分类，一些证明助理如下：

- 集合论：Isabelle/ZF、Metamath、Mizar；
- 简单类型论：HOL4、HOL Light、Isabelle/HOL、PVS；
- 依赖类型论：Agda、Lean、Matita、Rocq。

1.2 成功案例

数学中的例子：

- 四色定理；
- **Kepler** 猜想；
- **perfectoid** 空间的定义。

计算机科学中的例子：

- 硬件；
- 操作系统；
- 程序设计语言理论；
- 编译器；
- 安全性。

1.3 Lean

Lean 是一个证明助理，自 2012 年起主要由 Leonardo de Moura (Amazon Web Services) 开发。

它的数学库 **mathlib** 由一个大型贡献者社区开发。

我们使用 **Lean 4** 的社区版本。除其他库外，我们使用它的基础库、**mathlib4** 以及 **LoVelib**。Lean 是一个研究项目。

它的优势包括：

- 基于一种称为归纳构造演算的依赖类型论，逻辑表达能力很强；
- 扩展有经典公理和商类型；
- 元编程框架；
- 现代化的用户界面；
- 文档；
- 开源；
- 双关语的无尽来源（Lean Forward、Lean Together、Boolean，等等）。

1.4 我们的目标

我们希望你

- 掌握交互式定理证明中的基本理论和技术；
- 熟悉若干应用领域；
- 培养一些可用于较大项目的实践技能，无论该项目是业余项目、硕士或博士项目，还是工业项目；
- 准备好迁移到另一种证明助理，并运用已经学到的知识；
- 对该领域有足够深入的理解，能够开始阅读科学论文。

本课程既不是一门纯粹的逻辑基础课程，也不是 **Lean** 教程。**Lean** 是我们的手段，而不是目的本身。

第二章 LoVe 演示 1: 类型与项

我们的旅程从学习 Lean 的基础开始，首先研究项（表达式）及其类型。

2.1 Lean 的一种图景

作为第一近似，可以把 Lean 看作

Lean = 函数式程序设计 + 逻辑

在今天的课程中，我们介绍类型与项的语法。它们类似于简单类型 λ 演算或带类型的函数式程序设计语言（ML、OCaml、Haskell）中的类型与项。

2.2 类型

类型 σ 、 τ 、 u :

- 类型变量 α ;
- 基本类型 T ;
- 复合类型 $T \sigma_1 \cdots \sigma_N$ 。

某些类型构造子 T 采用中缀记法，例如 $\rightarrow$ （函数类型）。

函数箭头是右结合的： $\sigma_1 \rightarrow \sigma_2 \rightarrow \sigma_3 \rightarrow \tau = \sigma_1 \rightarrow (\sigma_2 \rightarrow (\sigma_3 \rightarrow \tau))$ 。

多态类型同样是可能的。在 Lean 中，类型变量必须用 $\forall$ 绑定，例如 $\forall \alpha, \alpha \rightarrow \alpha$ 。

2.3 项

项 t 、 u :

- 常量 c ;
- 变量 x ;
- 应用 $t\ u$;
- 匿名函数 $\text{fun } x \mapsto t$ (也称为 λ 表达式)。

柯里化: 函数可以

- 完全应用 (例如, 如果 f 至多接受 3 个参数, 则可写 $f\ x\ y\ z$);
- 部分应用 (例如 $f\ x\ y$ 、 $f\ x$);
- 完全不应用 (例如 f)。

应用是左结合的: $f\ x\ y\ z = ((f\ x)\ y)\ z$ 。

`#check` 报告其参数的类型。

```
#check ℕ
```

```
#check ℤ
```

```
#check Empty
```

```
#check Unit
```

```
#check Bool
```

```
#check ℕ → ℤ
```

```
#check ℤ → ℕ
```

```
#check Bool → ℕ → ℤ
```

```
#check (Bool → ℕ) → ℤ
```

```
#check ℕ → (Bool → ℕ) → ℤ
```

```
#check fun x : ℕ ↦ x
```

```
#check fun f : ℕ → ℕ ↦ fun g : ℕ → ℕ ↦ fun h : ℕ → ℕ ↦  
  fun x : ℕ ↦ h (g (f x))
```

```
#check fun (f g h : ℕ → ℕ) (x : ℕ) ↦ h (g (f x))
```

`opaque` 定义一个具有指定类型的任意常量。

```
opaque a : ℤ
```

```
opaque b : ℤ
```

```
opaque f : ℤ → ℤ
```

```
opaque g : ℤ → ℤ → ℤ
```

```
#check fun x : ℤ ↦ g (f (g a x)) (g x b)
```

#check fun x $\mapsto$ g (f (g a x)) (g x b)

#check fun x $\mapsto$ x

2.4 类型检查与类型推断

类型检查和类型推断是可判定问题（不过，一旦加入重载或子类型等特性，这一性质很快就会丧失）。

类型判断: $C \vdash t : \sigma$, 意思是在局部语境 C 中, t 具有类型 σ 。

类型规则:

————— Cst 若 c 在全局中以类型 σ 声明
 $C \vdash c : \sigma$

————— Var 若 $x : \sigma$ 是 C 中最右侧出现的 x
 $C \vdash x : \sigma$

$C \vdash t : \sigma \rightarrow \tau \quad C \vdash u : \sigma$
 ————— App
 $C \vdash tu : \tau$

$C, x : \sigma \vdash t : \tau$
 ————— Fun
 $C \vdash (\text{fun } x : \sigma \mapsto t) : \sigma \rightarrow \tau$

如果同一个变量 x 在语境 C 中出现多次, 则最右侧的出现会遮蔽其他出现。

2.5 类型栖居问题

给定一个类型 σ , 类型栖居 问题要求找出一个具有该类型的项。类型栖居问题是不可判定的。

一个递归过程如下:

1. 如果 σ 形如 $\tau \rightarrow \upsilon$, 则一个候选栖居元是形如 $\text{fun } x \mapsto _$ 的匿名函数。
2. 或者, 可以使用任意常量或变量 $x : \tau_1 \rightarrow \dots \rightarrow \tau_N \rightarrow \sigma$ 来构造项 $x _ \dots _$ 。

```
opaque  $\alpha$  : Type
```

```
opaque  $\beta$  : Type
```

```
opaque  $\gamma$  : Type
```

```
def someFunOfType : ( $\alpha \rightarrow \beta \rightarrow \gamma$ )  $\rightarrow$  (( $\beta \rightarrow \alpha$ )  $\rightarrow \beta$ )  $\rightarrow \alpha \rightarrow \gamma$  :=  
  fun f g a  $\mapsto$  f a (g (fun b  $\mapsto$  a))
```

第三章 LoVe 演示 2：程序与定理

我们继续学习 **Lean** 的基础。本讲聚焦于程序与定理，但暂时还不进行任何证明。我们将回顾如何定义新的类型和函数，以及如何把它们应当满足的性质陈述为定理。

3.1 类型定义

归纳类型（也称为 __ 归纳数据类型 __、代数数据类型，或简称为 __ 数据类型 __）是这样的类型：它恰好由那些能够通过有限次应用其 __ 构造子 __ 而构造出来的值组成。

3.1.1 自然数

使用一元记法定义自然数类型 **Nat** ($= \mathbb{N}$):

```
inductive Nat : Type where
  | zero : Nat
  | succ : Nat → Nat
```

```
#check Nat
#check Nat.zero
#check Nat.succ
```

```
#print 输出其参数的定义。
```

```
#print Nat
```

在命名空间 **MyNat** 之外，除非使用 **MyNat** 命名空间加以限定，**Nat** 指的是 **Lean** 核心库中定义的类型。

```
#print Nat
#print MyNat.Nat
```

3.1.2 算术表达式

```
inductive AExp : Type where
| num :  $\mathbb{Z}$  → AExp
| var : String → AExp
| add : AExp → AExp → AExp
| sub : AExp → AExp → AExp
| mul : AExp → AExp → AExp
| div : AExp → AExp → AExp
```

3.1.3 列表

```
inductive List ( $\alpha$  : Type) where
| nil : List  $\alpha$ 
| cons :  $\alpha$  → List  $\alpha$  → List  $\alpha$ 
```

```
#check List
#check List.nil
#check List.cons
#print List
```

```
#print List
#print MyList.List
```

3.2 函数定义

定义作用于归纳类型的函数时，语法非常紧凑：我们定义一个单一函数，并使用 `__ 模式匹配 __` 从构造子中取出参数。

```
def fib :  $\mathbb{N} \rightarrow \mathbb{N}$ 
| 0   => 0
| 1   => 1
| n + 2 => fib (n + 1) + fib n
```

当有多个参数时，用，分隔各个模式：

```
def add :  $\mathbb{N} \rightarrow \mathbb{N} \rightarrow \mathbb{N}$ 
| m, Nat.zero   => m
| m, Nat.succ n => Nat.succ (add m n)
```

`#eval` 和 `#reduce` 对项求值并输出其值。

```
#eval add 2 7
#reduce add 2 7
```

```
def mul :  $\mathbb{N} \rightarrow \mathbb{N} \rightarrow \mathbb{N}$ 
| _, Nat.zero   => Nat.zero
| m, Nat.succ n => add m (mul m n)
```

```
#eval mul 2 7
```

```
#print mul
```

```
def power :  $\mathbb{N} \rightarrow \mathbb{N} \rightarrow \mathbb{N}$ 
| _, Nat.zero   => 1
| m, Nat.succ n => mul m (power m n)
```

```
#eval power 2 5
```

`add`、`mul` 和 `power` 都是人为构造的例子。这些运算在 `Lean` 中已经分别以 `+`、`*` 和 `^` 的形式提供。

如果没有必要对某个参数做模式匹配，可以把它移到 `:` 的左侧，使之成为一个具名参数：

```
def powerParam (m :  $\mathbb{N}$ ) :  $\mathbb{N} \rightarrow \mathbb{N}$ 
| Nat.zero   => 1
| Nat.succ n => mul m (powerParam m n)
```

```
#eval powerParam 2 5
```

```
def iter (α : Type) (z : α) (f : α → α) : ℕ → α
| Nat.zero => z
| Nat.succ n => f (iter α z f n)
```

```
#check iter
```

```
def powerIter (m n : ℕ) : ℕ :=
  iter ℕ 1 (mul m) n
```

```
#eval powerIter 2 5
```

```
def append (α : Type) : List α → List α → List α
| List.nil,    ys => ys
| List.cons x xs, ys => List.cons x (append α xs ys)
```

因为 `append` 必须适用于任意元素类型的列表，所以元素类型本身也作为参数给出。因此，每次调用都必须提供该类型（如果 `Lean` 能够推断出该类型，也可以使用 `_`）。

```
#check append
#eval append ℕ [3, 1] [4, 1, 5]
#eval append _ [3, 1] [4, 1, 5]
```

如果类型参数写在 `{ }` 而不是 `( )` 中，它就是隐式参数；只要 `Lean` 能够推断出它，就无须在每次调用中显式提供。

```
def appendImplicit {α : Type} : List α → List α → List α
| List.nil,    ys => ys
| List.cons x xs, ys => List.cons x (appendImplicit xs ys)
```

```
#eval appendImplicit [3, 1] [4, 1, 5]
```

在定义名前加上 `@`，可以得到相应定义的版本，其中所有隐式参数都被显式化。当 `Lean` 无法确定如何实例化隐式参数时，这很有用。

```
#check @appendImplicit
#eval @appendImplicit ℕ [3, 1] [4, 1, 5]
#eval @appendImplicit _ [3, 1] [4, 1, 5]
```


别名:

```
`[]`      := `List.nil`
`x :: xs`  := `List.cons x xs`
`[x₁, ..., xN]` := `x₁ :: ... :: xN :: []`
```

```
def appendPretty {α : Type} : List α → List α → List α
| [], ys => ys
| x :: xs, ys => x :: appendPretty xs ys
```

```
def reverse {α : Type} : List α → List α
| [] => []
| x :: xs => reverse xs ++ [x]
```

```
def eval (env : String → ℤ) : AExp → ℤ
| AExp.num i    => i
| AExp.var x    => env x
| AExp.add e₁ e₂ => eval env e₁ + eval env e₂
| AExp.sub e₁ e₂ => eval env e₁ - eval env e₂
| AExp.mul e₁ e₂ => eval env e₁ * eval env e₂
| AExp.div e₁ e₂ => eval env e₁ / eval env e₂
```

```
#eval eval (fun x ↦ 7) (AExp.div (AExp.var "y") (AExp.num 0))
```

Lean 只接受那些它能够证明会终止的函数定义。特别地，它接受 __ 结构递归 __ 函数；这种函数每次恰好剥去一个构造子。

3.3 定理陈述

注意它与 `def` 命令的相似性。`theorem` 类似于 `def`，区别在于其结果是命题，而不是数据或函数。

```
theorem add_comm (m n : ℕ) :
  add m n = add n m :=
  sorry
```

```
theorem add_assoc (l m n : ℕ) :
  add (add l m) n = add l (add m n) :=
  sorry
```

```
theorem mul_comm (m n :  $\mathbb{N}$ ) :  
  mul m n = mul n m :=  
  sorry
```

```
theorem mul_assoc (l m n :  $\mathbb{N}$ ) :  
  mul (mul l m) n = mul l (mul m n) :=  
  sorry
```

```
theorem mul_add (l m n :  $\mathbb{N}$ ) :  
  mul l (add m n) = add (mul l m) (mul l n) :=  
  sorry
```

```
theorem reverse_reverse { $\alpha$  : Type} (xs : List  $\alpha$ ) :  
  reverse (reverse xs) = xs :=  
  sorry
```

公理类似于没有证明的定理。不透明声明类似于没有主体的定义。

```
opaque a :  $\mathbb{Z}$   
opaque b :  $\mathbb{Z}$ 
```

```
axiom a_less_b :  
  a < b
```

第四章 LoVe 演示 3：反向证明

`__策略__` 作用于一个证明目标，并且要么证明该目标，要么产生新的子目标。策略是一种 `__反向__` 证明机制：它们从目标出发，朝着可用的假设和定理回溯。

4.1 策略模式

策略式证明的语法为：

```
by
  _tactic1_
  ...
  _tacticN_
```

关键字 `by` 告诉 Lean，该证明是策略式的。

```
theorem fst_of_two_props :
  ∀ a b : Prop, a → b → a :=
by
  intro a b
  intro ha hb
  apply ha
```

注意， $a \rightarrow b \rightarrow a$ 会被解析为 $a \rightarrow (b \rightarrow a)$ 。

在 Lean 中，命题是类型为 `Prop` 的项。`Prop` 是一个类型，正如 `Nat` 和 `List Bool` 也是类型。事实上，命题与类型之间存在紧密对应；这一点将在第 4 讲中解释。

4.2 基本策略

`intro` 将由 $\forall$ 量化的变量, 或者蕴涵 $\rightarrow$ 的前提, 从目标的结论 ($\vdash$ 之后) 移动到目标的假设 ($\vdash$ 之前) 中。

`apply` 将目标的结论与指定定理的结论相匹配, 并把该定理的假设加入为新的目标。

```
theorem fst_of_two_props_params (a b : Prop) (ha : a) (hb : b) :
  a :=
  by apply ha
```

```
theorem prop_comp (a b c : Prop) (hab : a  $\rightarrow$  b) (hbc : b  $\rightarrow$  c) :
  a  $\rightarrow$  c :=
  by
    intro ha
    apply hbc
    apply hab
    apply ha
```

上述证明逐步说明如下:

- 假设我们有 `a` 的一个证明。
- 目标是 `c`; 如果能证明 `b`, 则可由 `hbc` 证明 `c`。
- 目标是 `b`; 如果能证明 `a`, 则可由 `hab` 证明 `b`。
- 我们已经由 `ha` 知道 `a`。

接下来, `exact` 将目标的结论与指定的定理相匹配, 并关闭该目标。在许多这类情形中也可以使用 `apply`, 但 `exact` 更清楚地表达了我们的意图。

```
theorem fst_of_two_props_exact (a b : Prop) (ha : a) (hb : b) :
  a :=
  by exact ha
```

`assumption` 在局部语境中寻找一个与目标结论相匹配的假设, 并用它证明目标。

```
theorem fst_of_two_props_assumption (a b : Prop)
  (ha : a) (hb : b) :
  a :=
  by assumption
```

`rfl` 证明 $l = r$, 其中等式两边在计算意义下语法相同。这里的计算包括定义展开、 β -规约 (把 `fun` 应用于一个参数)、`let`, 以及更多情形。

```
theorem  $\alpha$ _example { $\alpha$   $\beta$  : Type} (f :  $\alpha \rightarrow \beta$ ) :
  (fun x  $\mapsto$  f x) = (fun y  $\mapsto$  f y) :=
  by rfl
```

```
theorem  $\beta$ _example { $\alpha$   $\beta$  : Type} (f :  $\alpha \rightarrow \beta$ ) (a :  $\alpha$ ) :
  (fun x  $\mapsto$  f x) a = f a :=
  by rfl
```

```
def double (n :  $\mathbb{N}$ ) :  $\mathbb{N}$  :=
  n + n
```

```
theorem  $\delta$ _example :
  double 5 = 5 + 5 :=
  by rfl
```

`let` 引入一个局部作用域中的定义。下面, `n := 2` 只在表达式 `n + n` 中处于作用域内。

```
theorem  $\zeta$ _example :
  (let n :  $\mathbb{N}$  := 2
   n + n) = 4 :=
  by rfl
```

```
theorem  $\eta$ _example { $\alpha$   $\beta$  : Type} (f :  $\alpha \rightarrow \beta$ ) :
  (fun x  $\mapsto$  f x) = f :=
  by rfl
```

`(a, b)` 是一个序对, 其第一分量为 `a`, 第二分量为 `b`。`Prod.fst` 是所谓的投影, 它取出一个序对的第一分量。

```
theorem  $\iota$ _example { $\alpha$   $\beta$  : Type} (a :  $\alpha$ ) (b :  $\beta$ ) :
  Prod.fst (a, b) = a :=
  by rfl
```

4.3 关于逻辑联结词和量词的推理

引入规则:

```
#check True.intro
#check And.intro
#check Or.inl
#check Or.inr
#check Iff.intro
#check Exists.intro
```

消去规则:

```
#check False.elim
#check And.left
#check And.right
#check Or.elim
#check Iff.mp
#check Iff.mpr
#check Exists.elim
```

$\neg$ 的定义及相关定理:

```
#print Not
#check Classical.em
#check Classical.byContradiction
```

对 Not ($\neg$) 没有显式规则, 因为 $\neg p$ 被定义为 $p \rightarrow \text{False}$ 。

theorem And_swap (a b : Prop) :

```
  a  $\wedge$  b  $\rightarrow$  b  $\wedge$  a :=
  by
    intro hab
    apply And.intro
    apply And.right
    exact hab
    apply And.left
    exact hab
```

上述证明逐步说明如下:

- 假设我们知道 $a \wedge b$ 。
- 目标是 $b \wedge a$ 。
- 证明 b ; 若能证明一个右侧为 b 的合取, 则可做到这一点。
- 可以做到, 因为我们已经有 $a \wedge b$ 。
- 证明 a ; 若能证明一个左侧为 a 的合取, 则可做到这一点。
- 可以做到, 因为我们已经有 $a \wedge b$ 。

组合子 · 聚焦于某一个特定的子目标。跟在它后面的策略必须完整证明该子目标。在下面的证明中, 我们对两个子目标分别使用 ; 以使证明更有结构。

theorem And_swap_braces :

$\forall a b : \text{Prop}, a \wedge b \rightarrow b \wedge a :=$

by

 intro a b hab

 apply And.intro

 • exact And.right hab

 • exact And.left hab

注意上面我们如何把假设 `hab` 直接传给定理 `And.right` 和 `And.left`, 而不是等待这些定理的假设作为新的子目标出现。这是在一个总体上反向的证明中迈出的小小步正向推理。

opaque f : $\mathbb{N} \rightarrow \mathbb{N}$

theorem f5_if (h : $\forall n : \mathbb{N}, f\ n = n$) :

$f\ 5 = 5 :=$

by exact h 5

theorem Or_swap (a b : Prop) :

$a \vee b \rightarrow b \vee a :=$

by

 intro hab

 apply Or.elim hab

 • intro ha

 exact Or.inr ha

 • intro hb

 exact Or.inl hb

theorem modus_ponens (a b : Prop) :

```

(a → b) → a → b :=
by
  intro hab ha
  apply hab
  exact ha

theorem Not_Not_intro (a : Prop) :
  a → ¬¬ a :=
by
  intro ha hna
  apply hna
  exact ha

theorem Exists_double_iden :
  ∃ n : ℕ, double n = n :=
by
  apply Exists.intro 0
  rfl

```

4.4 关于等式的推理

```

#check Eq.refl
#check Eq.symm
#check Eq.trans
#check Eq.subst

```

上述规则可以直接使用：

```

theorem Eq_trans_symm {α : Type} (a b c : α)
  (hab : a = b) (hcb : c = b) :
  a = c :=
by
  apply Eq.trans
  · exact hab
  · apply Eq.symm
    exact hcb

```

rw 把单个等式作为从左到右的重写规则应用一次。若要从右到左应用一个等式，在其名称前加

上 $\leftarrow$ 。

```
theorem Eq_trans_symm_rw {α : Type} (a b c : α)
  (hab : a = b) (hcb : c = b) :
  a = c :=
by
  rw [hab]
  rw [hcb]
```

`rw` 可以展开定义。在下面, $\neg\neg a$ 变为 $\neg a \rightarrow \text{False}$, 而 $\neg a$ 变为 $a \rightarrow \text{False}$ 。

```
theorem a_proof_of_negation (a : Prop) :
  a → ¬¬ a :=
by
  rw [Not]
  rw [Not]
  intro ha
  intro hna
  apply hna
  exact ha
```

`simp` 穷尽地应用一组标准重写规则 (称为 `simp` 集)。可以使用 `@[simp]` 属性扩展该集合。还可以用语法 `simp [_theorem1, ..., _theoremN]` 临时把定理加入 `simp` 集。

```
theorem cong_two_args_1p1 {α : Type} (a b c d : α)
  (g : α → α → ℕ → α) (hab : a = b) (hcd : c = d) :
  g a c (1 + 1) = g b d 2 :=
by simp [hab, hcd]
```

`ac_rfl` 类似于 `rfl`, 但它能够在 `+`、`*` 和其他二元运算符的结合律与交换律意义下进行推理。

```
theorem abc_Eq_cba (a b c : ℕ) :
  a + b + c = c + b + a :=
by ac_rfl
```

4.5 数学归纳证明

`induction` 对指定变量执行归纳。它会为每个构造子产生一个具名子目标。

```

theorem add_zero (n : ℕ) :
  add 0 n = n :=
by
  induction n with
  | zero    => rfl
  | succ n' ih => simp [add, ih]

```

```

theorem add_succ (m n : ℕ) :
  add (Nat.succ m) n = Nat.succ (add m n) :=
by
  induction n with
  | zero    => rfl
  | succ n' ih => simp [add, ih]

```

```

theorem add_comm (m n : ℕ) :
  add m n = add n m :=
by
  induction n with
  | zero    => simp [add, add_zero]
  | succ n' ih => simp [add, add_succ, ih]

```

```

theorem add_assoc (l m n : ℕ) :
  add (add l m) n = add l (add m n) :=
by
  induction n with
  | zero    => rfl
  | succ n' ih => simp [add, ih]

```

`ac_rfl` 是可扩展的。我们可以使用类型类实例机制（第 5 讲会解释）把 `add` 注册为交换且结合的运算符。这对下面调用 `ac_rfl` 很有用。

```

instance Associative_add : Std.Associative add :=
  { assoc := add_assoc }

```

```

instance Commutative_add : Std.Commutative add :=
  { comm := add_comm }

```

```

theorem mul_add (l m n : ℕ) :

```

```

mul l (add m n) = add (mul l m) (mul l n) :=
by
  induction n with
  | zero    => rfl
  | succ n' ih =>
    simp [add, mul, ih]
    ac_rfl

```

4.6 清理策略

`clear` 删除未使用的变量或假设。

`rename` 改变变量或假设的名称。

```

theorem cleanup_example (a b c : Prop) (ha : a) (hb : b)
  (hab : a → b) (hbc : b → c) :
  c :=
by
  clear ha hab a
  apply hbc
  clear hbc c
  rename b => h
  exact h

```

第五章 LoVe 演示 4: 正向证明

在构造证明时，采用 __ 正向 __ 工作方式往往是合理的：从我们已经知道的事实出发，一步一步推进到目标。Lean 的结构化证明和原始证明项，是支持正向推理的两种风格。

5.1 结构化构造

结构化证明是在 Lean 的 __ 证明项 __ 之上添加的一层语法糖。

最简单的结构化证明就是某个定理的名称，可能还带有参数。

```
theorem add_comm (m n : ℕ) :  
  add m n = add n m :=  
  sorry
```

```
theorem add_comm_zero_left (n : ℕ) :  
  add 0 n = add n 0 :=  
  add_comm 0 n
```

等价的反向证明如下：

```
theorem add_comm_zero_left_by_exact (n : ℕ) :  
  add 0 n = add n 0 :=  
  by exact add_comm 0 n
```

`fix` 和 `assume` 将由 $\forall$ 量化的变量以及假设从目标移动到局部语境中。它们可以看作 `intro` 策略的结构化版本。

`show` 重复要证明的目标。它可以作为文档说明，也可以在计算等价的意义下重新表述目标。

```
theorem fst_of_two_props :
```

```

  ∀a b : Prop, a → b → a :=
fix a b : Prop
assume ha : a
assume hb : b
show a from
  ha

```

```

theorem fst_of_two_props_show (a b : Prop) (ha : a) (hb : b) :
  a :=
show a from
  ha

```

```

theorem fst_of_two_props_no_show (a b : Prop) (ha : a) (hb : b) :
  a :=
  ha

```

have 证明一个中间定理；该中间定理可以引用局部语境。

```

theorem prop_comp (a b c : Prop) (hab : a → b) (hbc : b → c) :
  a → c :=
assume ha : a
have hb : b :=
  hab ha
have hc : c :=
  hbc hb
show c from
  hc

```

```

theorem prop_comp_inline (a b c : Prop) (hab : a → b)
  (hbc : b → c) :
  a → c :=
assume ha : a
show c from
  hbc (hab ha)

```

5.2 关于联结词和量词的正向推理

theorem And_swap (a b : Prop) :

```

  a ∧ b → b ∧ a :=
  assume hab : a ∧ b
  have ha : a :=
    And.left hab
  have hb : b :=
    And.right hab
  show b ∧ a from
    And.intro hb ha

```

theorem Or_swap (a b : Prop) :

```

  a ∨ b → b ∨ a :=
  assume hab : a ∨ b
  show b ∨ a from
    Or.elim hab
    (assume ha : a
      show b ∨ a from
        Or.inr ha)
    (assume hb : b
      show b ∨ a from
        Or.inl hb)

```

def double (n : ℕ) : ℕ :=

```

  n + n

```

theorem Nat_exists_double_iden :

```

  ∃ n : ℕ, double n = n :=
  Exists.intro 0
  (show double 0 = 0 from
    by rfl)

```

theorem Nat_exists_double_iden_no_show :

```

  ∃ n : ℕ, double n = n :=
  Exists.intro 0 (by rfl)

```

theorem modus_ponens (a b : Prop) :

```

(a → b) → a → b :=
assume hab : a → b
assume ha : a
show b from
  hab ha

```

```

theorem not_not_intro (a : Prop) :
  a → ¬¬ a :=
assume ha : a
assume hna : ¬ a
show False from
  hna ha

```

正如可以在反向证明中运用正向推理, 也可以在正向证明中运用反向推理 (用 **by** 标示):

```

theorem Forall.one_point {α : Type} (t : α) (P : α → Prop) :
  (∀x, x = t → P x) ↔ P t :=
Iff.intro
  (assume hall : ∀x, x = t → P x
    show P t from
      by
        apply hall t
        rfl)
  (assume hp : P t
    fix x : α
    assume heq : x = t
    show P x from
      by
        rw [heq]
        exact hp)

```

```

theorem beast_666 (beast : ℕ) :
  (∀n, n = 666 → beast ≥ n) ↔ beast ≥ 666 :=
Forall.one_point _ _

```

```
#print beast_666
```

```

theorem Exists.one_point {α : Type} (t : α) (P : α → Prop) :
  (∃x : α, x = t ∧ P x) ↔ P t :=

```

```

Iff.intro
  (assume hex :  $\exists x, x = t \wedge P x$ 
   show  $P t$  from
     Exists.elim hex
       (fix x :  $\alpha$ 
        assume hand :  $x = t \wedge P x$ 
        have hxt :  $x = t :=$ 
          And.left hand
        have hpx :  $P x :=$ 
          And.right hand
        show  $P t$  from
          by
            rw [←hxt]
            exact hpx))
  (assume hp :  $P t$ 
   show  $\exists x : \alpha, x = t \wedge P x$  from
     Exists.intro t
       (have tt :  $t = t :=$ 
        by rfl
        show  $t = t \wedge P t$  from
          And.intro tt hp))

```

5.3 计算式证明

在非形式化数学中，我们经常使用等式、不等式或等价关系的传递链（例如 $a \geq b \geq c$ ）。在 Lean 中，`calc` 支持这种计算式证明。

语法：

```

calc
  _term0_ _op1_ _term1_ :=
    _proof1_
  _ _op2_ _term2_ :=
    _proof2_
  :
  _ _opN_ _termN_ :=
    _proofN_

```

```

theorem two_mul_example (m n :  $\mathbb{N}$ ) :

```



```

2 * m + n = m + n + m :=
calc
2 * m + n = m + m + n :=
  by rw [Nat.two_mul]
_ = m + n + m :=
  by ac_rfl

```

calc 可以省去一些重复、一些 have 标签, 以及一些传递性推理:

```

theorem two_mul_example_have (m n : ℕ) :
  2 * m + n = m + n + m :=
  have hmul : 2 * m + n = m + m + n :=
    by rw [Nat.two_mul]
  have hcomm : m + m + n = m + n + m :=
    by ac_rfl
  show _ from
    Eq.trans hmul hcomm

```

5.4 用策略进行正向推理

结构化证明命令 **have**、**let** 和 **calc** 也可作为策略使用。即使在策略模式中, 以正向方式陈述中间结果和定义也可能很有用。

```

theorem prop_comp_tactical (a b c : Prop) (hab : a → b)
  (hbc : b → c) :
  a → c :=
  by
    intro ha
    have hb : b :=
      hab ha
    let c' := c
    have hc : c' :=
      hbc hb
    exact hc

```

5.5 依赖类型

依赖类型是依赖类型论这一逻辑家族的定义性特征。

考虑一个函数 `pick`，它接受一个数 $n : \mathbb{N}$ ，并返回一个介于 0 与 n 之间的数。概念上，`pick` 具有一个依赖类型，即

$\text{pick} : (n : \mathbb{N}) \rightarrow \{i : \mathbb{N} \mid i \leq n\}$

我们可以把这个类型理解为一个以 $\mathbb{N}$ 为指标的族，其中每个成员的类型都可以依赖于指标：

$\text{pick} : \{i : \mathbb{N} \mid i \leq n\} \rightarrow \mathbb{N}$

但一个类型也可以依赖于另一个类型，例如 `List`（或 $\text{fun } \alpha \mapsto \text{List } \alpha$ ）以及 $\text{fun } \alpha \mapsto \alpha \rightarrow \alpha$ 。

一个项可以依赖于一个类型，例如 $\text{fun } \alpha \mapsto \text{fun } (x : \alpha) \mapsto x$ （多态恒等函数）。

当然，一个项也可以依赖于另一个项。

除非另有说明，__ 依赖类型 __ 指的是依赖于项的类型。我们说简单类型论不支持依赖类型时，所说的正是这个意义。

概括地说，在归纳构造演算中， $\text{fun } x \mapsto t$ 有四种情形（参见 Barendregt 的 λ 立方）：

主体 (t)	参数 (x)	描述
项	依赖于 项	简单类型匿名函数
类型	依赖于 项	依赖类型（严格地说）
项	依赖于 类型	多态项
类型	依赖于 类型	类型构造子

修订后的类型规则：

$$\frac{C \vdash t : (x : \sigma) \rightarrow \tau[x] \quad C \vdash u : \sigma}{C \vdash t u : \tau[u]} \text{App'}$$

$$\frac{C, x : \sigma \vdash t : \tau[x]}{C \vdash (\text{fun } x : \sigma \mapsto t) : (x : \sigma) \rightarrow \tau[x]} \text{Fun'}$$

如果 x 不出现在 $\tau[x]$ 中，这两条规则就退化为 `App` 和 `Fun`。

`App'` 的例子：

$$\vdash \text{pick} : (n : \mathbb{N}) \rightarrow \{i : \mathbb{N} // i \leq n\} \quad \vdash 5 : \mathbb{N}$$

App'

$$\vdash \text{pick } 5 : \{i : \mathbb{N} // i \leq 5\}$$

Fun' 的例子:

$$\alpha : \text{Type}, x : \alpha \vdash x : \alpha$$

Fun or Fun'

$$\alpha : \text{Type} \vdash (\text{fun } x : \alpha \mapsto x) : \alpha \rightarrow \alpha$$

Fun'

$$\vdash (\text{fun } \alpha : \text{Type} \mapsto \text{fun } x : \alpha \mapsto x) : (\alpha : \text{Type}) \rightarrow \alpha \rightarrow \alpha$$

值得注意的是, 全称量化只是依赖类型的一个别名:

$$\text{'}\forall x : \sigma, \tau\text{' := '}(x : \sigma) \rightarrow \tau\text{'}$$

这一点在下文会变得更加清楚。

5.6 PAT 原则

→ 既被用作蕴涵符号, 也被用作函数的类型构造子。这两对概念不仅看起来相同, 而且根据 PAT 原则, 它们就是相同的:

- PAT = 命题即类型;
- PAT = 证明即项。

类型:

- $\sigma \rightarrow \tau$ 是从 σ 到 τ 的全函数类型。
- $(x : \sigma) \rightarrow \tau[x]$ 是从 $x : \sigma$ 到 $\tau[x]$ 的依赖函数类型。

命题:

- $P \rightarrow Q$ 可以读作“P 蕴涵 Q”, 也可以看作把 P 的证明映射到 Q 的证明确的函数类型。
- $\forall x : \sigma, Q[x]$ 可以读作“对所有 x, Q[x] 成立”, 也可以看作类型 $(x : \sigma) \rightarrow Q[x]$ 的函数类型, 即把类型 σ 的值 x 映射到 Q[x] 的证明。

项:

- 常量是项。
- 变量是项。
- $t \ u$ 是把函数 t 应用于值 u。
- $\text{fun } x \mapsto t[x]$ 是把 x 映射到 $t[x]$ 的函数。

证明:

- 定理名或假设名是证明。
- $H\ t$ 是一个证明，它用项 t 实例化证明 H 的陈述中最前面的参数或量词。
- $H\ G$ 是一个证明，它用证明 G 消解 H 的陈述中最前面的假设。
- $\text{fun } h : P \mapsto H[h]$ 是 $P \rightarrow Q$ 的证明，前提是对 $h : P$, $H[h]$ 是 Q 的证明。
- $\text{fun } x : \sigma \mapsto H[x]$ 是 $\forall x : \sigma, Q[x]$ 的证明，前提是对 $x : \sigma$, $H[x]$ 是 $Q[x]$ 的证明。

```
theorem And_swap_raw (a b : Prop) :
  a ∧ b → b ∧ a :=
  fun hab : a ∧ b ↦ And.intro (And.right hab) (And.left hab)
```

```
theorem And_swap_tactical (a b : Prop) :
  a ∧ b → b ∧ a :=
  by
    intro hab
    apply And.intro
    apply And.right
    exact hab
    apply And.left
    exact hab
```

策略式证明会被规约为证明项。

```
#print And_swap
#print And_swap_raw
#print And_swap_tactical
```

5.7 通过模式匹配和递归进行归纳

由 PAT 原则可知，归纳证明与递归指定的证明项是同一回事。因此，作为 `induction` 策略的替代，也可以通过模式匹配和递归来进行归纳：

- 此时归纳假设可以在我们正在证明的定理名称下获得；
- 参数的良基性通常会被自动证明。

```
#check reverse
```

```
theorem reverse_append {α : Type} :
```

```

  ∀xs ys : List α,
    reverse (xs ++ ys) = reverse ys ++ reverse xs
| [],    ys => by simp [reverse]
| x :: xs, ys => by simp [reverse, reverse_append xs]

```

```

theorem reverse_append_tactical {α : Type} (xs ys : List α) :
  reverse (xs ++ ys) = reverse ys ++ reverse xs :=
  by
    induction xs with
    | nil      => simp [reverse]
    | cons x xs' ih => simp [reverse, ih]

```

```

theorem reverse_reverse {α : Type} :
  ∀xs : List α, reverse (reverse xs) = xs
| []    => by rfl
| x :: xs =>
  by simp [reverse, reverse_append, reverse_reverse xs]

```

第六章 LoVe 演示 5：函数式程序设计

我们更仔细地考察带类型函数式程序设计的基础：归纳类型、归纳证明、递归函数、模式匹配、结构体（记录）以及类型类。

6.1 归纳类型

回忆类型 `Nat` 的定义：

```
#print Nat
```

准则：

- 无冗余值：该类型不包含任何不能用构造子表示的值。
- 无混淆：以不同方式构造的值彼此不同。

对于 `Nat`：

- “无冗余值”意味着不存在诸如 `-1` 或 `ε` 这样的特殊值，因为它们不能由 `Nat.zero` 和 `Nat.succ` 的有限组合表示。
- “无混淆”保证了 `Nat.zero` $\neq$ `Nat.succ` n 。

此外，归纳类型的值总是有限的。`Nat.succ (Nat.succ ...)` 不是一个值。

6.2 结构归纳

__ 结构归纳 __ 是数学归纳到归纳类型上的推广。要证明性质 `P[n]` 对所有自然数 `n` 成立，只需证明基例

``P[0]``

以及归纳步

$\text{`}\forall k, P[k] \rightarrow P[k + 1]\text{'}$

对于列表，基例是

$\text{`}P[[]]\text{'}$

归纳步是

$\text{`}\forall y \text{ } ys, P[ys] \rightarrow P[y :: ys]\text{'}$

一般而言，每个构造子对应一个子目标；对于我们正在进行归纳的类型的构造子参数，都会有可用的归纳假设。

```
theorem Nat.succ_neq_self (n : ℕ) :
  Nat.succ n ≠ n :=
  by
    induction n with
    | zero    => simp
    | succ n' ih =>
      intro hsucc
      apply ih
      apply Nat.succ.inj hsucc
```

6.3 结构递归

结构递归 是一种递归形式，它允许我们从递归所作用的值上剥去一个构造子。这类函数保证在递归停止之前只会调用自身有限多次。这是确立函数终止性的先决条件。

```
def fact : ℕ → ℕ
| 0    => 1
| n + 1 => (n + 1) * fact n

def factThreeCases : ℕ → ℕ
| 0    => 1
| 1    => 1
| n + 1 => (n + 1) * factThreeCases n
```

对结构递归函数，**Lean** 可以自动证明终止性。对于更一般的递归模式，终止性检查可能失败。有时它失败是有充分理由的，如下例所示：

```
-fails def illegal :  $\mathbb{N} \rightarrow \mathbb{N} \mid n \Rightarrow \text{illegal } n + 1$ 
```

```
opaque immoral :  $\mathbb{N} \rightarrow \mathbb{N}$ 
```

```
axiom immoral_eq (n :  $\mathbb{N}$ ) :  
  immoral n = immoral n + 1
```

```
theorem proof_of_False :  
  False :=  
  have hi : immoral 0 = immoral 0 + 1 :=  
    immoral_eq 0  
  have him :  
    immoral 0 - immoral 0 = immoral 0 + 1 - immoral 0 :=  
    by rw [←hi]  
  have h0eq1 : 0 = 1 :=  
    by simp at him  
  show False from  
    by simp at h0eq1
```

6.4 模式匹配表达式

```
`match` _term1_, ..., _termM_ `with`  
| _pattern11_, ..., _pattern1M_ => _result1_  
  ⋮  
| _patternN1_, ..., _patternNM_ => _resultN_
```

match 允许在项内部进行模式匹配。

```
def bcount { $\alpha$  : Type} (p :  $\alpha \rightarrow \text{Bool}$ ) : List  $\alpha \rightarrow \mathbb{N}$   
| [] => 0  
| x :: xs =>  
  match p x with  
  | true => bcount p xs + 1  
  | false => bcount p xs
```

```
def min (a b :  $\mathbb{N}$ ) :  $\mathbb{N}$  :=  
  if a  $\leq$  b then a else b
```


6.5 结构体

Lean 提供了方便的语法来定义记录，或称结构体。它们本质上是非递归的、单构造子的归纳类型。

```
structure RGB where
```

```
  red  :  $\mathbb{N}$ 
```

```
  green :  $\mathbb{N}$ 
```

```
  blue  :  $\mathbb{N}$ 
```

```
#check RGB.mk
```

```
#check RGB.red
```

```
#check RGB.green
```

```
#check RGB.blue
```

RGB 结构体定义等价于下列一组定义：

```
inductive RGB : Type where
```

```
  | mk :  $\mathbb{N} \rightarrow \mathbb{N} \rightarrow \mathbb{N} \rightarrow \text{RGB}$ 
```

```
def RGB.red : RGB  $\rightarrow \mathbb{N}$ 
```

```
  | RGB.mk r _ _ => r
```

```
def RGB.green : RGB  $\rightarrow \mathbb{N}$ 
```

```
  | RGB.mk _ g _ => g
```

```
def RGB.blue : RGB  $\rightarrow \mathbb{N}$ 
```

```
  | RGB.mk _ _ b => b
```

可以通过扩展已有结构体来创建新的结构体：

```
structure RGBA extends RGB where
```

```
  alpha :  $\mathbb{N}$ 
```

一个 RGBA 是带有额外字段 $\text{alpha} : \mathbb{N}$ 的 RGB。

```
#print RGBA
```

```

def pureRed : RGB :=
  RGB.mk 0xff 0x00 0x00

def pureGreen : RGB :=
  { red   := 0x00
    green := 0xff
    blue  := 0x00 }

def semitransparentGreen : RGBA :=
  { pureGreen with
    alpha := 0x7f }

#print pureRed
#print pureGreen
#print semitransparentGreen

def shuffle (c : RGB) : RGB :=
  { red   := RGB.green c
    green := RGB.blue c
    blue  := RGB.red c }

```

使用模式匹配的另一定义:

```

def shufflePattern : RGB → RGB
| RGB.mk r g b => RGB.mk g b r

theorem shuffle_shuffle_shuffle (c : RGB) :
  shuffle (shuffle (shuffle c)) = c :=
  by rfl

```

6.6 类型类

`__ 类型类 __` 是一种结构体类型, 它把抽象常量及其性质组合在一起。通过给出这些常量的具体定义, 并证明相应性质成立, 可以把一个类型声明为某个类型类的实例。**Lean** 会根据类型取回相关实例。

```
#print Inhabited
```

```

instance Nat.Inhabited : Inhabited  $\mathbb{N}$  :=
  { default := 0 }

instance List.Inhabited { $\alpha$  : Type} : Inhabited (List  $\alpha$ ) :=
  { default := [] }

#eval (Inhabited.default :  $\mathbb{N}$ )
#eval (Inhabited.default : List Int)

def head { $\alpha$  : Type} [Inhabited  $\alpha$ ] : List  $\alpha$   $\rightarrow$   $\alpha$ 
| []    => Inhabited.default
| x :: _ => x

theorem head_head { $\alpha$  : Type} [Inhabited  $\alpha$ ] (xs : List  $\alpha$ ) :
  head [head xs] = head xs :=
  by rfl

#eval head ([] : List  $\mathbb{N}$ )

#check List.head

instance Fun.Inhabited { $\alpha$   $\beta$  : Type} [Inhabited  $\beta$ ] :
  Inhabited ( $\alpha \rightarrow \beta$ ) :=
  { default := fun a :  $\alpha$   $\mapsto$  Inhabited.default }

instance Prod.Inhabited { $\alpha$   $\beta$  : Type}
  [Inhabited  $\alpha$ ] [Inhabited  $\beta$ ] :
  Inhabited ( $\alpha \times \beta$ ) :=
  { default := (Inhabited.default, Inhabited.default) }

```

我们在第 3 讲中遇到过这些类型类:

```

#print Std.Associative
#print Std.Commutative

```

6.7 列表

List 是一个多态归纳类型, 由 List.nil 和 List.cons 构造:

#print List

cases 对指定项进行分类讨论。它产生的子目标数等于该项类型定义中的构造子数。该策略的行为与 **induction** 相同，区别在于它不产生归纳假设。下面是一个刻意构造的例子：

```
theorem head_head_cases {α : Type} [Inhabited α]
  (xs : List α) :
  head [head xs] = head xs :=
by
  cases xs with
  | nil      => rfl
  | cons x xs' => rfl
```

match 是结构化的对应物：

```
theorem head_head_match {α : Type} [Inhabited α]
  (xs : List α) :
  head [head xs] = head xs :=
match xs with
| List.nil      => by rfl
| List.cons x xs' => by rfl
```

cases 也可用于形如 $l = r$ 的假设。它把 r 与 l 匹配，并在整个目标中，用 l 中的对应项替换 r 中出现的所有变量。

```
theorem injection_example {α : Type} (x y : α) (xs ys : List α)
  (h : x :: xs = y :: ys) :
  x = y ∧ xs = ys :=
by
  cases h
  simp
```

如果 r 无法与 l 匹配，则不会产生任何子目标；证明即告完成。

```
theorem distinctness_example {α : Type} (y : α) (ys : List α)
  (h : [] = y :: ys) :
  False :=
by cases h
```

```
def map {α β : Type} (f : α → β) : List α → List β
| []    => []
| x :: xs => f x :: map f xs
```

```
def mapArgs {α β : Type} : (α → β) → List α → List β
| _, []    => []
| f, x :: xs => f x :: mapArgs f xs
```

```
#check List.map
```

```
theorem map_ident {α : Type} (xs : List α) :
  map (fun x ↦ x) xs = xs :=
by
  induction xs with
  | nil      => rfl
  | cons x xs' ih => simp [map, ih]
```

```
theorem map_comp {α β γ : Type} (f : α → β) (g : β → γ)
  (xs : List α) :
  map g (map f xs) = map (fun x ↦ g (f x)) xs :=
by
  induction xs with
  | nil      => rfl
  | cons x xs' ih => simp [map, ih]
```

```
theorem map_append {α β : Type} (f : α → β)
  (xs ys : List α) :
  map f (xs ++ ys) = map f xs ++ map f ys :=
by
  induction xs with
  | nil      => rfl
  | cons x xs' ih => simp [map, ih]
```

```
def tail {α : Type} : List α → List α
| []    => []
| _ :: xs => xs
```

```
def headOpt {α : Type} : List α → Option α
```

```
| [] => Option.none
| x :: _ => Option.some x
```

```
def headPre {α : Type} : (xs : List α) → xs ≠ [] → α
| [], hxs => by simp at *
| x :: _, hxs => x
```

```
#eval headOpt [3, 1, 4]
#eval headPre [3, 1, 4] (by simp)
```

```
def zip {α β : Type} : List α → List β → List (α × β)
| x :: xs, y :: ys => (x, y) :: zip xs ys
| [], _ => []
| _ :: _, [] => []
```

```
#check List.zip
```

```
def length {α : Type} : List α → ℕ
| [] => 0
| x :: xs => length xs + 1
```

```
#check List.length
```

`cases` 也可以结合 `Classical.em` 对命题进行分类讨论。会出现两个情形：一个情形中该命题为真，另一个情形中该命题为假。

```
#check Classical.em
```

```
theorem min_add_add (l m n : ℕ) :
  min (m + l) (n + l) = min m n + l :=
  by
    cases Classical.em (m ≤ n) with
    | inl h => simp [min, h]
    | inr h => simp [min, h]
```

```
theorem min_add_add_match (l m n : ℕ) :
  min (m + l) (n + l) = min m n + l :=
  match Classical.em (m ≤ n) with
  | Or.inl h => by simp [min, h]
```

```
| Or.inr h => by simp [min, h]
```

```
theorem min_add_add_if (l m n : ℕ) :
  min (m + l) (n + l) = min m n + l :=
  if h : m ≤ n then
    by simp [min, h]
  else
    by simp [min, h]
```

```
theorem length_zip {α β : Type} (xs : List α) (ys : List β) :
  length (zip xs ys) = min (length xs) (length ys) :=
  by
    induction xs generalizing ys with
    | nil      => simp [zip, min, length]
    | cons x xs' ih =>
      cases ys with
      | nil      => rfl
      | cons y ys' => simp [zip, length, ih ys', min_add_add]
```

```
theorem map_zip {α α' β β' : Type} (f : α → α')
  (g : β → β') :
  ∀xs ys,
  map (fun ab : α × β ↦
    (f (Prod.fst ab), g (Prod.snd ab)))
    (zip xs ys) =
  zip (map f xs) (map g ys)
| x :: xs, y :: ys => by simp [zip, map, map_zip f g xs ys]
| [], _          => by rfl
| _ :: _, []     => by rfl
```

6.8 二叉树

若归纳类型的构造子接受若干递归参数，则它们定义树状对象。__ 二叉树 __ 的节点至多有两个子节点。

```
#print Tree
```

算术表达式类型 `AExp` 也是树形数据结构的一个例子。

树的节点, 无论是内部节点还是叶节点, 通常都携带标签或其他标注。

归纳树不包含无限分支, 甚至不包含环。这比基于指针或引用的数据结构 (在命令式语言中) 表达能力弱, 但更容易推理。

递归定义 (以及归纳证明) 大致类似于列表上的情形, 不过我们可能需要在若干子节点上递归 (或调用归纳假设)。

```
def mirror {α : Type} : Tree α → Tree α
| Tree.nil      => Tree.nil
| Tree.node a l r => Tree.node a (mirror r) (mirror l)
```

```
theorem mirror_mirror {α : Type} (t : Tree α) :
  mirror (mirror t) = t :=
by
  induction t with
  | nil          => rfl
  | node a l r ih_l ih_r => simp [mirror, ih_l, ih_r]
```

```
theorem mirror_mirror_calc {α : Type} :
  ∀t : Tree α, mirror (mirror t) = t
| Tree.nil      => by rfl
| Tree.node a l r =>
  calc
    mirror (mirror (Tree.node a l r))
  = mirror (Tree.node a (mirror r) (mirror l)) :=
    by rfl
  _ = Tree.node a (mirror (mirror l))
    (mirror (mirror r)) :=
    by rfl
  _ = Tree.node a l (mirror (mirror r)) :=
    by rw [mirror_mirror_calc l]
  _ = Tree.node a l r :=
    by rw [mirror_mirror_calc r]
```

```
theorem mirror_Eq_nil_iff {α : Type} :
  ∀t : Tree α, mirror t = Tree.nil ↔ t = Tree.nil
| Tree.nil      => by simp [mirror]
| Tree.node _ _ => by simp [mirror]
```


6.9 依赖归纳类型 (可选)

```

inductive Vec ( $\alpha$  : Type) :  $\mathbb{N} \rightarrow$  Type where
  | nil                : Vec  $\alpha$  0
  | cons (a :  $\alpha$ ) {n :  $\mathbb{N}$ } (v : Vec  $\alpha$  n) : Vec  $\alpha$  (n + 1)

#check Vec.nil
#check Vec.cons

def listOfVec { $\alpha$  : Type} :  $\forall$ {n :  $\mathbb{N}$ }, Vec  $\alpha$  n  $\rightarrow$  List  $\alpha$ 
  | _, Vec.nil      => []
  | _, Vec.cons a v => a :: listOfVec v

def vecOfList { $\alpha$  : Type} :
   $\forall$ xs : List  $\alpha$ , Vec  $\alpha$  (List.length xs)
  | []      => Vec.nil
  | x :: xs => Vec.cons x (vecOfList xs)

theorem length_listOfVec { $\alpha$  : Type} :
   $\forall$ (n :  $\mathbb{N}$ ) (v : Vec  $\alpha$  n), List.length (listOfVec v) = n
  | _, Vec.nil      => by rfl
  | _, Vec.cons a v =>
    by simp [listOfVec, length_listOfVec _ v]

```

第七章 LoVe 演示 6: 归纳谓词

归纳谓词，也就是归纳定义的命题，是规定类型为 $\dots \rightarrow \text{Prop}$ 的函数的一种便利方式。它们让人想起形式系统，也让人想起 Prolog 中的 Horn 子句；Prolog 是逻辑程序设计语言的典范。

Lean 的一种可能图景是：

Lean = 函数式程序设计 + 逻辑程序设计 + 更多逻辑

7.1 引导性例子

7.1.1 偶数

数学家常常把集合定义为满足某些条件的最小集合。例如：

偶自然数集合 E 定义为在下列规则下封闭的最小集合：

(1) $0 \in E$ ；(2) 对每个 $k \in \mathbb{N}$ ，若 $k \in E$ ，则 $k + 2 \in E$ 。

在 Lean 中，我们可以如下定义相应的“是偶数”谓词：

```
inductive Even :  $\mathbb{N} \rightarrow \text{Prop}$  where
  | zero    : Even 0
  | add_two :  $\forall k : \mathbb{N}, \text{Even } k \rightarrow \text{Even } (k + 2)$ 
```

这应当看起来很熟悉。我们曾经使用相同的语法定义归纳类型，只是那里用的是 `Type` 而不是 `Prop`。

上面的命令引入了一个新的 unary 谓词 `Even`，以及两个构造子 `Even.zero` 和 `Even.add_two`，它们可用于构造证明项。由于归纳定义具有“无冗余值”保证，`Even.zero` 和 `Even.add_two` 是构造 `Even` 的证明的仅有两种方式。

根据 PAT 原则，`Even` 可以看作一个归纳类型，其值就是证明项。

```

theorem Even_4 :
  Even 4 :=
  have Even_0 : Even 0 :=
    Even.zero
  have Even_2 : Even 2 :=
    Even.add_two _ Even_0
  show Even 4 from
    Even.add_two _ Even_2

```

为什么不能直接递归地定义 **Even**? 确实, 为什么不这样做呢?

```

def evenRec :  $\mathbb{N} \rightarrow \text{Bool}$ 
| 0    => true
| 1    => false
| k + 2 => evenRec k

```

两种风格各有优缺点。

递归版本要求我们指定一个为假的情形 (1), 并且要求我们关心终止性。另一方面, 由于它具有计算内容, 它能很好地配合 `rfl`、`simp`、`#reduce` 和 `#eval`。

归纳版本通常被认为更加抽象、更加优雅。每条规则都可以独立于其他规则陈述。

定义 **Even** 的另一种方式, 是把它定义为一个非递归定义:

```

def evenNonrec (k :  $\mathbb{N}$ ) : Prop :=
  k % 2 = 0

```

数学家大概会认为这是最令人满意的定义。但归纳版本是一个便利而直观的例子, 也很典型地代表了许多现实的归纳定义。

7.1.2 网球比赛

迁移系统由迁移规则组成; 这些规则共同规定一个二元谓词, 连接“之前”状态和“之后”状态。作为迁移系统的一个简单样本, 我们考虑一局网球比赛中从 **0-0** 开始可能发生的比分迁移。

```

inductive Score : Type where
| vs      :  $\mathbb{N} \rightarrow \mathbb{N} \rightarrow \text{Score}$ 
| advServ : Score

```

```
| advRecv : Score
| gameServ : Score
| gameRecv : Score
```

```
infixr:50 " - " => Score.vs
```

```
inductive Step : Score → Score → Prop where
| serv_0_15   : ∀n, Step (0-n) (15-n)
| serv_15_30  : ∀n, Step (15-n) (30-n)
| serv_30_40  : ∀n, Step (30-n) (40-n)
| serv_40_game : ∀n, n < 40 → Step (40-n) Score.gameServ
| serv_40_adv  : Step (40-40) Score.advServ
| serv_adv_40  : Step Score.advServ (40-40)
| serv_adv_game : Step Score.advServ Score.gameServ
| recv_0_15   : ∀n, Step (n-0) (n-15)
| recv_15_30  : ∀n, Step (n-15) (n-30)
| recv_30_40  : ∀n, Step (n-30) (n-40)
| recv_40_game : ∀n, n < 40 → Step (n-40) Score.gameRecv
| recv_40_adv  : Step (40-40) Score.advRecv
| recv_adv_40  : Step Score.advRecv (40-40)
| recv_adv_game : Step Score.advRecv Score.gameRecv
```

```
infixr:45 " ~ " => Step
```

注意, 虽然 `Score.vs` 允许任意自然数作为参数, 但 `Step` 构造子的表述保证了, 从 `0-0` 出发只能到达合法的网球比分。

我们可以提出并形式化回答这样的问题: 比分是否可能回到 `0-0`?

```
theorem no_Step_to_0_0 (s : Score) :
  ¬ s ~ 0-0 :=
by
  intro h
  cases h
```

7.1.3 自反传递闭包

最后一个引导性例子是关系 `R` 的自反传递闭包, 我们把它建模为一个二元谓词 `Star R`。

```
inductive Star {α : Type} (R : α → α → Prop) : α → α → Prop
where
| base (a b : α) : R a b → Star R a b
| refl (a : α) : Star R a a
| trans (a b c : α) : Star R a b → Star R b c → Star R a c
```

第一条规则把 R 嵌入 $\text{Star } R$ 。第二条规则实现自反闭包。第三条规则实现传递闭包。

这个定义确实优雅。如果对此有所怀疑，不妨尝试把 Star 实现为递归函数：

```
def starRec {α : Type} (R : α → α → Bool) :
  α → α → Bool :=
  sorry
```

7.1.4 一个非例子

并非所有归纳定义都是合法的。

```
-fails inductive Illegal : Prop where | intro : ¬ Illegal → Illegal
```

7.2 逻辑符号

真值 False 和 True 、联结词 $\wedge$ 、 $\vee$ 和 $\leftrightarrow$ 、存在量词 $\exists$ ，以及等式谓词 $=$ ，全都定义为归纳命题或归纳谓词。相比之下， $\forall$ 和 $\rightarrow$ 内建于逻辑之中。

语法糖：

```
`∃x : α, P` := `Exists (fun x : α ↦ P)`
`x = y` := `Eq x y`
```

```
inductive And (a b : Prop) : Prop where
| intro : a → b → And a b
```

```
inductive Or (a b : Prop) : Prop where
| inl : a → Or a b
| inr : b → Or a b
```

```
inductive Iff (a b : Prop) : Prop where
```

```
| intro : (a → b) → (b → a) → Iff a b
```

```
inductive Exists {α : Type} (P : α → Prop) : Prop where
```

```
| intro : ∀ a : α, P a → Exists P
```

```
inductive True : Prop where
```

```
| intro : True
```

```
inductive False : Prop where
```

```
inductive Eq {α : Type} : α → α → Prop where
```

```
| refl : ∀ a : α, Eq a a
```

```
#print And
```

```
#print Or
```

```
#print Iff
```

```
#print Exists
```

```
#print True
```

```
#print False
```

```
#print Eq
```

7.3 规则归纳

正如我们可以对一个项进行归纳，我们也可以对一个证明项进行归纳。

这称为 __ 规则归纳 __，因为归纳作用在引入规则上（即证明项的构造子上）。由于 **PAT** 原则，这一点按预期工作。

```
theorem mod_two_Eq_zero_of_Even (n : ℕ) (h : Even n) :
```

```
  n % 2 = 0 :=
```

```
by
```

```
  induction h with
```

```
  | zero      => rfl
```

```
  | add_two k hk ih => simp [ih]
```

```
theorem Not_Even_two_mul_add_one (m n : ℕ)
```

```
  (hm : m = 2 * n + 1) :
```

```

  ¬ Even m :=
by
  intro h
  induction h generalizing n with
  | zero      => linarith
  | add_two k hk ih =>
    apply ih (n - 1)
    cases n with
    | zero    => simp at *
    | succ n' =>
      simp at *
      linarith

```

`linarith` 证明涉及线性算术等式或不等式的目标。“线性”意味着它只处理 `+` 和 `-`，不处理 `*` 和 `/` (但支持乘以常数)。

```

theorem linarith_example (i : Int) (hi : i > 5) :
  2 * i + 3 > 11 :=
by linarith

```

```

theorem Star_Star_Iff_Star {α : Type} (R : α → α → Prop)
  (a b : α) :
  Star (Star R) a b ↔ Star R a b :=
by
  apply Iff.intro
  · intro h
    induction h with
    | base a b hab      => exact hab
    | refl a            => apply Star.refl
    | trans a b c hab hbc ihab ihbc =>
      apply Star.trans a b
      · exact ihab
      · exact ihbc
  · intro h
    apply Star.base
    exact h

```

```

@[simp] theorem Star_Star_Eq_Star {α : Type}
  (R : α → α → Prop) :

```

```

Star (Star R) = Star R :=
by
  apply funext
  intro a
  apply funext
  intro b
  apply propext
  apply Star_Star_Iff_Star

#check funext
#check propext

```

7.4 消去

给定一个归纳谓词 P ，它的引入规则通常形如 $\forall \dots, \dots \rightarrow P \dots$ ，并可用于证明形如 $\vdash P \dots$ 的目标。

消去则反向工作：它从形如 $P \dots$ 的定理或假设中抽取信息。消去有多种形式：模式匹配、`cases` 和 `induction` 策略，以及自定义消去规则（例如 `And.left`）。

- `cases` 的工作方式类似于 `induction`，但没有归纳假设。
- `match` 同样可用。

现在我们终于能够理解 $h : l = r$ 时的 `cases h`，以及 `cases Classical.em h` 是如何工作的。

```

#print Eq

theorem cases_Eq_example {α : Type} (l r : α) (h : l = r)
  (P : α → α → Prop) :
  P l r :=
by
  cases h
  sorry

#check Classical.em
#print Or

theorem cases_Classical_em_example {α : Type} (a : α)
  (P Q : α → Prop) :
  Q a :=

```



```

by
  have hor : P a v  $\rightarrow$  P a :=
    Classical.em (P a)
  cases hor with
  | inl hl => sorry
  | inr hr => sorry

```

我们常常希望重写形如 $P(c \dots)$ 的具体项, 其中 c 通常是一个构造子。为了支持这种消去式推理, 可以陈述并证明一个 __ 反演规则 __。

典型的反演规则:

$$\text{`}\forall x y, P(c x y) \rightarrow (\exists \dots, \dots \wedge \dots) v \dots v (\exists \dots, \dots \wedge \dots)\text{'}$$

把引入和消去合并到一个定理中也可能有用; 这样得到的定理可用于同时重写目标中的假设和结论:

$$\text{`}\forall x y, P(c x y) \leftrightarrow (\exists \dots, \dots \wedge \dots) v \dots v (\exists \dots, \dots \wedge \dots)\text{'}$$

```

theorem Even_Iff (n :  $\mathbb{N}$ ) :
  Even n  $\leftrightarrow$  n = 0  $\vee$  ( $\exists m : \mathbb{N}, n = m + 2 \wedge \text{Even } m$ ) :=
by
  apply Iff.intro
  · intro hn
    cases hn with
    | zero      => simp
    | add_two k hk =>
      apply Or.inr
      apply Exists.intro k
      simp [hk]
  · intro hor
    cases hor with
    | inl heq => simp [heq, Even.zero]
    | inr hex =>
      cases hex with
      | intro k hand =>
        cases hand with
        | intro heq hk =>
          simp [heq, Even.add_two _ hk]

```

```

theorem Even_Iff_struct (n :  $\mathbb{N}$ ) :

```

```

Even n ↔ n = 0 ∨ (∃m : ℕ, n = m + 2 ∧ Even m) :=
|ff.intro
  (assume hn : Even n
   match hn with
   | Even.zero      =>
     show 0 = 0 ∨ _ from
       by simp
   | Even.add_two k hk =>
     show _ ∨ (∃m, k + 2 = m + 2 ∧ Even m) from
       Or.inr (Exists.intro k (by simp [*])))
  (assume hor : n = 0 ∨ (∃m, n = m + 2 ∧ Even m)
   match hor with
   | Or.inl heq =>
     show Even n from
       by simp [heq, Even.zero]
   | Or.inr hex =>
     match hex with
     | Exists.intro m hand =>
       match hand with
       | And.intro heq hm =>
         show Even n from
           by simp [heq, Even.add_two _ hm])

```

7.5 更多例子

7.5.1 有序列表

```

inductive Sorted : List ℕ → Prop where
  | nil : Sorted []
  | single (x : ℕ) : Sorted [x]
  | two_or_more (x y : ℕ) {zs : List ℕ} (hle : x ≤ y)
    (hsorted : Sorted (y :: zs)) :
    Sorted (x :: y :: zs)

theorem Sorted_nil :
  Sorted [] :=
  Sorted.nil

```

```

theorem Sorted_2 :
  Sorted [2] :=
  Sorted.single _

theorem Sorted_3_5 :
  Sorted [3, 5] :=
  by
    apply Sorted.two_or_more
    · simp
    · exact Sorted.single _

theorem Sorted_3_5_raw :
  Sorted [3, 5] :=
  Sorted.two_or_more _ _ (by simp) (Sorted.single _)

theorem sorted_7_9_9_11 :
  Sorted [7, 9, 9, 11] :=
  Sorted.two_or_more _ _ (by simp)
  (Sorted.two_or_more _ _ (by simp)
    (Sorted.two_or_more _ _ (by simp)
      (Sorted.single _)))

theorem Not_Sorted_17_13 :
  ¬ Sorted [17, 13] :=
  by
    intro h
    cases h with
    | two_or_more _ _ hlet hsorted => simp at hlet

```

7.5.2 回文

```

inductive Palindrome {α : Type} : List α → Prop where
  | nil : Palindrome []
  | single (x : α) : Palindrome [x]
  | sandwich (x : α) (xs : List α) (hxs : Palindrome xs) :
    Palindrome ([x] ++ xs ++ [x])

```

```
-fails def palindromeRec {α : Type} : List α → Bool | [] => true | [_] => true | ([x] ++ xs +
+ [x]) => palindromeRec xs | _ => false
```

```
theorem Palindrome_aa {α : Type} (a : α) :
  Palindrome [a, a] :=
  Palindrome.sandwich a _ Palindrome.nil
```

```
theorem Palindrome_aba {α : Type} (a b : α) :
  Palindrome [a, b, a] :=
  Palindrome.sandwich a _ (Palindrome.single b)
```

```
theorem Palindrome_reverse {α : Type} (xs : List α)
  (hxs : Palindrome xs) :
  Palindrome (reverse xs) :=
  by
    induction hxs with
    | nil          => exact Palindrome.nil
    | single x     => exact Palindrome.single x
    | sandwich x xs hxs ih =>
      · simp [reverse, reverse_append]
        exact Palindrome.sandwich _ _ ih
```

7.5.3 满二叉树

```
#check Tree
```

```
inductive IsFull {α : Type} : Tree α → Prop where
  | nil : IsFull Tree.nil
  | node (a : α) (l r : Tree α)
    (hl : IsFull l) (hr : IsFull r)
    (hiff : l = Tree.nil ↔ r = Tree.nil) :
    IsFull (Tree.node a l r)
```

```
theorem IsFull_singleton {α : Type} (a : α) :
  IsFull (Tree.node a Tree.nil Tree.nil) :=
  by
    apply IsFull.node
```

- exact IsFull.nil
- exact IsFull.nil
- rfl

```

theorem IsFull_mirror {α : Type} (t : Tree α)
  (ht : IsFull t) :
  IsFull (mirror t) :=
by
  induction ht with
  | nil          => exact IsFull.nil
  | node a l r hl hr hiff ih_l ih_r =>
    • rw [mirror]
      apply IsFull.node
    • exact ih_r
    • exact ih_l
    • simp [mirror_Eq_nil_Iff, *]
```

```

theorem IsFull_mirror_struct_induct {α : Type} (t : Tree α) :
  IsFull t → IsFull (mirror t) :=
by
  induction t with
  | nil          =>
    • intro ht
      exact ht
  | node a l r ih_l ih_r =>
    • intro ht
      cases ht with
      | node _ _ _ hl hr hiff =>
        • rw [mirror]
          apply IsFull.node
        • exact ih_r hr
        • apply ih_l hl
        • simp [mirror_Eq_nil_Iff, *]
```

7.5.4 一阶项

```

inductive Term (α β : Type) : Type where
  | var : β → Term α β
```

| fn : $\alpha \rightarrow \text{List } (\text{Term } \alpha \beta) \rightarrow \text{Term } \alpha \beta$

inductive WellFormed { $\alpha \beta : \text{Type}$ } (arity : $\alpha \rightarrow \mathbb{N}$) :

Term $\alpha \beta \rightarrow \text{Prop}$ where

| var ($x : \beta$) : WellFormed arity (Term.var x)

| fn ($f : \alpha$) (ts : List (Term $\alpha \beta$))

(hargs : $\forall t \in \text{ts}, \text{WellFormed } \text{arity } t$)

(hlen : length ts = arity f) :

WellFormed arity (Term.fn f ts)

inductive VariableFree { $\alpha \beta : \text{Type}$ } : Term $\alpha \beta \rightarrow \text{Prop}$ where

| fn ($f : \alpha$) (ts : List (Term $\alpha \beta$))

(hargs : $\forall t \in \text{ts}, \text{VariableFree } t$) :

VariableFree (Term.fn f ts)

第八章 LoVe 演示 7：带效应程序设计

Monad 是函数式程序设计中的一个重要抽象。它们概括了带副作用的计算，从而在纯函数式程序设计语言中提供带效应程序设计。**Haskell** 表明，**monad** 可以非常成功地用于编写命令式程序。对我们而言，**monad** 本身就很有趣，此外还有两个原因：

- 它们提供了公理化推理的一个很好例子。
- 它们是对 **Lean** 本身进行编程所必需的（元编程，第 8 讲）。

8.1 引导性例子

考虑如下程序设计任务：

实现一个函数 ``sum257 ns``，它把自然数列表 ``ns`` 中第二个、第五个和第七个元素相加。结果使用 ``Option ℕ``，这样当列表少于七个元素时，可以返回 ``Option.none``。

一个直接的解法如下：

```
def nth {α : Type} : List α → Nat → Option α
| [], _    => Option.none
| x :: _, 0 => Option.some x
| _ :: xs, n + 1 => nth xs n
```

```
def sum257 (ns : List ℕ) : Option ℕ :=
  match nth ns 1 with
  | Option.none    => Option.none
  | Option.some n₂ =>
    match nth ns 4 with
    | Option.none    => Option.none
    | Option.some n₅ =>
      match nth ns 6 with
```

```
| Option.none => Option.none
| Option.some n7 => Option.some (n2 + n5 + n7)
```

由于所有这些关于 `option` 的模式匹配, 代码显得很丑。

我们可以把所有丑陋之处放进一个函数中, 并称它为 `connect`:

```
def connect {α : Type} {β : Type} :
  Option α → (α → Option β) → Option β
| Option.none, _ => Option.none
| Option.some a, f => f a

def sum257Connect (ns : List ℕ) : Option ℕ :=
  connect (nth ns 1)
    (fun n2 ↦ connect (nth ns 4)
      (fun n5 ↦ connect (nth ns 6)
        (fun n7 ↦ Option.some (n2 + n5 + n7))))
```

与其自行定义 `connect`, 我们可以使用 Lean 预定义的一般 `bind` 运算。也可以使用 `pure` 取代 `Option.some`:

```
#check bind
```

```
def sum257Bind (ns : List ℕ) : Option ℕ :=
  bind (nth ns 1)
    (fun n2 ↦ bind (nth ns 4)
      (fun n5 ↦ bind (nth ns 6)
        (fun n7 ↦ pure (n2 + n5 + n7))))
```

通过使用 `bind` 和 `pure`, `sum257Bind` 不再引用构造子 `Option.none` 和 `Option.some`。

语法糖:

```
`ma >>= f` := `bind ma f`
```

```
def sum257Op (ns : List ℕ) : Option ℕ :=
  nth ns 1 >>=
    fun n2 ↦ nth ns 4 >>=
      fun n5 ↦ nth ns 6 >>=
        fun n7 ↦ pure (n2 + n5 + n7)
```


语法糖:

```
do
  let a ← ma
  t

:= ma »= (fun a ↦ t)
```

```
do
  ma
  t

:= ma »= (fun _ ↦ t)
```

```
def sum257Dos (ns : List ℕ) : Option ℕ :=
do
  let n2 ← nth ns 1
do
  let n5 ← nth ns 4
do
  let n7 ← nth ns 6
  pure (n2 + n5 + n7)
```

这些 `do` 可以合并:

```
def sum257Do (ns : List ℕ) : Option ℕ :=
do
  let n2 ← nth ns 1
  let n5 ← nth ns 4
  let n7 ← nth ns 6
  pure (n2 + n5 + n7)
```

尽管这种记法带有命令式风味, 该函数仍然是一个纯函数式程序。

8.2 两个运算和三条定律

`Option` 类型构造子是 `monad` 的一个例子。

一般而言, `monad` 是一个类型构造子 `m`, 它依赖于某个类型参数 α (即 `m  $\alpha$`), 并且配备两个特别的运算:

```
`pure {α : Type} : α → m α`
`bind {α β : Type} : m α → (α → m β) → m β`
```

对于 Option:

```
`pure` := `Option.some`
`bind` := `connect`
```

直观地说, 可以把 monad 理解为一个“盒子”:

- pure 把数据放入盒子。
- bind 允许我们访问盒子中的数据并修改它（甚至可能改变其类型, 因为结果是一个 $m \beta$ monad, 而不是 $m \alpha$ monad）。

不存在一般方法可以从 monad 中取出数据, 也就是说, 无法一般地从 $m \alpha$ 得到 α 。

概括地说, pure a 不提供副作用, 只提供一个包含值 a 的盒子; 而 bind ma f (也写作 $ma \gg= f$) 执行 ma, 然后以 ma 的盒中结果 a 执行 f。

option monad 只是众多实例中的一个。

类型	效应
id	无效应
Option	简单异常
$\text{fun } \alpha \mapsto \sigma \rightarrow \alpha \times \sigma$	传递一个类型为 σ 的状态
Set	返回 α 值的非确定性计算
$\text{fun } \alpha \mapsto t \rightarrow \alpha$	读取类型为 t 的元素（例如配置）
$\text{fun } \alpha \mapsto \mathbb{N} \times \alpha$	附加运行时间（例如用于建模时间复杂度）
$\text{fun } \alpha \mapsto \text{String} \times \alpha$	附加文本输出（例如用于日志）
IO	与操作系统交互
TacticM	与证明助理交互

以上所有都是一元类型构造子 $m : \text{Type} \rightarrow \text{Type}$ 。

某些效应可以组合（例如 $\text{Option } (t \rightarrow \alpha)$ ）。

某些效应不可执行（例如 $\text{Set } \alpha$ ）。尽管如此, 它们对于在逻辑中抽象地建模程序仍然有用。

特定的 monad 可能提供某种方式, 在不满足 bind 要求的“取出后再放回 monad”条件下, 提取储存在 monad 中的盒中值。

monad 有若干好处, 包括:

- 它们提供方便且高度可读的 do 记法。

- 它们支持泛型运算, 例如 $\text{mmap } \{\alpha \beta : \text{Type}\} : (\alpha \rightarrow m \beta) \rightarrow \text{List } \alpha \rightarrow m (\text{List } \beta)$, 这些运算在所有 `monad` 上以统一方式工作。

`bind` 和 `pure` 运算通常要求满足三条定律。作为第一个程序的纯数据可以被化简掉:

```
do
  let a' ← pure a,
  f a'
= f a
```

作为第二个程序的纯数据可以被化简掉:

```
do
  let a ← ma
  pure a
= ma
```

嵌套程序 `ma`、`f`、`g` 可以用下面这条结合律展平:

```
do
  let b ←
    do
      let a ← ma
      f a
  g b
= do let a ← ma let b ← f a g b
```

8.3 Monad 的类型类

`Monad` 是一种数学结构, 因此我们使用类把它们加入为类型类。可以把类型类看成以类型为参数的结构; 在这里, 它以类型构造子 $m : \text{Type} \rightarrow \text{Type}$ 为参数。

```
class LawfulMonad (m : Type → Type)
  extends Pure m, Bind m where
  pure_bind {α β : Type} (a : α) (f : α → m β) :
    (pure a >>= f) = f a
  bind_pure {α : Type} (ma : m α) :
    (ma >>= pure) = ma
  bind_assoc {α β γ : Type} (f : α → m β) (g : β → m γ)
```

```
(ma : m α) :
((ma >>= f) >>= g) = (ma >>= (fun a ↦ f a >>= g))
```

逐步说明:

- 我们正在创建一个以一元类型构造子 m 为参数的结构。
- 该结构继承名为 **Bind** 和 **Pure** 的结构中的字段以及所有语法糖; 这些结构为 m 提供 **bind** 和 **pure** 运算以及一些语法糖。
- 该定义在 **Bind** 和 **Pure** 已经提供的字段之外增加三个字段, 用于存放这些定律的证明。

要用一个具体 **monad** 实例化这个定义, 我们必须提供类型构造子 m (例如 **Option**)、**bind** 和 **pure** 运算符, 以及这些定律的证明。

8.4 无效应

我们的第一个 **monad** 是平凡 **monad** $m := \text{id}$ (即 $m := (\text{fun } \alpha \mapsto \alpha)$)。

```
def id.pure {α : Type} : α → id α
| a => a
```

```
def id.bind {α β : Type} : id α → (α → id β) → id β
| a, f => f a
```

```
instance id.LawfulMonad : LawfulMonad id :=
{ pure      := id.pure
  bind      := id.bind
  pure_bind :=
    by
      intro α β a f
      rfl
  bind_pure :=
    by
      intro α ma
      rfl
  bind_assoc :=
    by
      intro α β γ f g ma
      rfl }
```

8.5 基本异常

如上所见, `option` 类型提供了一种基本异常机制。

```
def Option.pure {α : Type} : α → Option α :=
  Option.some

def Option.bind {α β : Type} :
  Option α → (α → Option β) → Option β
| Option.none, _ => Option.none
| Option.some a, f => f a

instance Option.LawfulMonad : LawfulMonad Option :=
{ pure      := Option.pure
  bind      := Option.bind
  pure_bind :=
    by
      intro α β a f
      rfl
  bind_pure :=
    by
      intro α ma
      cases ma with
      | none  => rfl
      | some _ => rfl
  bind_assoc :=
    by
      intro α β γ f g ma
      cases ma with
      | none  => rfl
      | some _ => rfl }

def Option.throw {α : Type} : Option α :=
  Option.none

def Option.catch {α : Type} : Option α → Option α → Option α
| Option.none, ma' => ma'
| Option.some a, _ => Option.some a
```

8.6 可变状态

状态 `monad` 提供了一个对应于可变状态的抽象。某些编译器能够识别状态 `monad`，从而生成高效的命令式代码。

```
def Action (σ α : Type) : Type :=
  σ → α × σ

def Action.read {σ : Type} : Action σ σ
| s => (s, s)

def Action.write {σ : Type} (s : σ) : Action σ Unit
| _ => ((), s)

def Action.pure {σ α : Type} (a : α) : Action σ α
| s => (a, s)

def Action.bind {σ : Type} {α β : Type} (ma : Action σ α)
  (f : α → Action σ β) :
  Action σ β
| s =>
  match ma s with
  | (a, s') => f a s'
```

`Action.pure` 类似于 `return` 语句；它不改变状态。

`Action.bind` 类似于两个语句关于状态的顺序组合。

```
instance Action.LawfulMonad {σ : Type} :
  LawfulMonad (Action σ) :=
  { pure      := Action.pure
    bind      := Action.bind
    pure_bind :=
      by
        intro α β a f
        rfl
    bind_pure :=
      by
        intro α ma
```

```

    rfl
  bind_assoc :=
  by
    intro  $\alpha$   $\beta$   $\gamma$   $f$   $g$   $ma$ 
    rfl }

def increasingly : List  $\mathbb{N}$   $\rightarrow$  Action  $\mathbb{N}$  (List  $\mathbb{N}$ )
| []      => pure []
| (n :: ns) =>
  do
    let prev  $\leftarrow$  Action.read
    if n < prev then
      increasingly ns
    else
      do
        Action.write n
        let ns'  $\leftarrow$  increasingly ns
        pure (n :: ns')

#eval increasingly [1, 2, 3, 2] 0
#eval increasingly [1, 2, 3, 2, 4, 5, 2] 0

```

8.7 非确定性

集合 `monad` 存储任意数量、可能无限多个 α 值。

```

#check Set

def Set.pure { $\alpha$  : Type} :  $\alpha \rightarrow$  Set  $\alpha$ 
| a => {a}

def Set.bind { $\alpha$   $\beta$  : Type} : Set  $\alpha \rightarrow$  ( $\alpha \rightarrow$  Set  $\beta$ )  $\rightarrow$  Set  $\beta$ 
| A, f => {b |  $\exists a, a \in A \wedge b \in f a$ }

instance Set.LawfulMonad : LawfulMonad Set :=
{ pure      := Set.pure
  bind      := Set.bind
  pure_bind :=

```

```

by
  intro  $\alpha$   $\beta$  a f
  simp [Set.pure, Set.bind]
bind_pure :=
by
  intro  $\alpha$  ma
  simp [Set.pure, Set.bind]
bind_assoc :=
by
  intro  $\alpha$   $\beta$   $\gamma$  f g ma
  simp [Set.bind]
  aesop }

```

aesop 是一种通用证明搜索策略。除其他工作外，它会对假设中的逻辑符号 $\wedge$ 、 $\vee$ 、 $\leftrightarrow$ 和 $\exists$ 进行消去，并在目标中引入 $\wedge$ 、 $\leftrightarrow$ 和 $\exists$ ；它还会经常调用化简器。它可能成功证明目标，也可能失败，或者部分成功，把若干未完成的子目标留给用户。

aesop 的一个替代是 **grind**；它受可满足性模理论求解器启发。它使用若干推理引擎协同工作，以支持涉及等式推理、分类讨论以及线性算术等内容目标。不同于 **simp**，**grind** 以无方向的方式（即不是从左到右）推理等式。它系统地从目标中已有的等式推出新等式。例如，如果目标包含假设 $b = a$ 和 $f\ b \neq f\ a$ ，则 **grind** 会从 $b = a$ 推出 $f\ b = f\ a$ ，并发现它与 $f\ b \neq f\ a$ 矛盾。

8.8 一个泛型算法：遍历列表

我们考虑一个泛型带效应程序 **mmap**，它遍历一个列表并把函数 **f** 应用于每个元素。

```

def nthFine { $\alpha$  : Type} (xss : List (List  $\alpha$ )) (n :  $\mathbb{N}$ ) :
  List (Option  $\alpha$ ) :=
  List.map (fun xs  $\mapsto$  nth xs n) xss

#eval nthFine [[11, 12, 13, 14], [21, 22, 23]] 2
#eval nthFine [[11, 12, 13, 14], [21, 22, 23]] 3

def mmap {m : Type  $\rightarrow$  Type} [LawfulMonad m] { $\alpha$   $\beta$  : Type}
  (f :  $\alpha \rightarrow m\ \beta$ ) :
  List  $\alpha \rightarrow m$  (List  $\beta$ )
| []    => pure []
| a :: as =>
  do

```



```

let b ← f a
let bs ← mmap f as
pure (b :: bs)

```

```

def nthCoarse {α : Type} (xss : List (List α)) (n : ℕ) :
  Option (List α) :=
  mmap (fun xs ↦ nth xs n) xss

```

```

#eval nthCoarse [[11, 12, 13, 14], [21, 22, 23]] 2
#eval nthCoarse [[11, 12, 13, 14], [21, 22, 23]] 3

```

```

theorem mmap_append {m : Type → Type} [LawfulMonad m]
  {α β : Type} (f : α → m β) :
  ∀ as as' : List α, mmap f (as ++ as') =
  do
    let bs ← mmap f as
    let bs' ← mmap f as'
    pure (bs ++ bs')
| [], _ =>
  by simp [mmap, LawfulMonad.bind_pure, LawfulMonad.pure_bind]
| a :: as, as' =>
  by simp [mmap, mmap_append _ as as', LawfulMonad.pure_bind,
    LawfulMonad.bind_assoc]

```

第九章 LoVe 演示 8：元编程

用户可以用自定义策略和工具扩展 **Lean**。这类程序设计，即对证明器本身进行编程，称为元编程。

Lean 的元编程框架大体上使用与 **Lean** 输入语言本身相同的概念和语法。抽象语法树 __ 反映 __ 内部数据结构，例如表达式（项）的内部结构。证明器的内部机制通过 **Lean** 接口暴露出来，我们可以用这些接口来

- 访问当前语境和目标；
- 合一表达式；
- 查询并修改环境；
- 设置属性。

Lean 本身的大部分都是用 **Lean** 实现的。

应用示例：

- 证明目标变换；
- 启发式证明搜索；
- 判定过程；
- 定义生成器；
- 建议工具；
- 导出器。

Lean 元编程框架的优点：

- 用户不需要学习另一门程序设计语言来编写元程序；他们可以使用与定义证明器库中普通对象相同的构造和记法。
- 该库中的一切都可用于元编程目的。
- 元程序可以在同一个交互式环境中编写和调试，这鼓励一种同时开发形式化库与支持性自动化的风格。

open Lean

```
open Lean.Meta
open Lean.Elab.Tactic
open Lean.TSyntax
```

9.1 策略组合子

在编写自己的策略时，我们常常需要在若干目标上重复某些动作，或者在某个策略失败时恢复。策略组合子在这类情形中很有帮助。

`repeat'` 在所有（子……子）目标上反复应用其参数，直到不能再继续应用为止。

```
theorem repeat'_example :
  Even 4 ∧ Even 7 ∧ Even 3 ∧ Even 0 :=
  by
    repeat' apply And.intro
    repeat' apply Even.add_two
    repeat' sorry
```

“first” 组合子 `first | … | … | …` 先尝试第一个参数。若失败，则应用第二个参数。若仍失败，则应用第三个参数，依此类推。

```
theorem repeat'_first_example :
  Even 4 ∧ Even 7 ∧ Even 3 ∧ Even 0 :=
  by
    repeat' apply And.intro
    repeat'
      first
      | apply Even.add_two
      | apply Even.zero
    repeat' sorry
```

`all_goals` 将其参数恰好应用一次到每个目标上。只有当该参数在所有目标上都成功时，它才成功。

```
theorem all_goals_example : Even 4 ∧ Even 7 ∧ Even 3 ∧ Even 0 := by repeat' apply
And.intro all_goals apply Even.add_two -fails repeat' sorry
```

`try` 把其参数转换为一个永不失败的策略。

```
theorem all_goals_try_example :
  Even 4 ∧ Even 7 ∧ Even 3 ∧ Even 0 :=
  by
    repeat' apply And.intro
    all_goals try apply Even.add_two
    repeat sorry
```

`any_goals` 将其参数恰好应用一次到每个目标上。只要该参数在任一目标上成功，它就成功。

```
theorem any_goals_example :
  Even 4 ∧ Even 7 ∧ Even 3 ∧ Even 0 :=
  by
    repeat' apply And.intro
    any_goals apply Even.add_two
    repeat' sorry
```

`solve | ... | ... | ...` 类似于 `first`，区别是只有当某个参数完整证明当前目标时，它才成功。

```
theorem any_goals_solve_repeat_first_example :
  Even 4 ∧ Even 7 ∧ Even 3 ∧ Even 0 :=
  by
    repeat' apply And.intro
    any_goals
      solve
      | repeat'
        first
        | apply Even.add_two
        | apply Even.zero
    repeat' sorry
```

组合子 `repeat'` 很容易导致无限循环:

```
-loops theorem repeat' _Not_example : ¬ Even 1 := by repeat' apply Not.intro
```

9.2 宏

我们从真正的元编程开始：把一个自定义策略编码为宏。该策略体现了我们在上面的 `solve` 例子中硬编码的行为：

```
macro "intro_and_even" : tactic =>
  `(tactic|
    (repeat' apply And.intro
      any_goals
      solve
      | repeat'
        first
        | apply Even.add_two
        | apply Even.zero))
```

让我们应用自定义策略:

```
theorem intro_and_even_example :
  Even 4 ∧ Even 7 ∧ Even 3 ∧ Even 0 :=
by
  intro_and_even
  repeat' sorry
```

9.3 元编程 Monad

MetaM 是低层元编程 monad。TacticM 在 MetaM 的基础上加入目标管理。

- MetaM 是一种状态 monad，除其他内容外，它提供对全局语境的访问，包括所有定义和归纳类型、记法以及属性（例如 @[simp] 定理列表）。TacticM 还额外提供对目标列表的访问。
- MetaM 和 TacticM 的行为类似于 option monad。元程序 failure 会使 monad 进入错误状态。
- MetaM 和 TacticM 支持追踪，因此我们可以使用 logInfo 显示消息。
- 与其他 monad 一样，MetaM 和 TacticM 支持 for-in、continue 和 return 等命令式构造。

```
def traceGoals : TacticM Unit :=
do
  logInfo m!"Lean version {Lean.versionString}"
  logInfo "All goals:"
  let goals ← getUnsolvedGoals
  logInfo m!"{goals}"
  match goals with
```

```

| [] => return
| _ :: _ =>
  logInfo "First goal's target:"
  let target ← getMainTarget
  logInfo m! "{target}"

elab "trace_goals" : tactic =>
  traceGoals

theorem Even_18_and_Even_20 ( $\alpha$  : Type) (a :  $\alpha$ ) :
  Even 18  $\wedge$  Even 20 :=
by
  apply And.intro
  trace_goals
  intro_and_even

```

9.4 第一个例子：一个 assumption 策略

我们定义一个 **hypothesis** 策略，它的行为本质上与预定义的 **assumption** 策略相同。

```

def hypothesis : TacticM Unit :=
  withMainContext
  (do
    let target ← getMainTarget
    let lctx ← getLCtx
    for ldecl in lctx do
      if ! LocalDecl.isImplementationDetail ldecl then
        let eq ← isDefEq (LocalDecl.type ldecl) target
        if eq then
          let goal ← getMainGoal
          MVarId.assign goal (LocalDecl.toExpr ldecl)
          return
    failure)

elab "hypothesis" : tactic =>
  hypothesis

theorem hypothesis_example { $\alpha$  : Type} {p :  $\alpha \rightarrow \text{Prop}$ } {a :  $\alpha$ }

```

```
(hpa : p a) :
p a :=
by hypothesis
```

9.5 表达式

元编程框架围绕表达式或项的类型 `Expr` 展开。

```
#print Expr
```

9.5.1 名称

可以用反引号创建字面名称：

- 带一个反引号的名称 ``n` 不会检查是否存在。
- 带两个反引号的名称 ```n` 会被解析并检查。

```
#check `x
#eval `x
#eval `Even      -- wrong
#eval `LoVe.Even  -- suboptimal
#eval ``Even
#eval "EvenThough -fails"
```

9.5.2 常量

```
#check Expr.const

#eval ppExpr (Expr.const ``Nat.add [])
#eval ppExpr (Expr.const ``Nat [])
```

9.5.3 Sort (第 12 讲)

```
#check Expr.sort
```

```
#eval ppExpr (Expr.sort Level.zero)
```

```
#eval ppExpr (Expr.sort (Level.succ Level.zero))
```

9.5.4 自由变量

```
#check Expr.fvar
```

```
#check FVarId.mk `n
```

```
#eval ppExpr (Expr.fvar (FVarId.mk `n))
```

9.5.5 元变量

```
#check Expr.mvar
```

9.5.6 应用

```
#check Expr.app
```

```
#eval ppExpr (Expr.app (Expr.const ``Nat.succ [])  
  (Expr.const ``Nat.zero []))
```

9.5.7 匿名函数与束缚变量

```
#check Expr.bvar
```

```
#check Expr.lam
```

```
#eval ppExpr (Expr.bvar 0)
```



```
#eval ppExpr (Expr.lam `x (Expr.const ``Nat []) (Expr.bvar 0)
  BinderInfo.default)
```

```
#eval ppExpr (Expr.lam `x (Expr.const ``Nat [])
  (Expr.lam `y (Expr.const ``Nat []) (Expr.bvar 1)
    BinderInfo.default)
  BinderInfo.default)
```

9.5.8 依赖函数类型

```
#check Expr.forallE
```

```
#eval ppExpr (Expr.forallE `n (Expr.const ``Nat [])
  (Expr.app (Expr.const ``Even []) (Expr.bvar 0))
  BinderInfo.default)
```

```
#eval ppExpr (Expr.forallE `dummy (Expr.const `Nat [])
  (Expr.const `Bool []) BinderInfo.default)
```

9.5.9 其他构造子

```
#check Expr.letE
#check Expr.lit
#check Expr.mdata
#check Expr.proj
```

9.6 第二个例子：析取合取的策略

我们定义一个 `destruct_and` 策略，用来自动消去前提中的 $\wedge$ ，从而自动化如下证明：

```
theorem abc_a (a b c : Prop) (h : a  $\wedge$  b  $\wedge$  c) :
  a :=
  And.left h
```

```
theorem abc_b (a b c : Prop) (h : a ∧ b ∧ c) :
  b :=
  And.left (And.right h)
```

```
theorem abc_bc (a b c : Prop) (h : a ∧ b ∧ c) :
  b ∧ c :=
  And.right h
```

```
theorem abc_c (a b c : Prop) (h : a ∧ b ∧ c) :
  c :=
  And.right (And.right h)
```

我们的策略依赖一个辅助函数。该函数以要作为表达式使用的假设 h 为参数:

```
partial def destructAndExpr (hP : Expr) : TacticM Bool :=
  withMainContext
  (do
    let target ← getMainTarget
    let P ← inferType hP
    let eq ← isDefEq P target
    if eq then
      let goal ← getMainGoal
      MVarId.assign goal hP
      return true
    else
      match Expr.and? P with
      | Option.none      => return false
      | Option.some (Q, R) =>
        let hQ ← mkAppM ``And.left #[hP]
        let success ← destructAndExpr hQ
        if success then
          return true
        else
          let hR ← mkAppM ``And.right #[hP]
          destructAndExpr hR)

#check Expr.and?
```

```

def destructAnd (name : Name) : TacticM Unit :=
  withMainContext
    (do
      let h ← getFVarFromUserName name
      let success ← destructAndExpr h
      if ! success then
        failure)

elab "destruct_and" h:ident : tactic =>
  destructAnd (getId h)

```

让我们检查该策略确实能工作:

```

theorem abc_a_again (a b c : Prop) (h : a ∧ b ∧ c) :
  a :=
  by destruct_and h

theorem abc_b_again (a b c : Prop) (h : a ∧ b ∧ c) :
  b :=
  by destruct_and h

theorem abc_bc_again (a b c : Prop) (h : a ∧ b ∧ c) :
  b ∧ c :=
  by destruct_and h

theorem abc_c_again (a b c : Prop) (h : a ∧ b ∧ c) :
  c :=
  by destruct_and h

theorem abc_ac (a b c : Prop) (h : a ∧ b ∧ c) : a ∧ c := by destruct_and h -fails

```

9.7 第三个例子: 直接证明查找器

最后, 我们实现一个 `prove_direct` 工具, 它遍历数据库中的所有定理, 并检查其中是否有某个定理可用于证明当前目标。

```

def isTheorem : ConstantInfo → Bool
| ConstantInfo.axiomInfo _ => true

```

```
| ConstantInfo.thmInfo _ => true
| _                      => false
```

```
def applyConstant (name : Name) : TacticM Unit :=
do
  let cst ← mkConstWithFreshMVarLevels name
  liftMetaTactic (fun goal ↦ MVarId.apply goal cst)
```

```
def andThenOnSubgoals (tac1 tac2 : TacticM Unit) :
  TacticM Unit :=
do
  let origGoals ← getGoals
  let mainGoal ← getMainGoal
  setGoals [mainGoal]
  tac1
  let subgoals1 ← getUnsolvedGoals
  let mut newGoals := []
  for subgoal in subgoals1 do
    let assigned ← MVarId.isAssigned subgoal
    if ! assigned then
      setGoals [subgoal]
  tac2
  let subgoals2 ← getUnsolvedGoals
  newGoals := newGoals ++ subgoals2
  setGoals (newGoals ++ List.tail origGoals)
```

```
def proveUsingTheorem (name : Name) : TacticM Unit :=
  andThenOnSubgoals (applyConstant name) hypothesis
```

```
def proveDirect : TacticM Unit :=
do
  let origGoals ← getUnsolvedGoals
  let goal ← getMainGoal
  setGoals [goal]
  let env ← getEnv
  for (name, info)
    in SMap.toList (Environment.constants env) do
    if isTheorem info && ! ConstantInfo.isUnsafe info then
      try
```

```

    proveUsingTheorem name
    logInfo m!"Proved directly by {name}"
    setGoals (List.tail origGoals)
    return
  catch _ =>
    continue
failure

```

```

elab "prove_direct" : tactic =>
  proveDirect

```

让我们检查该策略确实能工作:

```

theorem Nat.symm (x y :  $\mathbb{N}$ ) (h : x = y) :
  y = x :=
  by prove_direct

```

```

theorem Nat.symm_manual (x y :  $\mathbb{N}$ ) (h : x = y) :
  y = x :=
  by
    apply symm
    hypothesis

```

```

theorem Nat.trans (x y z :  $\mathbb{N}$ ) (hxy : x = y) (hyz : y = z) :
  x = z :=
  by prove_direct

```

```

theorem List.reverse_twice (xs : List  $\mathbb{N}$ ) :
  List.reverse (List.reverse xs) = xs :=
  by prove_direct

```

Lean 有 apply?:

```

theorem List.reverse_twice_apply? (xs : List  $\mathbb{N}$ ) :
  List.reverse (List.reverse xs) = xs :=
  by apply?

```

第十章 LoVe 演示 9：操作语义

在本讲以及接下来的两讲中，我们将看到如何使用 **Lean** 规定程序设计语言的语法和语义，并对这些语义进行推理。

10.1 首先：形式化项目

你可以用一个验证项目替代两次作业，该项目价值 **20** 分。如果选择这样做，请在本周结束前给授课教师发送电子邮件。对于一个完全成功的项目，我们期望大约 **200** 行（或更多）**Lean** 代码，包括定义和证明。

下面给出一些项目想法。

计算机科学：

- 带静态数组或其他特性的扩展 **WHILE** 语言；
- 函数式数据结构（例如平衡树）；
- 函数式算法（例如冒泡排序、归并排序、**Tarjan** 算法）；
- 从表达式或命令式程序到某种目标语言的编译器，例如栈机；
- 类型系统（例如 **Benjamin Pierce** 的 **Types and Programming Languages**）；
- 安全性质（例如 **Volpano-Smith** 风格的非干涉分析）；
- 一阶项理论，包括匹配和项重写；
- 自动机理论；
- 上下文无关文法或正则表达式的规范化；
- 进程代数和双模拟；
- 证明系统的可靠性以及可能的完备性（例如 **Gentzen** 的相继式演算、自然演绎、表方法）；
- 分离逻辑；
- 使用 **Hoare** 逻辑验证程序。

数学：

- 图；
- 组合学；

- 数论。

元编程：

- 自定义策略；
- 自定义诊断工具。

以往评价：

问：你觉得这个项目怎么样？

答：令人愉快。

答：有趣且困难。

答：不错。我认为这种形式很好，因为它让人有机会做有挑战性的练习，并且即使未完成也能提交。

答：我真的非常喜欢。我觉得这是一种很好的学习方式：找一个你喜欢的东西，稍微深入进去，遇到困难，然后求助。我希望我能多做一些这样的事！

答：能有一些时间尝试自己解决感兴趣的内容，这很棒。

答：实际上非常有趣!!!

答：很有帮助。它提供了机会，让我在课程的某个特定方面多花一些时间。

10.2 形式语义

形式语义有助于规定程序设计语言本身以及单个程序，并对它们进行推理。

它可以构成经验证编译器、解释器、验证器、静态分析器、类型检查器等工具的基础。没有形式化证明，这些工具几乎总是错误的。

在这一领域，证明助理被广泛使用。每年约有 10-20% 的 POPL 论文被部分或完全形式化。这种成功的原因包括：

- 除归纳类型、归纳谓词和递归函数外，开始时几乎不需要其他机制（背景库、策略）。
- 这些证明往往有许多情形，这很适合计算机处理。
- 当我们为程序设计语言扩展更多特性时，证明助理会跟踪哪些内容需要修改。

典型例子：WebAssembly。引用 Conrad Watt 的话（略有删节）：

我们完成了 WebAssembly 语言核心执行语义和类型系统的完整 Isabelle 机械化。

为完成该证明，官方 WebAssembly 规范中由我们的证明和建模工作揭示出的**若干缺陷**

必须被修正。在某些情形中，这意味着类型系统**原本是不可靠的**。

我们一直与工作组保持建设性对话，在新特性加入时验证它们。尤其是，**WebAssembly** 实现与其宿主环境交互的机制，在工作组最初的论文中并未形式化规定。扩展我们的机械化以建模这一特性，揭示了 **WebAssembly** 规范中的一个缺陷，该缺陷**破坏了类型系统的可靠性**。

10.3 一个极简命令式语言

状态 s 是从变量名到值的函数 ($\text{String} \rightarrow \mathbb{N}$)。

WHILE 是一个极简命令式语言，具有如下文法：

```
S ::= skip          -- 空操作
    | x := a         -- 赋值
    | S; S           -- 顺序组合
    | if B then S else S -- 条件语句
    | while B do S    -- while 循环
```

其中 S 表示语句（也称为命令或程序）， x 表示变量， a 表示算术表达式， B 表示布尔表达式。

```
#print State
```

```
inductive Stmt : Type where
| skip      : Stmt
| assign    : String → (State → ℕ) → Stmt
| seq       : Stmt → Stmt → Stmt
| ifThenElse : (State → Prop) → Stmt → Stmt → Stmt
| whileDo   : (State → Prop) → Stmt → Stmt
```

```
infixr:90 "; " => Stmt.seq
```

在我们的文法中，我们有意不规定算术表达式和布尔表达式的语法。在 **Lean** 中，我们有如下选择：

- 可以使用第 2 讲中的 **AExp** 这类类型，并类似地处理布尔表达式。
- 可以决定算术表达式只是从状态到自然数的函数 ($\text{State} \rightarrow \mathbb{N}$)，而布尔表达式是谓词 ($\text{State} \rightarrow \text{Prop}$ 或 $\text{State} \rightarrow \text{Bool}$)。

这对应于深嵌入与浅嵌入的区别：

- 某种语法（表达式、公式、程序等）的 __ 深嵌入 __ 由证明助理中规定的抽象语法树（例如 **AExp**）以及一个语义（例如 **eval**）组成。
- 相比之下，__ 浅嵌入 __ 只是复用逻辑中的相应机制（例如项、函数、谓词类型）。

深嵌入允许我们对语法（及其语义）进行推理。浅嵌入更加轻量，因为我们可以直接使用它，而不必定义语义。

我们将使用程序的深嵌入（因为它有趣），以及赋值和布尔表达式的浅嵌入（因为它们无聊）。

程序

```
while x > y do skip; x := x - 1
```

被建模为

```
def sillyLoop : Stmt :=
  Stmt.withDo (fun s ↦ s "x" > s "y")
  (Stmt.skip;
   Stmt.assign "x" (fun s ↦ s "x" - 1))
```

10.4 大步语义

__ 操作语义 __ 对应于一个理想化的解释器（用类似 **Prolog** 的语言规定）。它有两种主要变体：

- 大步语义；
- 小步语义。

在 __ 大步语义 __（也称为 __ 自然语义 __）中，判断形如 $(S, s) \Rightarrow t$ ：

从状态 `s` 开始，执行 `S` 会在状态 `t` 中终止。

例子：

```
`(x := x + y; y := 0, [x ↦ 3, y ↦ 5]) ⇒ [x ↦ 8, y ↦ 0]`
```

推导规则：

$$\frac{}{(\text{skip}, s) \Rightarrow s} \text{Skip}$$

$$\frac{}{(x := a, s) \Rightarrow s[x \mapsto s(a)]} \text{Assign}$$

$(S, s) \Rightarrow t \quad (T, t) \Rightarrow u$	
$\hline (S; T, s) \Rightarrow u$	Seq
$(S, s) \Rightarrow t$	
$\hline (\text{if } B \text{ then } S \text{ else } T, s) \Rightarrow t$	If-True 若 $s(B)$ 为真
$(T, s) \Rightarrow t$	
$\hline (\text{if } B \text{ then } S \text{ else } T, s) \Rightarrow t$	If-False 若 $s(B)$ 为假
$(S, s) \Rightarrow t \quad (\text{while } B \text{ do } S, t) \Rightarrow u$	
$\hline (\text{while } B \text{ do } S, s) \Rightarrow u$	While-True 若 $s(B)$ 为真
$\hline (\text{while } B \text{ do } S, s) \Rightarrow s$	While-False 若 $s(B)$ 为假

上面, $s(e)$ 表示表达式 e 在状态 s 中的值, 而 $s[x \mapsto s(e)]$ 表示一个状态: 它除变量 x 被绑定到值 $s(e)$ 外, 与 s 完全相同。

在 **Lean** 中, 该判断对应于一个归纳谓词, 而推导规则对应于该谓词的引入规则。与递归函数相比, 使用归纳谓词使我们能够处理非终止 (例如发散的 **while**) 和非确定性 (例如多线程)。

inductive BigStep : Stmt × State → State → Prop where

```

| skip (s) :
  BigStep (Stmt.skip, s) s
| assign (x a s) :
  BigStep (Stmt.assign x a, s) (s[x ↦ a s])
| seq (S T s t u) (hS : BigStep (S, s) t)
  (hT : BigStep (T, t) u) :
  BigStep (S; T, s) u
| if_true (B S T s t) (hcond : B s)
  (hbody : BigStep (S, s) t) :
  BigStep (Stmt.ifThenElse B S T, s) t
| if_false (B S T s t) (hcond : ¬ B s)
  (hbody : BigStep (T, s) t) :
  BigStep (Stmt.ifThenElse B S T, s) t
| while_true (B S s t u) (hcond : B s)

```

```

(hbody : BigStep (S, s) t)
(hrest : BigStep (Stmt.whileDo B S, t) u) :
BigStep (Stmt.whileDo B S, s) u
| while_false (B S s) (hcond :  $\neg$  B s) :
BigStep (Stmt.whileDo B S, s) s

```

```
infix:110 "  $\implies$  " => BigStep
```

```

theorem sillyLoop_from_1_BigStep :
  (sillyLoop, (fun _  $\mapsto$  0)["x"  $\mapsto$  1])  $\implies$  (fun _  $\mapsto$  0) :=
by
  rw [sillyLoop]
  apply BigStep.while_true
  · simp
  · apply BigStep.seq
    · apply BigStep.skip
    · apply BigStep.assign
  · simp
  apply BigStep.while_false
  simp

```

10.5 大步语义的性质

有了大步语义后，我们可以

- 证明程序设计语言的性质，例如程序之间的等价性证明和确定性；
- 对具体程序进行推理，证明把终止状态 t 与初始状态 s 联系起来的定理。

```

theorem BigStep_deterministic {Ss l r} (hl : Ss  $\implies$  l)
  (hr : Ss  $\implies$  r) :
  l = r :=
by
  induction hl generalizing r with
  | skip s =>
    cases hr with
  | skip => rfl
  | assign x a s =>
    cases hr with

```

```

| assign => rfl
| seq S T s l0 l hS hT ihS ihT =>
  cases hr with
  | seq ___ r0 _ hS' hT' =>
    cases ihS hS' with
    | refl =>
      cases ihT hT' with
      | refl => rfl
| if_true B S T s l hB hS ih =>
  cases hr with
  | if_true _ _ _ _ _ hB' hS' => apply ih hS'
  | if_false _ _ _ _ _ hB' hS' => aesop
| if_false B S T s l hB hT ih =>
  cases hr with
  | if_true _ _ _ _ _ hB' hS' => aesop
  | if_false _ _ _ _ _ hB' hS' => apply ih hS'
| while_true B S s l0 l hB hS hw ihS ihw =>
  cases hr with
  | while_true ___ r0 hB' hB' hS' hw' =>
    cases ihS hS' with
    | refl =>
      cases ihw hw' with
      | refl => rfl
  | while_false ___ hB' => aesop
| while_false B S s hB =>
  cases hr with
  | while_true ___ s' _ hB' hS hw => aesop
  | while_false ___ hB' _ => rfl

```

theorem BigStep_terminates {S s} : $\exists t, (S, s) \Rightarrow t := \text{sorry}$ –unprovable

我们可以定义关于大步语义的反演规则:

```

@[simp] theorem BigStep_skip_iff {s t} :
  (Stmt.skip, s)  $\Rightarrow$  t  $\leftrightarrow$  t = s :=
by
  apply Iff.intro
  · intro h
    cases h with
    | skip => rfl

```

```

• intro h
rw [h]
apply BigStep.skip

```

```

@[simp] theorem BigStep_assign_iff {x a s t} :
  (Stmt.assign x a, s)  $\implies$  t  $\leftrightarrow$  t = s[x  $\mapsto$  a s] :=
by
  apply Iff.intro
  • intro h
    cases h with
    | assign => rfl
  • intro h
    rw [h]
    apply BigStep.assign

```

```

@[simp] theorem BigStep_seq_iff {S T s u} :
  (S; T, s)  $\implies$  u  $\leftrightarrow$  ( $\exists$ t, (S, s)  $\implies$  t  $\wedge$  (T, t)  $\implies$  u) :=
by
  apply Iff.intro
  • intro h
    cases h with
    | seq =>
      apply Exists.intro
      apply And.intro <;>
      assumption
  • intro h
    cases h with
    | intro s' h' =>
      cases h' with
      | intro hS hT =>
        apply BigStep.seq <;>
        assumption

```

```

@[simp] theorem BigStep_if_iff {B S T s t} :
  (Stmt.ifThenElse B S T, s)  $\implies$  t  $\leftrightarrow$ 
  (B s  $\wedge$  (S, s)  $\implies$  t)  $\vee$  ( $\neg$  B s  $\wedge$  (T, s)  $\implies$  t) :=
by
  apply Iff.intro
  • intro h

```

```

cases h with
| if_true _ _ _ _ hB hS =>
  apply Or.intro_left
  aesop
| if_false _ _ _ _ hB hT =>
  apply Or.intro_right
  aesop
• intro h
cases h with
| inl h =>
  cases h with
  | intro hB hS =>
    apply BigStep.if_true <;>
    assumption
| inr h =>
  cases h with
  | intro hB hT =>
    apply BigStep.if_false <;>
    assumption

```

theorem BigStep_while_Iff {B S s u} :

(Stmt.whileDo B S, s) $\Rightarrow$ u $\leftrightarrow$

(B s $\wedge \exists t, (S, s) \Rightarrow t \wedge (\text{Stmt.whileDo } B \text{ } S, t) \Rightarrow u$)

$\vee (\neg B \text{ } s \wedge u = s) :=$

by

apply Iff.intro

• intro h

cases h with

| while_true _ _ _ t _ hB hS hw => aesop

| while_false _ _ _ hB => aesop

• intro h

cases h with

| inl hex =>

cases hex with

| intro t h =>

cases h with

| intro hB h =>

cases h with

| intro hS hwhile =>

```

      apply BigStep.while_true <;>
      assumption
| inr h =>
  cases h with
| intro hB hus =>
  rw [hus]
  apply BigStep.while_false
  assumption

```

```

@[simp] theorem BigStep_while_true_iff {B S s u}
  (hcond : B s) :
  (Stmt.whileDo B S, s)  $\implies$  u  $\leftrightarrow$ 
  ( $\exists t$ , (S, s)  $\implies$  t  $\wedge$  (Stmt.whileDo B S, t)  $\implies$  u) :=
by
  rw [BigStep_while_iff]
  simp [hcond]

```

```

@[simp] theorem BigStep_while_false_iff {B S s t}
  (hcond :  $\neg$  B s) :
  (Stmt.whileDo B S, s)  $\implies$  t  $\leftrightarrow$  t = s :=
by
  rw [BigStep_while_iff]
  simp [hcond]

```

10.6 小步语义

大步语义

- 不允许我们对中间状态进行推理;
- 不允许我们表达非终止或交错执行 (用于多线程)。

小步语义 (也称为 __ 结构化操作语义 __) 解决了上述问题。

判断形如 $(S, s) \Rightarrow (T, t)$:

从状态 `s` 开始, 执行 `S` 的一步后, 得到状态 `t`, 并剩下程序 `T` 继续执行。

一次执行是有限或无限链 $(S_0, s_0) \Rightarrow (S_1, s_1) \Rightarrow \dots$ 。

一对 (S, s) 称为一个 __ 配置 __。若不存在形如 $(S, s) \Rightarrow _$ 的迁移, 则它是 __ 终止配置 __。

例子:

```
`(x := x + y; y := 0, [x ↦ 3, y ↦ 5])`
`⇒ (skip; y := 0, [x ↦ 8, y ↦ 5])`
`⇒ (y := 0, [x ↦ 8, y ↦ 5])`
`⇒ (skip, [x ↦ 8, y ↦ 0])`
```

推导规则:

Assign

$$(x := a, s) \Rightarrow (\text{skip}, s[x \mapsto s(a)])$$

Seq-Step

$$(S, s) \Rightarrow (S', s')$$

$$(S; T, s) \Rightarrow (S'; T, s')$$

Seq-Skip

$$(\text{skip}; S, s) \Rightarrow (S, s)$$

If-True 若 $s(B)$ 为真

$$(\text{if } B \text{ then } S \text{ else } T, s) \Rightarrow (S, s)$$

If-False 若 $s(B)$ 为假

$$(\text{if } B \text{ then } S \text{ else } T, s) \Rightarrow (T, s)$$

$$(\text{while } B \text{ do } S, s) \Rightarrow (\text{if } B \text{ then } (S; \text{while } B \text{ do } S) \text{ else skip}, s)$$

没有关于 `skip` 的规则 (为什么?)。

inductive SmallStep : Stmt × State → Stmt × State → Prop where

```
| assign (x a s) :
  SmallStep (Stmt.assign x a, s) (Stmt.skip, s[x ↦ a s])
| seq_step (S S' T s s') (hS : SmallStep (S, s) (S', s')) :
  SmallStep (S; T, s) (S'; T, s')
| seq_skip (T s) :
  SmallStep (Stmt.skip; T, s) (T, s)
| if_true (B S T s) (hcond : B s) :
  SmallStep (Stmt.ifThenElse B S T, s) (S, s)
| if_false (B S T s) (hcond : ¬ B s) :
  SmallStep (Stmt.ifThenElse B S T, s) (T, s)
```



```
| whileDo (B S s) :
  SmallStep (Stmt.whileDo B S, s)
  (Stmt.ifThenElse B (S; Stmt.whileDo B S) Stmt.skip, s)
```

```
infixr:100 "  $\Rightarrow$  " => SmallStep
```

```
infixr:100 "  $\Rightarrow^*$  " => RTC SmallStep
```

```
theorem sillyLoop_from_1_SmallStep :
```

```
  (sillyLoop, (fun _  $\mapsto$  0)["x"  $\mapsto$  1])  $\Rightarrow^*$ 
  (Stmt.skip, (fun _  $\mapsto$  0)) :=
```

```
by
```

```
  rw [sillyLoop]
```

```
  apply RTC.head
```

- apply SmallStep.whileDo
- apply RTC.head
- apply SmallStep.if_true
 - aesop
- apply RTC.head
 - apply SmallStep.seq_step
 - apply SmallStep.seq_skip
 - apply RTC.head
 - apply SmallStep.seq_step
 - apply SmallStep.assign
 - apply RTC.head
 - apply SmallStep.seq_skip
 - apply RTC.head
 - apply SmallStep.whileDo
 - apply RTC.head
 - apply SmallStep.if_false
 - simp
 - simp
 - apply RTC.refl

有了小步语义后，我们可以 __ 定义 __ 大步语义：

$\langle (S, s) \Rightarrow t \rangle$ 当且仅当 $\langle (S, s) \Rightarrow^* (\text{skip}, t) \rangle$

其中 r^* 表示关系 r 的自反传递闭包。

或者，如果我们已经定义了大步语义，也可以 __ 证明 __ 上面的等价性定理，以验证我们的定义。

小步语义的主要缺点是，现在我们有二个关系 $\Rightarrow$ 和 $\Rightarrow^*$ ，推理往往更复杂。

10.7 小步语义的性质

我们可以证明, 一个配置 (S, s) 是终止配置当且仅当 $S = \text{skip}$ 。这保证我们没有遗忘推导规则。

```

theorem SmallStep_final (S s) :
  (¬ ∃ T t, (S, s) ⇒ (T, t)) ↔ S = Stmt.skip :=
by
  induction S with
  | skip =>
    simp
    intros T t hstep
    cases hstep
  | assign x a =>
    simp
    apply Exists.intro Stmt.skip
    apply Exists.intro (s[x ↦ a s])
    apply SmallStep.assign
  | seq S T ihS ihT =>
    simp
    cases Classical.em (S = Stmt.skip) with
    | inl h =>
      rw [h]
      apply Exists.intro T
      apply Exists.intro s
      apply SmallStep.seq_skip
    | inr h =>
      simp [h] at ihS
      cases ihS with
      | intro S' hS₀ =>
        cases hS₀ with
        | intro s' hS =>
          apply Exists.intro (S'; T)
          apply Exists.intro s'
          apply SmallStep.seq_step
          assumption
    | ifThenElse B S T ihS ihT =>
      simp
      cases Classical.em (B s) with

```

```

| inl h =>
  apply Exists.intro S
  apply Exists.intro s
  apply SmallStep.if_true
  assumption
| inr h =>
  apply Exists.intro T
  apply Exists.intro s
  apply SmallStep.if_false
  assumption
| whileDo B S ih =>
  simp
  apply Exists.intro
    (Stmt.ifThenElse B (S; Stmt.whileDo B S)
      Stmt.skip)
  apply Exists.intro s
  apply SmallStep.whileDo

```

```

theorem SmallStep_deterministic {Ss LI Rr}
  (hl : Ss  $\Rightarrow$  LI) (hr : Ss  $\Rightarrow$  Rr) :
  LI = Rr :=
by
  induction hl generalizing Rr with
  | assign x a s =>
    cases hr with
    | assign _ _ => rfl
  | seq_step S S1 T s s1 hS1 ih =>
    cases hr with
    | seq_step S S2 _ s2 hS2 =>
      have hSS1S2 :=
        ih hS2
      aesop
    | seq_skip => cases hS1
  | seq_skip T s =>
    cases hr with
    | seq_step _ S _ s' hskip => cases hskip
    | seq_skip => rfl
  | if_true B S T s hB =>
    cases hr with

```

```

| if_true => rfl
| if_false => aesop
| if_false B S T s hB =>
  cases hr with
  | if_true => aesop
  | if_false => rfl
| whileDo B S s =>
  cases hr with
  | whileDo => rfl

```

我们也可以定义关于小步语义的反演规则。这里给出三个例子：

```

theorem SmallStep_skip {S s t} :
  ¬ ((Stmt.skip, s) ⇒ (S, t)) :=
by
  intro h
  cases h

```

```

@[simp] theorem SmallStep_seq_iff {S T s Ut} :
  (S; T, s) ⇒ Ut ↔
  (∃ S' t, (S, s) ⇒ (S', t) ∧ Ut = (S'; T, t))
  ∨ (S = Stmt.skip ∧ Ut = (T, s)) :=
by
  apply Iff.intro
  · intro hST
    cases hST with
    | seq_step _ S' _ _ s' hS =>
      apply Or.intro_left
      apply Exists.intro S'
      apply Exists.intro s'
      aesop
    | seq_skip =>
      apply Or.intro_right
      aesop
  · intro hor
    cases hor with
    | inl hex =>
      cases hex with
      | intro S' hex' =>

```

```

cases hex' with
| intro s' hand =>
  cases hand with
  | intro hS hUt =>
    rw [hUt]
    apply SmallStep.seq_step
    assumption
| inr hand =>
  cases hand with
  | intro hS hUt =>
    rw [hS, hUt]
    apply SmallStep.seq_skip

```

@[simp] theorem SmallStep_if_Iff {B S T s Us} :

(Stmt.ifThenElse B S T, s) $\Rightarrow$ Us $\leftrightarrow$

(B s $\wedge$ Us = (S, s)) $\vee$ ($\neg$ B s $\wedge$ Us = (T, s)) :=

by

apply Iff.intro

• intro h

cases h with

| if_true _ _ _ hB => aesop

| if_false _ _ _ hB => aesop

• intro hor

cases hor with

| inl hand =>

cases hand with

| intro hB hUs =>

rw [hUs]

apply SmallStep.if_true

assumption

| inr hand =>

cases hand with

| intro hB hUs =>

rw [hUs]

apply SmallStep.if_false

assumption

10.7.1 大步语义与小步语义的等价性 (可选)

一个更重要的结果是大步语义与小步语义之间的联系:

$$\text{`(S, s) } \Longrightarrow t \leftrightarrow (S, s) \Rightarrow^* (\text{Stmt.skip}, t)\text{'}$$

下面给出的证明超出了本课程范围。

```

theorem RTC_SmallStep_seq {S T s u}
  (h : (S, s)  $\Rightarrow^*$  (Stmt.skip, u)) :
  (S; T, s)  $\Rightarrow^*$  (Stmt.skip; T, u) :=
by
  apply RTC.lift (fun Ss  $\mapsto$  (Prod.fst Ss; T, Prod.snd Ss)) _ h
  intro Ss Ss' hrtc
  cases Ss with
  | mk S s =>
    cases Ss' with
    | mk S' s' =>
      apply SmallStep.seq_step
      assumption

theorem RTC_SmallStep_of_BigStep {Ss t} (hS : Ss  $\Longrightarrow$  t) :
  Ss  $\Rightarrow^*$  (Stmt.skip, t) :=
by
  induction hS with
  | skip => exact RTC.refl
  | assign =>
    apply RTC.single
    apply SmallStep.assign
  | seq S T s t u hS hT ihS ihT =>
    apply RTC.trans
    · exact RTC_SmallStep_seq ihS
    · apply RTC.head
      apply SmallStep.seq_skip
      assumption
  | if_true B S T s t hB hst ih =>
    apply RTC.head
    · apply SmallStep.if_true
      assumption
    · assumption

```

```

| if_false B S T s t hB hst ih =>
  apply RTC.head
  · apply SmallStep.if_false
  assumption
  · assumption
| while_true B S s t u hB hS hw ihS ihw =>
  apply RTC.head
  · apply SmallStep_whileDo
  · apply RTC.head
  · apply SmallStep.if_true
  assumption
  · apply RTC.trans
    · exact RTC_SmallStep_seq ihS
    · apply RTC.head
    apply SmallStep.seq_skip
  assumption
| while_false B S s hB =>
  apply RTC.tail
  apply RTC.single
  apply SmallStep_whileDo
  apply SmallStep.if_false
  assumption

```

theorem BigStep_of_SmallStep_of_BigStep {Ss₀ Ss₁ S₂}

(h₁ : Ss₀ ⇒ Ss₁) :

Ss₁ ⇒ S₂ → Ss₀ ⇒ S₂ :=

by

induction h₁ generalizing s₂ with

```

| assign x a s      => simp
| seq_step S S' T s s' hS ih => aesop
| seq_skip T s      => simp
| if_true B S T s hB      => aesop
| if_false B S T s hB     => aesop
| whileDo B S s        => aesop

```

theorem BigStep_of_RTC_SmallStep {Ss t} :

Ss ⇒* (Stmt.skip, t) → Ss ⇒ t :=

by

intro hS

```

induction hS using RTC.head_induction_on with
| refl =>
  apply BigStep.skip
| head Ss Ss' hST hsmallIT ih =>
  cases Ss' with
| mk S' s' =>
  apply BigStep_of_SmallStep_of_BigStep hST
  apply ih

```

```

theorem BigStep_Iff_RTC_SmallStep {Ss t} :
  Ss  $\implies$  t  $\leftrightarrow$  Ss  $\Rightarrow^*$  (Stmt.skip, t) :=
  Iff.intro RTC_SmallStep_of_BigStep BigStep_of_RTC_SmallStep

```


第十一章 LoVe 演示 10: Hoare 逻辑

我们回顾规定程序设计语言语义的第二种方式: **Hoare 逻辑**。如果说操作语义对应于一个理想化解释器, 那么 **Hoare 逻辑** (也称为 **公理语义**) 对应于一个验证器。Hoare 逻辑特别适合对具体程序进行推理。

```
open Lean
open Lean.Meta
open Lean.Elab.Tactic
```

11.1 Hoare 三元组

Hoare 逻辑的基本判断通常称为 **Hoare 三元组**。它们形如

$\{P\} S \{Q\}$

其中 S 是语句, P 和 Q 是关于状态变量的逻辑公式, 分别称为 **前置条件** 和 **后置条件**。

预期含义:

如果在执行 S 之前 P 成立, 并且执行正常终止, 那么终止时 Q 成立。

这是一个 **部分正确性** 陈述: 如果程序正常终止 (即没有运行时错误、没有无限循环或发散), 那么程序是正确的。

所有下列 Hoare 三元组都是有效的 (相对于预期含义):

```
{True} b := 4 {b = 4}
{a = 2} b := 2 * a {a = 2 ∧ b = 4}
{b ≥ 5} b := b + 1 {b ≥ 6}
{False} skip {b = 100}
{True} while i ≠ 100 do i := i + 1 {i = 100}
```

11.2 Hoare 规则

下面是一组用于推理 WHILE 程序的完备规则:

$$\frac{}{\{P\} \text{ skip } \{P\}} \text{ Skip}$$

$$\frac{}{\{Q[a/x]\} x := a \{Q\}} \text{ Assign}$$

$$\frac{\{P\} S \{R\} \quad \{R\} S' \{Q\}}{\{P\} S; S' \{Q\}} \text{ Seq}$$

$$\frac{\{P \wedge B\} S \{Q\} \quad \{P \wedge \neg B\} S' \{Q\}}{\{P\} \text{ if } B \text{ then } S \text{ else } S' \{Q\}} \text{ If}$$

$$\frac{\{P \wedge B\} S \{P\}}{\{P\} \text{ while } B \text{ do } S \{P \wedge \neg B\}} \text{ While}$$

$$\frac{P' \rightarrow P \quad \{P\} S \{Q\} \quad Q \rightarrow Q'}{\{P'\} S \{Q'\}} \text{ Conseq}$$

$Q[a/x]$ 表示把 Q 中的 x 替换为 a 。

在 While 规则中, P 称为 __ 不变量 __。

除 Conseq 外, 这些规则都是语法驱动的: 看一眼程序, 就能立即知道应当应用哪条规则。

示例推导:

$$\frac{\frac{}{\{a = 2\} b := a \{b = 2\}} \text{ Assign} \quad \frac{}{\{b = 2\} c := b \{c = 2\}} \text{ Assign}}{\{a = 2\} b := a; c := b \{c = 2\}} \text{ Seq}$$

$$\frac{}{x > 10 \rightarrow x > 5 \quad \{x > 5\} y := x \{y > 5\} \quad y > 5 \rightarrow y > 0} \text{ Assign}$$

Conseq

$$\{x > 10\} y := x \{y > 0\}$$

各种 __ 导出规则 __ 可以被证明相对于标准规则是正确的。例如，我们可以为 `skip`、`:=` 和 `while` 导出双向规则：

$$P \rightarrow Q$$

$$\frac{}{\text{Skip}'}$$

$$\{P\} \text{ skip } \{Q\}$$

$$P \rightarrow Q[a/x]$$

$$\frac{}{\text{Assign}'}$$

$$\{P\} x := a \{Q\}$$

$$\{P \wedge B\} S \{P\} \quad P \wedge \neg B \rightarrow Q$$

$$\frac{}{\text{While}'}$$

$$\{P\} \text{ while } B \text{ do } S \{Q\}$$

11.3 Hoare 逻辑的语义方法

我们可以而且将会在 `Lean` 中以语义方式定义 `Hoare` 三元组。

我们将使用状态上的谓词 (`State → Prop`) 表示前置条件和后置条件，遵循浅嵌入风格。

```
def PartialHoare (P : State → Prop) (S : Stmt)
```

```
  (Q : State → Prop) : Prop :=
```

```
  ∀ s t, P s → (S, s) ⇒ t → Q t
```

```
notation "{* " P " *}" " (" S ")" " {* " Q " *}" =>
```

```
  PartialHoare P S Q
```

```
theorem skip_intro {P} :
```

```
  {P} (Stmt.skip) {P} :=
```

```
  by
```

```
    intro s t hs hst
```

```
    cases hst
```

```
    assumption
```

```

theorem assign_intro (P) {x a} :
  { * fun s  $\mapsto$  P (s[x  $\mapsto$  a s]) * } (Stmt.assign x a) { * P * } :=
  by
    intro s t P' hst
    cases hst with
    | assign => assumption

theorem seq_intro {P Q R S T} (hS : { * P * } (S) { * Q * })
  (hT : { * Q * } (T) { * R * }) :
  { * P * } (S; T) { * R * } :=
  by
    intro s t hs hst
    cases hst with
    | seq _ _ _ u d hS' hT' =>
      apply hT
      · apply hS
      · exact hs
      · assumption
      · assumption

theorem if_intro {B P Q S T}
  (hS : { * fun s  $\mapsto$  P s  $\wedge$  B s * } (S) { * Q * })
  (hT : { * fun s  $\mapsto$  P s  $\wedge$   $\neg$  B s * } (T) { * Q * }) :
  { * P * } (Stmt.ifThenElse B S T) { * Q * } :=
  by
    intro s t hs hst
    cases hst with
    | if_true _ _ _ _ hB hS' =>
      apply hS
      exact And.intro hs hB
      assumption
    | if_false _ _ _ _ hB hT' =>
      apply hT
      exact And.intro hs hB
      assumption

theorem while_intro (P) {B S}
  (h : { * fun s  $\mapsto$  P s  $\wedge$  B s * } (S) { * P * }) :
  { * P * } (Stmt.whileDo B S) { * fun s  $\mapsto$  P s  $\wedge$   $\neg$  B s * } :=

```

```

by
  intro s t hs hst
  generalize ws_eq : (Stmt.whileDo B S, s) = Ss
  rw [ws_eq] at hst
  induction hst generalizing s with
  | skip s'           => aesop
  | assign x a s'      => aesop
  | seq S T s' t' u hS hT ih   => aesop
  | if_true B S T s' t' hB hS ih => aesop
  | if_false B S T s' t' hB hT ih => aesop
  | while_true B' S' s' t' u hB' hS hw ih_hS ih_hw =>
    cases ws_eq
    apply ih_hw
    · apply h
    · apply And.intro <;>
      assumption
    · exact hS
    · rfl
  | while_false B' S' s' hB'   => aesop

```

```

theorem consequence {P P' Q Q' S}
  (h : {* P *} (S) {* Q *}) (hp : ∀s, P' s → P s)
  (hq : ∀s, Q s → Q' s) :
  {* P' *} (S) {* Q' *} :=
fix s t : State
assume hs : P' s
assume hst : (S, s) ⇒ t
show Q' t from
  hq _ (h s t (hp s hs) hst)

```

```

theorem consequence_left (P') {P Q S}
  (h : {* P *} (S) {* Q *}) (hp : ∀s, P' s → P s) :
  {* P' *} (S) {* Q *} :=
consequence h hp (by aesop)

```

```

theorem consequence_right (Q) {Q' P S}
  (h : {* P *} (S) {* Q *}) (hq : ∀s, Q s → Q' s) :
  {* P *} (S) {* Q' *} :=
consequence h (by aesop) hq

```

```

theorem skip_intro' {P Q} (h :  $\forall s, P\ s \rightarrow Q\ s$ ) :
  { * P * } (Stmt.skip) { * Q * } :=
  consequence skip_intro h (by aesop)

```

```

theorem assign_intro' {P Q x a}
  (h :  $\forall s, P\ s \rightarrow Q\ (s[x \mapsto a])$ ):
  { * P * } (Stmt.assign x a) { * Q * } :=
  consequence (assign_intro Q) h (by aesop)

```

```

theorem seq_intro' {P Q R S T} (hT : { * Q * } (T) { * R * })
  (hS : { * P * } (S) { * Q * }) :
  { * P * } (S; T) { * R * } :=
  seq_intro hS hT

```

```

theorem while_intro' {B P Q S} (I)
  (hS : { * fun s  $\mapsto I\ s \wedge B\ s$  * } (S) { * I * })
  (hP :  $\forall s, P\ s \rightarrow I\ s$ )
  (hQ :  $\forall s, \neg B\ s \rightarrow I\ s \rightarrow Q\ s$ ) :
  { * P * } (Stmt.whileDo B S) { * Q * } :=
  consequence (while_intro I hS) hP (by aesop)

```

```

theorem assign_intro_forward (P) {x a} :
  { * P * }
  (Stmt.assign x a)
  { * fun s  $\mapsto \exists n_0, P\ (s[x \mapsto n_0]) \wedge s\ x = a\ (s[x \mapsto n_0])$  * } :=
  by
    apply assign_intro'
    intro s hP
    apply Exists.intro (s x)
    simp [*]

```

```

theorem assign_intro_backward (Q) {x a} :
  { * fun s  $\mapsto \exists n', Q\ (s[x \mapsto n']) \wedge n' = a\ s$  * }
  (Stmt.assign x a)
  { * Q * } :=
  by
    apply assign_intro'
    aesop

```

11.4 第一个程序：交换两个变量

```
def SWAP : Stmt :=
  Stmt.assign "t" (fun s ↦ s "a");
  Stmt.assign "a" (fun s ↦ s "b");
  Stmt.assign "b" (fun s ↦ s "t")

theorem SWAP_correct (a₀ b₀ : ℕ) :
  { * fun s ↦ s "a" = a₀ ∧ s "b" = b₀ * }
  (SWAP)
  { * fun s ↦ s "a" = b₀ ∧ s "b" = a₀ * } :=
  by
    apply PartialHoare.seq_intro'
    apply PartialHoare.seq_intro'
    apply PartialHoare.assign_intro
    apply PartialHoare.assign_intro
    apply PartialHoare.assign_intro'
    aesop
```

11.5 第二个程序：两个数相加

```
def ADD : Stmt :=
  Stmt.withDo (fun s ↦ s "n" ≠ 0)
    (Stmt.assign "n" (fun s ↦ s "n" - 1);
     Stmt.assign "m" (fun s ↦ s "m" + 1))

theorem ADD_correct (n₀ m₀ : ℕ) :
  { * fun s ↦ s "n" = n₀ ∧ s "m" = m₀ * }
  (ADD)
  { * fun s ↦ s "n" = 0 ∧ s "m" = n₀ + m₀ * } :=
  PartialHoare.with_intro' (fun s ↦ s "n" + s "m" = n₀ + m₀)
  (by
    apply PartialHoare.seq_intro'
    · apply PartialHoare.assign_intro
    · apply PartialHoare.assign_intro'
    aesop)
```

(by aesop)

(by aesop)

我们是如何想到这个不变量的？不变量必须

1. 在进入循环前为真；
2. 如果在某次迭代前为真，则在该次迭代后仍为真；
3. 足够强，能够推出所期望的循环后置条件。

不变量 **True** 满足 1 和 2，但通常不满足 3。类似地，**False** 满足 2 和 3，但通常不满足 1。合适的不变量常常具有如下形式

已完成的工作 + 剩余的工作 = 期望的结果

其中 + 是某个合适的运算符。进入循环时，__ 已完成的工作 __ 通常为 0。退出循环时，__ 剩余的工作 __ 应当为 0。

对于 **ADD** 循环：

- __ 已完成的工作 __ 是 m ；
- __ 剩余的工作 __ 是 n ；
- __ 期望的结果 __ 是 $n_0 + m_0$ 。

11.6 一个验证条件生成器

验证条件生成器（VCG）是自动应用 **Hoare** 规则的程序，它们产生必须由用户证明的 __ 验证条件 __。用户通常还必须在程序中以标注形式提供足够强的循环不变量。

我们可以使用 **Lean** 的元编程框架定义一个简单的 VCG。

数百种程序验证工具都建立在这些原则之上。

VCG 通常从后置条件出发向后工作，使用反向规则（即以后置条件为任意 Q 的方式陈述的规则）。这很有效，因为 **Assign** 是反向的。

```
def Stmt.invWhileDo (I B : State → Prop) (S : Stmt) : Stmt :=
  Stmt.whileDo B S
```

```
theorem invWhile_intro {B I Q S}
  (hS : {* fun s ↦ I s ∧ B s *} (S) {* I *})
  (hQ : ∀s, ¬ B s → I s → Q s) :
```



```
{* I *} (Stmt.invWhileDo I B S) {* Q *} :=
while_intro' I hS (by aesop) hQ
```

```
theorem invWhile_intro' {B I P Q S}
  (hS : {s : Stmt | I s ∧ B s} → {s : Stmt | I s})
  (hP : ∀s, P s → I s) (hQ : ∀s, ¬ B s → I s → Q s) :
  {s : Stmt | P s} (Stmt.invWhileDo I B S) {s : Stmt | Q s} :=
while_intro' I hS hP hQ
```

```
def matchPartialHoare : Expr → Option (Expr × Expr × Expr)
| (Expr.app (Expr.app (Expr.app
  (Expr.const ``PartialHoare _) P) S) Q) =>
  Option.some (P, S, Q)
| _ =>
  Option.none
```

```
partial def vcg : TacticM Unit :=
do
  let goals ← getUnsolvedGoals
  if goals.length != 0 then
    let target ← getMainTarget
    match matchPartialHoare target with
    | Option.none => return
    | Option.some (P, S, Q) =>
      if Expr.isAppOfArity S ``Stmt.skip 0 then
        if Expr.isMVar P then
          applyConstant ``PartialHoare.skip_intro
        else
          applyConstant ``PartialHoare.skip_intro'
      else if Expr.isAppOfArity S ``Stmt.assign 2 then
        if Expr.isMVar P then
          applyConstant ``PartialHoare.assign_intro
        else
          applyConstant ``PartialHoare.assign_intro'
      else if Expr.isAppOfArity S ``Stmt.seq 2 then
        andThenOnSubgoals
          (applyConstant ``PartialHoare.seq_intro') vcg
      else if Expr.isAppOfArity S ``Stmt.ifThenElse 3 then
```

```

    andThenOnSubgoals
      (applyConstant ``PartialHoare.if_intro) vcg
  else if Expr.isAppOfArity S ``Stmt.invWhileDo 3 then
    if Expr.isMVar P then
      andThenOnSubgoals
        (applyConstant ``PartialHoare.invWhile_intro) vcg
    else
      andThenOnSubgoals
        (applyConstant ``PartialHoare.invWhile_intro')
      vcg
  else
    failure

elab "vcg" : tactic =>
  vcg

```

11.7 重新考察第二个程序：两个数相加

```

theorem ADD_correct_vcg (n0 m0 : ℕ) :
  { * fun s ↦ s "n" = n0 ∧ s "m" = m0 * }
  (ADD)
  { * fun s ↦ s "n" = 0 ∧ s "m" = n0 + m0 * } :=
show { * fun s ↦ s "n" = n0 ∧ s "m" = m0 * }
  (Stmt.invWhileDo (fun s ↦ s "n" + s "m" = n0 + m0)
    (fun s ↦ s "n" ≠ 0)
    (Stmt.assign "n" (fun s ↦ s "n" - 1);
      Stmt.assign "m" (fun s ↦ s "m" + 1)))
  { * fun s ↦ s "n" = 0 ∧ s "m" = n0 + m0 * } from
by
  vcg <;>
  aesop

```

11.8 总正确性的 Hoare 三元组

__ 总正确性 __ 断言程序不仅是部分正确的，而且总会正常终止。总正确性的 Hoare 三元组形如

$[P] S [Q]$

预期含义:

如果在执行 `S` 之前 `P` 成立, 则执行会正常终止, 并且在最终状态中 `Q` 成立。

对于确定性程序, 一个等价表述如下:

如果在执行 `S` 之前 `P` 成立, 则存在一个状态, 使得执行在该状态正常终止, 且 `Q` 在该状态中成立。

例子:

$\text{`}[i \leq 10] \text{ while } i \neq 10 \text{ do } i := i + 1 [i = 10]\text{'}$

在我们的 WHILE 语言中, 这只影响 while 循环; 此时循环必须用一个 __ 变式 __ V (每次迭代都会减小的自然数) 标注:

$[P \wedge B \wedge V = v_0] S [P \wedge V < v_0]$

While-Var

$[P] \text{ while } B \text{ do } S [P \wedge \neg B]$

对于上面的例子, 什么是合适的变式? # LoVe 演示 11: 指称语义

我们回顾规定程序设计语言语义的第三种方式: 指称语义。指称语义试图直接规定程序的含义。

如果操作语义是一个理想化解释器, Hoare 逻辑是一个理想化验证器, 那么指称语义就是一个理想化编译器。

11.9 组合性

__ 指称语义 __ 把每个程序的含义定义为一个数学对象:

$\text{`}[] : \text{syntax} \rightarrow \text{semantics}\text{'}$

指称语义的一个关键性质是 __ 组合性 __: 复合语句的含义应当根据其组成部分的含义来定义。这排除了

$\text{`}[S] = \{(s, t) \mid (S, s) \Rightarrow t\}\text{'}$

因为操作语义并不是组合式的。

简言之, 我们希望有

$\text{`}[S; T] = \dots [S] \dots [T] \dots\text{'}$

$\text{`}[if B then S else T] = \dots [S] \dots [T] \dots\text{'}$

$\text{`}[while B do S] = \dots [S] \dots\text{'}$

算术表达式上的求值函数

``eval : AExp → ((String → ℤ) → ℤ)``

就是一个指称语义。我们希望命令式程序也具有同样的形式。

11.10 关系式指称语义

我们可以把命令式程序的语义表示为从初始状态到终止状态的函数，或者更一般地，表示为初始状态与终止状态之间的关系： $\text{Set}(\text{State} \times \text{State})$ 。

对于 `skip`、`:=`、`;` 以及 `if then else`，指称语义很容易给出：

```
def denote : Stmt → Set (State × State)
| Stmt.skip          => Id
| Stmt.assign x a     => {(s, t) | t = s[x ↦ a s]}
| Stmt.seq S T       => denote S ○ denote T
| Stmt.ifThenElse B S T =>
  (denote S ↓ B) ∪ (denote T ↓ (fun s ↦ ¬ B s))
| Stmt.whileDo B S    => sorry
```

我们把 `denote S` 写作 $\llbracket S \rrbracket$ 。对于 `while`，我们希望写成

```
`((denote S ○ denote (Stmt.whileDo B S)) ↓ B)`
`∪ (Id ↓ (fun s ↦ ¬ B s))`
```

但由于递归调用 `Stmt.whileDo B S`，这不是良基的。

我们寻找的是一个 X ，使得

```
`X = ((denote S ○ X) ↓ B) ∪ (Id ↓ (fun s ↦ ¬ B s))`
```

换言之，我们寻找一个不动点。

本讲的大部分内容都用于构造一个最小不动点算子 `lfp`，它也将允许我们定义 `while` 情形：

```
`lfp (fun X ↦ ((denote S ○ X) ↓ B) ∪ (Id ↓ (fun s ↦ ¬ B s)))`
```

11.11 不动点

f 的一个 不动点 （或 `fixed point`）是方程

$X = f X$

中 X 的一个解。

一般而言，不动点可能根本不存在（例如 $f := \text{Nat.succ}$ ），也可能有多个不动点（例如 $f := \text{id}$ ）。但在 f 满足某些条件时，可以保证存在唯一的 __ 最小不动点 __ 和唯一的 __ 最大不动点 __。

考虑这个 __ 不动点方程 __：

$X = (\text{fun } (P : \mathbb{N} \rightarrow \text{Prop}) (n : \mathbb{N}) \mapsto n = 0 \vee \exists m : \mathbb{N}, n = m + 2 \wedge P m) X$
 $= (\text{fun } n : \mathbb{N} \mapsto n = 0 \vee \exists m : \mathbb{N}, n = m + 2 \wedge X m)$

其中 $X : \mathbb{N} \rightarrow \text{Prop}$, $f := (\text{fun } (P : \mathbb{N} \rightarrow \text{Prop}) (n : \mathbb{N}) \mapsto n = 0 \vee \exists m : \mathbb{N}, n = m + 2 \wedge P m)$ 。

上面的例子只有一个不动点。该不动点方程唯一地把 X 规定为刻画偶数的谓词。

一般而言，最小不动点和最大不动点可以不同：

$X = X$

这里，最小不动点是 $\text{fun } _ \mapsto \text{False}$ ，最大不动点是 $\text{fun } _ \mapsto \text{True}$ 。按照惯例， $\text{False} < \text{True}$ ，因此 $(\text{fun } _ \mapsto \text{False}) < (\text{fun } _ \mapsto \text{True})$ 。类似地， $\emptyset < @\text{Set.univ } \mathbb{N}$ 。

对于程序设计语言的语义：

- X 将具有类型 $\text{Set } (\text{State} \times \text{State})$ （它同构于 $\text{State} \rightarrow \text{State} \rightarrow \text{Prop}$ ），表示状态之间的关系；
- f 将对应于多执行一次循环迭代（若条件 B 为真），或者对应于恒等关系（若 B 为假）。

最小不动点对应于程序的有限执行，而这正是我们关心的一切。

关键观察：

归纳谓词对应于最小不动点，但它们内建于 Lean 的逻辑（归纳构造演算）之中。

11.12 单调函数

令 α 和 β 是带偏序 $\leq$ 的类型。函数 $f : \alpha \rightarrow \beta$ 称为 __ 单调 __ 的，如果

对所有 a_1, a_2 ，有 $a_1 \leq a_2 \rightarrow f a_1 \leq f a_2$

许多集合上的运算（例如 $\cup$ ）、关系上的运算（例如 $\circ$ ）以及函数上的运算（例如 $\text{fun } x \mapsto x$ 、 $\text{fun } _ \mapsto k$ 、 $\circ$ ）都是单调的，或保持单调性。

所有单调函数 $f : \text{Set } \alpha \rightarrow \text{Set } \alpha$ 都有最小不动点和最大不动点。

非单调函数的例子：

`f A = (if A =  $\emptyset$  then Set.univ else  $\emptyset$ )`

假设 α 非空, 则有 $\emptyset \subseteq \text{Set.univ}$, 但 $f \emptyset = \text{Set.univ} \not\subseteq \emptyset = f \text{Set.univ}$ 。

```
def Monotone {α β : Type} [PartialOrder α] [PartialOrder β]
  (f : α → β) : Prop :=
  ∀ a₁ a₂, a₁ ≤ a₂ → f a₁ ≤ f a₂
```

```
theorem Monotone_id {α : Type} [PartialOrder α] :
  Monotone (fun a : α ↦ a) :=
  by
    intro a₁ a₂ ha
    exact ha
```

```
theorem Monotone_const {α β : Type} [PartialOrder α]
  [PartialOrder β] (b : β) :
  Monotone (fun _ : α ↦ b) :=
  by
    intro a₁ a₂ ha
    exact le_refl b
```

```
theorem Monotone_union {α β : Type} [PartialOrder α]
  (f g : α → Set β) (hf : Monotone f) (hg : Monotone g) :
  Monotone (fun a ↦ f a ∪ g a) :=
  by
    intro a₁ a₂ ha b hb
    cases hb with
    | inl h => exact Or.inl (hf a₁ a₂ ha h)
    | inr h => exact Or.inr (hg a₁ a₂ ha h)
```

我们将在习题中证明下列两个定理。

```
theorem Monotone_comp {α β : Type} [PartialOrder α]
  (f g : α → Set (β × β)) (hf : Monotone f)
  (hg : Monotone g) :
  Monotone (fun a ↦ f a ○ g a) :=
  sorry
```

```
theorem Monotone_restrict {α β : Type} [PartialOrder α]
```

```

(f :  $\alpha \rightarrow \text{Set } (\beta \times \beta)$ ) (P :  $\beta \rightarrow \text{Prop}$ ) (hf : Monotone f) :
Monotone (fun a  $\mapsto$  f a  $\downarrow$  P) :=
sorry

```

11.13 完全格

为了在集合上定义最小不动点，我们需要 $\subseteq$ 和 $\bigcap$: $\bigcap \{A \mid f A \subseteq A\}$ 。完全格抽象地刻画了这一概念。__ 完全格 __ 是一个有序类型 α ，其中每个类型为 $\text{Set } \alpha$ 的集合都有下确界。

更精确地说，一个完全格由以下内容组成：

- 一个偏序 $\leq : \alpha \rightarrow \alpha \rightarrow \text{Prop}$ （即一个自反、反对称且传递的二元谓词）；
- 一个算子 $\text{Inf} : \text{Set } \alpha \rightarrow \alpha$ ，称为 __ 下确界 __。

此外， $\text{Inf } A$ 必须满足下列两个性质：

- $\text{Inf } A$ 是 A 的下界：对所有 $b \in A$ ，有 $\text{Inf } A \leq b$ ；
- $\text{Inf } A$ 是最大下界：对所有满足 $\forall a, a \in A \rightarrow b \leq a$ 的 b ，有 $b \leq \text{Inf } A$ 。

警告： $\text{Inf } A$ 不一定是 A 的元素。

例子：

- 对所有 α ， $\text{Set } \alpha$ 关于 $\subseteq$ 和 $\bigcap$ 构成实例；
- Prop 关于 $\rightarrow$ 和 $\forall$ 构成实例 ($\text{Inf } A := \forall a \in A, a$)；
- $\text{ENat} := \mathbb{N} \cup \{\infty\}$ ；
- $\text{EReal} := \mathbb{R} \cup \{-\infty, \infty\}$ ；
- 若 α 是完全格，则 $\beta \rightarrow \alpha$ 是完全格；
- 若 α, β 是完全格，则 $\alpha \times \beta$ 是完全格。

有限例子（请原谅 ASCII 图）：

```

      Z      Inf {}      = ?
    /  \     Inf {Z}     = ?
  A    B     Inf {A, B}   = ?
  \  /      Inf {Z, A}    = ?
    Y       Inf {Z, A, B, Y} = ?

```

非例子：

- $\mathbb{N}$ 、 $\mathbb{Z}$ 、 $\mathbb{Q}$ 、 $\mathbb{R}$ ： $\emptyset$ 没有下确界。
- $\text{ERat} := \mathbb{Q} \cup \{-\infty, \infty\}$ ： $\text{Inf } \{q \mid 2 < q * q\} = \text{sqrt } 2$ 不在 ERat 中。

```

class CompleteLattice (α : Type)
  extends PartialOrder α : Type where
  Inf    : Set α → α
  Inf_le : ∀A b, b ∈ A → Inf A ≤ b
  le_Inf : ∀A b, (∀a, a ∈ A → b ≤ a) → b ≤ Inf A

```

对于集合:

```

instance Set.CompleteLattice {α : Type} :
  CompleteLattice (Set α) :=
  { @Set.PartialOrder α with
    Inf      := fun X ↦ {a | ∀A, A ∈ X → a ∈ A}
    Inf_le   := by aesop
    le_Inf   := by aesop }

```

11.14 最小不动点

```

def lfp {α : Type} [CompleteLattice α] (f : α → α) : α :=
  CompleteLattice.Inf {a | f a ≤ a}

```

```

theorem lfp_le {α : Type} [CompleteLattice α] (f : α → α)
  (a : α) (h : f a ≤ a) :
  lfp f ≤ a :=
  CompleteLattice.Inf_le _ _ h

```

```

theorem le_lfp {α : Type} [CompleteLattice α] (f : α → α)
  (a : α) (h : ∀a', f a' ≤ a' → a ≤ a') :
  a ≤ lfp f :=
  CompleteLattice.le_Inf _ _ h

```

Knaster-Tarski 定理: 对任意单调函数 f :

- $\text{lfp } f$ 是一个不动点: $\text{lfp } f = f (\text{lfp } f)$ (定理 `lfp_eq`);
- $\text{lfp } f$ 小于任何其他不动点: $X = f X \rightarrow \text{lfp } f \leq X$ 。

```

theorem lfp_eq {α : Type} [CompleteLattice α] (f : α → α)
  (hf : Monotone f) :

```



```

lfp f = f (lfp f) :=
by
have h : f (lfp f) ≤ lfp f :=
  by
    apply le_lfp
    intro a' ha'
    apply le_trans
    · apply hf
    apply lfp_le
    assumption
    · assumption
  apply le_antisymm
  · apply lfp_le
  apply hf
  assumption
  · assumption

```

11.15 关系式指称语义, 续

```

def denote : Stmt → Set (State × State)
| Stmt.skip      => Id
| Stmt.assign x a  => {(s, t) | t = s[x ↦ a s]}
| Stmt.seq S T    => denote S ○ denote T
| Stmt.ifThenElse B S T =>
  (denote S ↓ B) ∪ (denote T ↓ (fun s ↦ ¬ B s))
| Stmt.whileDo B S  =>
  lfp (fun X ↦ ((denote S ○ X) ↓ B)
    ∪ (Id ↓ (fun s ↦ ¬ B s)))

```

notation (priority := high) "⟦ S ⟧" => denote S

```

theorem Monotone_while_lfp_arg (S B) :
  Monotone (fun X ↦ ⟦S⟧ ○ X ↓ B ∪ Id ↓ (fun s ↦ ¬ B s)) :=
by
  apply Monotone_union
  · apply SorryTheorems.Monotone_restrict
  apply SorryTheorems.Monotone_comp

```

- exact Monotone_const _
 - exact Monotone_id
 - apply SorryTheorems.Monotone_restrict
- exact Monotone_const _

11.16 程序等价性的应用

基于指称语义，我们引入程序等价的概念： $S_1 \sim S_2$ 。（与习题 9 比较。）

```
def DenoteEquiv (S1 S2 : Stmt) : Prop :=
  [[S1]] = [[S2]]
```

```
infix:50 (priority := high) " ~ " => DenoteEquiv
```

从定义可明显看出， $\sim$ 是一个等价关系。

程序等价性可用于把子程序替换为具有相同语义的其他子程序。下面的同余规则实现了这一点：

```
theorem DenoteEquiv.seq_congr {S1 S2 T1 T2 : Stmt}
  (hS : S1 ~ S2) (hT : T1 ~ T2) :
  S1; T1 ~ S2; T2 :=
by
  simp [DenoteEquiv, denote] at *
  simp [*]
```

```
theorem DenoteEquiv.if_congr {B} {S1 S2 T1 T2 : Stmt}
  (hS : S1 ~ S2) (hT : T1 ~ T2) :
  Stmt.ifThenElse B S1 T1 ~ Stmt.ifThenElse B S2 T2 :=
by
  simp [DenoteEquiv, denote] at *
  simp [*]
```

```
theorem DenoteEquiv.while_congr {B} {S1 S2 : Stmt}
  (hS : S1 ~ S2) :
  Stmt.whileDo B S1 ~ Stmt.whileDo B S2 :=
by
  simp [DenoteEquiv, denote] at *
  simp [*]
```

将这些证明的简洁性与大步语义中相应证明进行比较（习题 8）。

让我们证明一些程序等价。

```
theorem DenoteEquiv.skip_assign_id {x} :
  Stmt.assign x (fun s ↦ s x) ~ Stmt.skip :=
  by simp [DenoteEquiv, denote, Id]

theorem DenoteEquiv.seq_skip_left {S} :
  Stmt.skip; S ~ S :=
  by simp [DenoteEquiv, denote, Id, comp]

theorem DenoteEquiv.seq_skip_right {S} :
  S; Stmt.skip ~ S :=
  by simp [DenoteEquiv, denote, Id, comp]

theorem DenoteEquiv.if_seq_while {B S} :
  Stmt.ifThenElse B (S; Stmt.whileDo B S) Stmt.skip
  ~ Stmt.whileDo B S :=
  by
    simp [DenoteEquiv, denote]
    apply Eq.symm
    apply lfp_eq
    apply Monotone_while_lfp_arg
```

11.17 指称语义与大步语义的等价性

11.18 （可选）

```
theorem denote_of_BigStep (Ss : Stmt × State) (t : State)
  (h : Ss ⇒ t) :
  (Prod.snd Ss, t) ∈ ⟦Prod.fst Ss⟧ :=
  by
    induction h with
    | skip s => simp [denote]
    | assign x a s => simp [denote]
```

```

| seq S T s t u hS hT ihS ihT =>
  simp [denote]
  aesop
| if_true B S T s t hB hS ih =>
  simp at *
  simp [denote, *]
| if_false B S T s t hB hT ih =>
  simp at *
  simp [denote, *]
| while_true B S s t u hB hS hw ihS ihw =>
  rw [Eq.symm DenoteEquiv.if_seq_while]
  simp at *
  simp [denote, *]
  aesop
| while_false B S s hB =>
  rw [Eq.symm DenoteEquiv.if_seq_while]
  simp at *
  simp [denote, *]

```

theorem BigStep_of_denote :

```

  ∀S : Stmt, ∀s t : State, (s, t) ∈ ⟦S⟧ → (S, s) ⇒ t
| Stmt.skip,      s, t => by simp [denote]
| Stmt.assign x a, s, t => by simp [denote]
| Stmt.seq S T,   s, t =>
  by
    intro hst
    simp [denote] at hst
    cases hst with
    | intro u hu =>
      cases hu with
      | intro hsu hut =>
        apply BigStep.seq
        · exact BigStep_of_denote _ _ _ hsu
        · exact BigStep_of_denote _ _ _ hut
| Stmt.ifThenElse B S T, s, t =>
  by
    intro hst
    simp [denote] at hst
    cases hst with

```

```

| inl htrue =>
  cases htrue with
| intro hst hB =>
  apply BigStep.if_true
  · exact hB
  · exact BigStep_of_denote ___ hst
| inr hfalse =>
  cases hfalse with
| intro hst hB =>
  apply BigStep.if_false
  · exact hB
  · exact BigStep_of_denote ___ hst
| Stmt.whileDo B S, s, t =>
  by
  have hw :  $\llbracket \text{Stmt.whileDo } B \ S \rrbracket$ 
     $\leq \{st \mid (\text{Stmt.whileDo } B \ S, \text{Prod.fst } st) \Rightarrow \text{Prod.snd } st\} :=$ 
  by
  apply lfp_le ___
  intro uv huv
  cases uv with
| mk u v =>
  simp at huv
  cases huv with
| inl hand =>
  cases hand with
| intro hst hB =>
  cases hst with
| intro w hw =>
  cases hw with
| intro huw hw =>
  apply BigStep.while_true
  · exact hB
  · exact BigStep_of_denote ___ huw
  · exact hw
| inr hand =>
  cases hand with
| intro hvu hB =>
  cases hvu

```

```

      apply BigStep.while_false
    exact hB
  apply hw

```

```

theorem denote_iff_BigStep (S : Stmt) (s t : State) :
  (s, t) ∈ ⟦S⟧ ↔ (S, s) ⟹ t :=
  Iff.intro (BigStep_of_denote S s t) (denote_of_BigStep (S, s) t)

```

11.19 基于归纳谓词的更简单方法 (可选)

```

inductive Awhile (B : State → Prop)
  (r : Set (State × State)) :
  State → State → Prop
where
  | true {s t u} (hcond : B s) (hbody : (s, t) ∈ r)
    (hrest : Awhile B r t u) :
    Awhile B r s u
  | false {s} (hcond : ¬ B s) :
    Awhile B r s s

def denoteAwhile : Stmt → Set (State × State)
  | Stmt.skip      => Id
  | Stmt.assign x a  => {(s, t) | t = s[x ↦ a s]}
  | Stmt.seq S T    => denoteAwhile S ○ denoteAwhile T
  | Stmt.ifThenElse B S T =>
    (denoteAwhile S ↓ B)
    u (denoteAwhile T ↓ (fun s ↦ ¬ B s))
  | Stmt.whileDo B S  =>
    {st | Awhile B (denoteAwhile S) (Prod.fst st)
      (Prod.snd st)}

```

第十二章 LoVe 演示 12: 数学的逻辑基础

我们将更深入地考察 **Lean** 的逻辑基础。这里描述的大多数特性，尤其与定义数学对象以及证明关于这些对象的定理有关。

12.1 宇宙

不仅项具有类型，类型本身也具有类型。例如，

```
`@And.intro :  $\forall a\ b, a \rightarrow b \rightarrow a \wedge b`$ 
```

并且

```
` $\forall a\ b, a \rightarrow b \rightarrow a \wedge b$  : Prop`
```

那么，**Prop** 的类型是什么？**Prop** 与我们目前构造出的几乎所有其他类型具有相同的类型：

```
`Prop : Type`
```

Type 的类型又是什么？若有类型标注 **Type : Type**，就会导出一个矛盾，称为 **Girard 悖论**；它类似于 **Russell 悖论**。因此实际情形是：

```
`Type : Type 1`
```

```
`Type 1 : Type 2`
```

```
`Type 2 : Type 3`
```

```
⋮
```

别名：

```
`Type` := `Type 0`
```

```
`Prop` := `Sort 0`
```

```
`Type u` := `Sort (u + 1)`
```

类型的类型（**Sort u**、**Type u** 与 **Prop**）称为 __ 宇宙 __。Sort u 中的 u 称为 __ 宇宙层级 __。

该层级由如下类型判定刻画：

Sort
 $C \vdash \text{Sort } u : \text{Sort } (u + 1)$

#check @And.intro
 #check $\forall a \ b : \text{Prop}, a \rightarrow b \rightarrow a \wedge b$
 #check Prop
 #check $\mathbb{N}$
 #check Type
 #check Type 1
 #check Type 2

universe u v

#check Type u

#check Sort 0
 #check Sort 1
 #check Sort 2
 #check Sort u

#check Type _

12.2 Prop 的特殊性

Prop 在许多方面不同于其他宇宙。

12.2.1 非谓词性

函数类型 $\sigma \rightarrow \tau$ 被置于 σ 与 τ 所在两个宇宙中较大的那个宇宙:

$C \vdash \sigma : \text{Type } u \quad C \vdash \tau : \text{Type } v$

SimpleArrow-Type

$C \vdash \sigma \rightarrow \tau : \text{Type } (\max u \ v)$

对于依赖类型, 这一规则推广为

$C \vdash \sigma : \text{Type } u \quad C, x : \sigma \vdash \tau[x] : \text{Type } v$

Arrow-Type

$$C \vdash (x : \sigma) \rightarrow \tau[x] : \text{Type} (\max u v)$$

宇宙 $\text{Type } v$ 的这种行称为 __ 谓词性 __。

为了仍然强制使诸如 $\forall n : \mathbb{N}, n = n$ 的表达式具有类型 Prop , 我们需要一条针对 Prop 的特殊类型规则:

$$C \vdash \sigma : \text{Sort } u \quad x : \sigma \vdash \tau[x] : \text{Prop} \quad \text{-----} \quad \text{Arrow-Prop}$$

$$C \vdash (\forall x : \sigma, \tau[x]) : \text{Prop}$$

Prop 的这种行称为 __ 非谓词性 __。

规则 Arrow-Type 与 Arrow-Prop 可以推广为一条统一规则:

$$C \vdash \sigma : \text{Sort } u \quad C, x : \sigma \vdash \tau[x] : \text{Sort } v \quad \text{-----} \quad \text{Arrow}$$

$$C \vdash (x : \sigma) \rightarrow \tau[x] : \text{Sort} (\text{imax } u v)$$

其中

$$\text{`imax } u 0 = 0`$$

$$\text{`imax } u (v + 1) = \max u (v + 1)`$$

```
#check fun (α : Type u) (β : Type v) ↦ α → β
```

```
#check ∀n : ℕ, n = n
```

12.2.2 证明无关性

Prop 与 $\text{Type } u$ 的第二个差异是 __ 证明无关性 __:

$$\text{`}\forall (a : \text{Prop}) (h_1 h_2 : a), h_1 = h_2\text{'}$$

这使得关于依赖类型的推理更容易。

若把命题看作类型、把证明看作该类型的一个元素, 则证明无关性意味着一个命题要么是空类型, 要么恰有一个居民。

证明无关性可以由 `rfl` 证明。

证明无关性的一个不便后果是: 它阻止我们通过模式匹配和递归来执行规则归纳。

```
#check proof_irrel
```

```
theorem proof_irrel {a : Prop} (h1 h2 : a) :
  h1 = h2 :=
  by rfl
```

12.2.3 无大消去

Prop 与 **Type u** 的进一步差异是: **Prop** 不允许 __ 大消去 __; 这意味着不能从一个命题的证明中提取数据。

这是为了允许证明无关性所必需的。

```
-fails def unsquare (i : ℤ) (hsq : ∃j, i = j * j) : ℤ := match hsq with | Exists.intro j _ => j
```

如果上面的定义被接受, 我们便可以如下推出 **False**。

令

```
`hsq1` := `Exists.intro 3 (by linarith)`
`hsq2` := `Exists.intro (-3) (by linarith)`
```

为 $\exists j, (9 : \mathbb{Z}) = j * j$ 的两个证明。于是

```
`unsquare 9 hsq1 = 3`
`unsquare 9 hsq2 = -3`
```

然而, 由证明无关性知 $hsq1 = hsq2$ 。因此

```
`unsquare 9 hsq2 = 3`
```

从而

```
`3 = -3`
```

这给出矛盾。

作为折中, **Lean** 允许 __ 小消去 __。它之所以称为小消去, 是因为它只消去到 **Prop**, 而大消去可以消去到任意大的宇宙 **Sort u**。这意味着, 只要 **match** 表达式本身仍是一个证明, 我们就可以用 **match** 分析证明的结构、提取存在量词的见证, 等等。

作为进一步的折中, **Lean** 对 __ 句法单元素类型 __ 允许大消去: 这些是在 **Prop** 中的类型, **Lean** 能够从句法上确定其基数为 0 或 1。这包括诸如 **False** 与 $a \wedge b$ 的命题; 它们至多能以一种方式被证明。

12.3 选择公理

Lean 的逻辑包含选择公理；它使我们能够从任意非空类型中取得一个任意元素。

考虑 Lean 的归纳谓词 `Nonempty`：

```
#print Nonempty
```

该谓词断言 α 至少有一个元素。

为了证明 `Nonempty  $\alpha$` ，我们必须向 `Nonempty.intro` 提供一个 α 的值：

```
theorem Nat.Nonempty :
```

```
  Nonempty  $\mathbb{N}$  :=
```

```
  Nonempty.intro 0
```

由于 `Nonempty` 处于 `Prop` 中，大消去不可用，因此我们不能从 `Nonempty  $\alpha$` 的证明中提取用于证明它的那个元素。

选择公理允许我们在能够证明 `Nonempty  $\alpha$` 时，取得某个类型为 α 的元素：

```
#check Classical.choice
```

它只会给出 α 的某个任意元素；我们无法知道这是否就是用来证明 `Nonempty  $\alpha$` 的那个元素。

常量 `Classical.choice` 是非可计算的，这就是一些逻辑学家偏好在不使用该公理的系统中工作的原因。

```
#eval Classical.choice Nat.Nonempty -fails
```

```
#reduce Classical.choice Nat.Nonempty
```

选择公理只是在 Lean 核心库中的一个公理，这给用户留下了使用或不使用它的自由。

使用它的定义必须标记为 `noncomputable`：

```
noncomputable def arbitraryNat :  $\mathbb{N}$  :=
```

```
  Classical.choice Nat.Nonempty
```

下面的工具依赖于选择。

12.3.1 排中律

```
#check Classical.em
```

12.3.2 Hilbert 选择

```
#check Classical.choose
#check Classical.choose_spec
```

12.3.3 集合论选择公理

```
#check Classical.axiomOfChoice
```

12.4 子类型

子类型化是一种从既有类型创建新类型的机制。

给定基类型元素上的一个谓词, `__子类型__` 只包含基类型中满足该性质的元素。更准确地说, 子类型包含“元素-证明”对: 它把基类型中的一个元素与该元素满足该性质的一个证明结合起来。

对于那些无法定义为归纳类型的类型, 子类型化很有用。例如, 任何试图按照如下方式定义有限集类型的尝试都注定失败:

```
-- wrong
inductive Finset (α : Type) : Type where
  | empty : Finset α
  | insert : α → Finset α → Finset α
```

为什么这并不能刻画有限集?

给定一个基类型与一个性质, 子类型的语法为

```
`{` _variable_ `:` _base-type_ `//` _property-applied-to-variable_ `}``
```

别名:

```
`{x :  $\tau$  // P[x]}` := `@Subtype  $\tau$  (fun x  $\mapsto$  P[x])`
```

例子:

```
`{i :  $\mathbb{N}$  // i  $\leq$  n}`      := `@Subtype  $\mathbb{N}$  (fun i  $\mapsto$  i  $\leq$  n)`  

`{a :  $\alpha$  // a  $\in$  A}`      := `@Subtype  $\alpha$  (fun a  $\mapsto$  a  $\in$  A)`  

`{A : Set  $\alpha$  // Set.Finite A}` := `@Subtype (Set  $\alpha$ ) Set.Finite`
```

12.4.1 第一个例子: 满二叉树

```
#check Tree  
#check IsFull  
#check mirror  
#check IsFull_mirror  
#check mirror_mirror
```

```
def FullTree ( $\alpha$  : Type) : Type :=  
  {t : Tree  $\alpha$  // IsFull t}
```

```
#print Subtype  
#check Subtype.mk
```

为了定义 `FullTree` 的元素, 我们必须提供一个 `Tree`, 以及它是满的证明:

```
def nilFullTree : FullTree  $\mathbb{N}$  :=  
  Subtype.mk Tree.nil IsFull.nil  
  
def fullTree6 : FullTree  $\mathbb{N}$  :=  
  Subtype.mk (Tree.node 6 Tree.nil Tree.nil)  
  (by  
    apply IsFull.node  
    apply IsFull.nil  
    apply IsFull.nil  
    rfl)
```

```
#reduce Subtype.val fullTree6  
#check Subtype.property fullTree6
```

我们可以把 `Tree` 上既有的运算提升到 `FullTree`:

```
def FullTree.mirror {α : Type} (t : FullTree α) :
  FullTree α :=
  Subtype.mk (LoVe.mirror (Subtype.val t))
  (by
    apply IsFull_mirror
    apply Subtype.property t)

#reduce Subtype.val (FullTree.mirror fullTree6)
```

当然, 我们也可以证明关于被提升运算的定理:

```
theorem FullTree.mirror_mirror {α : Type}
  (t : FullTree α) :
  (FullTree.mirror (FullTree.mirror t)) = t :=
  by
    apply Subtype.eq
    simp [FullTree.mirror, LoVe.mirror_mirror]

#check Subtype.eq
```

12.4.2 第二个例子: 向量

```
def Vector (α : Type) (n : ℕ) : Type :=
  {xs : List α // List.length xs = n}

def vector123 : Vector ℤ 3 :=
  Subtype.mk [1, 2, 3] (by rfl)

def Vector.neg {n : ℕ} (v : Vector ℤ n) : Vector ℤ n :=
  Subtype.mk (List.map Int.neg (Subtype.val v))
  (by
    rw [List.length_map]
    exact Subtype.property v)

theorem Vector.neg_neg (n : ℕ) (v : Vector ℤ n) :
  Vector.neg (Vector.neg v) = v :=
  by
```

```
apply Subtype.eq
simp [Vector.neg]
```

12.5 商类型

商是在数学中非常有力的构造，可用于构造 $\mathbb{Z}$ 、 $\mathbb{Q}$ 、 $\mathbb{R}$ 以及许多其他类型。

与子类型化一样，取商也是从既有类型构造新类型。不同的是，子类型只含有基类型中满足某性质的元素；而商类型含有基类型的所有元素，只是基类型中原本不同的某些元素在商类型中被视为相等。用数学语言说，商类型同构于基类型的一个划分。

为了定义商类型，我们需要提供作为来源的类型，以及该类型上的一个等价关系；该关系决定哪些元素被视为相等。

```
#check Quotient
#print Setoid
```

```
#check Quotient.mk
#check Quotient.sound
#check Quotient.exact
```

```
#check Quotient.lift
#check Quotient.lift₂
#check @Quotient.inductionOn
```

12.6 第一个例子：整数

我们把整数 $\mathbb{Z}$ 构造为自然数对 $\mathbb{N} \times \mathbb{N}$ 上的一个商。

自然数对 (p, n) 表示整数 $p - n$ 。非负整数 p 可表示为 $(p, 0)$ ；负整数 $-n$ 可表示为 $(0, n)$ 。然而，同一个整数可能有许多表示；例如 $(7, 0)$ 、 $(8, 1)$ 、 $(9, 2)$ 与 $(10, 3)$ 都表示整数 7。

我们可以使用哪个等价关系？

我们希望当 $p_1 - n_1 = p_2 - n_2$ 时，两个对 (p_1, n_1) 与 (p_2, n_2) 相等。然而，这行不通，因为 $\mathbb{N}$ 上的减法行为不好（例如 $0 - 1 = 0$ ）。因此我们使用 $p_1 + n_2 = p_2 + n_1$ 。

```
instance Int.Setoid : Setoid (ℕ × ℕ) :=
```

```

{ r :=
  fun pn1 pn2 :  $\mathbb{N} \times \mathbb{N}$   $\mapsto$ 
    Prod.fst pn1 + Prod.snd pn2 =
    Prod.fst pn2 + Prod.snd pn1
iseqv :=
  { refl :=
    by
      intro pn
      rfl
  symm :=
    by
      intro pn1 pn2 h
      rw [h]
  trans :=
    by
      intro pn1 pn2 pn3 h12 h23
      linarith } }

```

```

theorem Int.Setoid_Iff (pn1 pn2 :  $\mathbb{N} \times \mathbb{N}$ ) :
  pn1  $\approx$  pn2  $\leftrightarrow$ 
  Prod.fst pn1 + Prod.snd pn2 =
  Prod.fst pn2 + Prod.snd pn1 :=
  by rfl

```

```

def Int : Type :=
  Quotient Int.Setoid

```

```

def Int.zero : Int :=
  [(0, 0)]

```

```

theorem Int.zero_Eq (m :  $\mathbb{N}$ ) :
  Int.zero = [(m, m)] :=
  by
    rw [Int.zero]
    apply Quotient.sound
    rw [Int.Setoid_Iff]
    simp

```

```

def Int.add : Int  $\rightarrow$  Int  $\rightarrow$  Int :=

```


Quotient.lift₂

```
(fun pn1 pn2 :  $\mathbb{N} \times \mathbb{N} \mapsto$ 
   $\llbracket (\text{Prod.fst } pn_1 + \text{Prod.fst } pn_2,$ 
     $\text{Prod.snd } pn_1 + \text{Prod.snd } pn_2) \rrbracket$ )
(by
  intro pn1 pn2 pn1' pn2' h1 h2
  apply Quotient.sound
  rw [Int.Setoid_iff] at *
  linarith)
```

```
theorem Int.add_Eq (p1 n1 p2 n2 :  $\mathbb{N}$ ) :
  Int.add  $\llbracket (p_1, n_1) \rrbracket$   $\llbracket (p_2, n_2) \rrbracket$  =
   $\llbracket (p_1 + p_2, n_1 + n_2) \rrbracket$  :=
  by rfl
```

```
theorem Int.add_zero (i : Int) :
  Int.add Int.zero i = i :=
  by
    induction i using Quotient.inductionOn with
    | h pn =>
      cases pn with
      | mk p n => simp [Int.zero, Int.add]
```

这样的定义语法会很好用:

```
-fails def Int.add : Int → Int → Int |  $\llbracket (p_1, n_1) \rrbracket, \llbracket (p_2, n_2) \rrbracket$  =>  $\llbracket (p_1 + p_2, n_1 + n_2) \rrbracket$ 
```

但是这会有危险:

```
-fails def Int.fst : Int →  $\mathbb{N}$  |  $\llbracket (p, n) \rrbracket$  => p
```

利用 Int.fst, 我们能够推出 **False**。首先, 我们有

```
`Int.fst  $\llbracket (0, 0) \rrbracket$  = 0`
`Int.fst  $\llbracket (1, 1) \rrbracket$  = 1`
```

但由于 $\llbracket (0, 0) \rrbracket = \llbracket (1, 1) \rrbracket$, 便得到

```
`0 = 1`
```

12.6.1 第二个例子: 无序对

__ 无序对 __ 是不区分第一分量与第二分量的对。它们通常写作 $\{a, b\}$ 。

我们将引入无序对类型 `UPair`, 即以“含有相同元素”这一关系, 对对 (a, b) 取商。

```
instance UPair.Setoid ( $\alpha$  : Type) : Setoid ( $\alpha \times \alpha$ ) :=
{ r :=
  fun ab1 ab2 :  $\alpha \times \alpha$   $\mapsto$ 
    ({Prod.fst ab1, Prod.snd ab1} : Set  $\alpha$ ) =
    ({Prod.fst ab2, Prod.snd ab2} : Set  $\alpha$ )
  iseqv :=
    { refl := by simp
      symm := by aesop
      trans := by aesop } }
```

```
theorem UPair.Setoid_Iff { $\alpha$  : Type} (ab1 ab2 :  $\alpha \times \alpha$ ) :
  ab1  $\approx$  ab2  $\leftrightarrow$ 
  ({Prod.fst ab1, Prod.snd ab1} : Set  $\alpha$ ) =
  ({Prod.fst ab2, Prod.snd ab2} : Set  $\alpha$ ) :=
  by rfl
```

```
def UPair ( $\alpha$  : Type) : Type :=
  Quotient (UPair.Setoid  $\alpha$ )
```

```
#check UPair.Setoid
```

很容易证明, 我们的对确实是无序的:

```
theorem UPair.mk_symm { $\alpha$  : Type} (a b :  $\alpha$ ) :
  ([[(a, b)]] : UPair  $\alpha$ ) = [[(b, a)]] :=
  by
    apply Quotient.sound
    rw [UPair.Setoid_Iff]
    aesop
```

无序对的另一种表示是基数为 1 或 2 的集合。下面的运算把 `UPair` 转换为这种表示:

```
def Set_of_UPair { $\alpha$  : Type} : UPair  $\alpha$   $\rightarrow$  Set  $\alpha$  :=
  Quotient.lift (fun ab :  $\alpha \times \alpha$   $\mapsto$  {Prod.fst ab, Prod.snd ab})
  (by
    intro ab1 ab2 h
    rw [UPair.Setoid_Iff] at *
```

exact h)

12.6.2 通过规范化与子类型化给出的替代定义

商类型的每个元素对应一个 $\approx$ -等价类。若存在一种系统的方法，能够为每个等价类取得一个规范代表元，则我们可以用子类型代替商，只保留这些规范代表元。

考虑上面的商类型 `Int`。我们可以说，一个对 (p, n) 是 __ 规范的 __，当且仅当 p 或 n 为 0。

```
inductive Int.IsCanonical :  $\mathbb{N} \times \mathbb{N} \rightarrow \text{Prop}$  where
  | nonpos {n :  $\mathbb{N}$ } : Int.IsCanonical (0, n)
  | nonneg {p :  $\mathbb{N}$ } : Int.IsCanonical (p, 0)
```

```
def Int : Type :=
  {pn :  $\mathbb{N} \times \mathbb{N}$  // Int.IsCanonical pn}
```

对自然数对进行规范化很容易：

```
def Int.normalize :  $\mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N} \times \mathbb{N}$ 
  | (p, n) => if p  $\geq$  n then (p - n, 0) else (0, n - p)
```

```
theorem Int.IsCanonical_normalize (pn :  $\mathbb{N} \times \mathbb{N}$ ) :
  Int.IsCanonical (Int.normalize pn) :=
  by
    cases pn with
    | mk p n =>
      simp [Int.normalize]
      cases Classical.em (p  $\geq$  n) with
      | inl hpn =>
        simp [*]
        exact Int.IsCanonical.nonneg
      | inr hpn =>
        simp [*]
        exact Int.IsCanonical.nonpos
```

对于无序对，除了总是把较小的元素放在前面（或后面）之外，并没有显然的标准形。这要求在 α 上有一个线性序 $\leq$ 。

```
def UPair.IsCanonical { $\alpha$  : Type} [LinearOrder  $\alpha$ ] :
```

$$\alpha \times \alpha \rightarrow \text{Prop}$$
$$| (a, b) \Rightarrow a \leq b$$

```
def UPair (α : Type) [LinearOrder α] : Type :=  
  {ab : α × α // UPair.IsCanonical ab}
```

第十三章 LoVe 演示 13: 基本数学结构

我们介绍关于基本数学结构的定义与证明，例如群、域和线性序。

13.1 单个二元运算上的类型类

在数学上，__ 群 __ 是一个集合 G ，配备一个二元运算 $\cdot : G \times G \rightarrow G$ ，并满足下列性质；这些性质称为群公理：

- 结合律：对所有 $a, b, c \in G$ ，有 $(a \cdot b) \cdot c = a \cdot (b \cdot c)$ ；
- 单位元：存在一个元素 $e \in G$ ，使得对所有 $a \in G$ ，有 $e \cdot a = a$ 且 $a \cdot e = a$ ；
- 逆元：对每个 $a \in G$ ，存在一个逆元素 $b \in G$ ，使得 $b \cdot a = e$ 且 $a \cdot b = e$ 。

群的例子包括 * 带 + 的 $\mathbb{Z}$ ；* 带 + 的 $\mathbb{R}$ ；* 带 * 的 $\mathbb{R} \setminus \{0\}$ 。

在 Lean 中，群的类型类可以如下定义：

```
class Group (α : Type) where
  mul      : α → α → α
  one      : α
  inv      : α → α
  mul_assoc : ∀ a b c, mul (mul a b) c = mul a (mul b c)
  one_mul   : ∀ a, mul one a = a
  mul_left_inv : ∀ a, mul (inv a) a = one
```

不过，在 Lean 中，群是一个更大的代数结构层级的一部分：

类型类	性质	例子
Semigroup	* 的结合律	$\mathbb{R}, \mathbb{Q}, \mathbb{Z}, \mathbb{N}$
Monoid	带单位元 1 的 Semigroup	$\mathbb{R}, \mathbb{Q}, \mathbb{Z}, \mathbb{N}$
LeftCancelSemigroup	带 $c * a = c * b \rightarrow a = b$ 的 Semigroup	
RightCancelSemigroup	带 $a * c = b * c \rightarrow a = b$ 的 Semigroup	

类型类	性质	例子
Group	带逆元 $^{-1}$ 的 Monoid	

这些结构中的大多数都有交换版本: CommSemigroup、CommMonoid、CommGroup。

__ 乘法 __ 结构 (基于 $*$ 、 1 、 $^{-1}$) 会被复制, 以产生 __ 加法 __ 版本 (基于 $+$ 、 0 、 $-$):

类型类	性质	例子
AddSemigroup	$+$ 的结合律	$\mathbb{R}, \mathbb{Q}, \mathbb{Z}, \mathbb{N}$
AddMonoid	带单位元 0 的 AddSemigroup	$\mathbb{R}, \mathbb{Q}, \mathbb{Z}, \mathbb{N}$
AddLeftCancelSemigroup	带 $c + a = c + b \rightarrow a = b$ 的 AddSemigroup	$\mathbb{R}, \mathbb{Q}, \mathbb{Z}, \mathbb{N}$
AddRightCancelSemigroup	带 $a + c = b + c \rightarrow a = b$ 的 AddSemigroup	$\mathbb{R}, \mathbb{Q}, \mathbb{Z}, \mathbb{N}$
AddGroup	带逆元 $-$ 的 AddMonoid	$\mathbb{R}, \mathbb{Q}, \mathbb{Z}$

```
#print Group
#print AddGroup
```

我们来定义自己的类型, 即模 2 整数, 并把它注册为一个加法群。

```
inductive Int2 : Type where
  | zero
  | one

def Int2.add : Int2 → Int2 → Int2
  | Int2.zero, a      => a
  | Int2.one, Int2.zero => Int2.one
  | Int2.one, Int2.one  => Int2.zero

instance Int2.AddGroup : AddGroup Int2 :=
{ add      := Int2.add
  zero     := Int2.zero
  neg      := fun a => a
  add_assoc :=
  by
    intro a b c
    cases a <|>
```

```

      cases b <;>
      cases c <;>
      rfl
zero_add    :=
  by
    intro a
    cases a <;>
    rfl
add_zero    :=
  by
    intro a
    cases a <;>
    rfl
neg_add_cancel :=
  by
    intro a
    cases a <;>
    rfl
nsmul       :=
  @nsmulRec Int2 (Zero.mk Int2.zero) (Add.mk Int2.add)
zsmul       :=
  @zsmulRec Int2 (Zero.mk Int2.zero) (Add.mk Int2.add)
  (Neg.mk (fun a ↦ a))
  (@nsmulRec Int2 (Zero.mk Int2.zero) (Add.mk Int2.add)) }

```

`nsmul` 和 `zsmul` 是冗余的。出于技术原因，它们是必需的。

```
#reduce Int2.one + 0 - 0 - Int2.one
```

另一个例子：列表是一个 `AddMonoid`：

```

instance List.AddMonoid {α : Type} : AddMonoid (List α) :=
{ zero    := []
  add     := fun xs ys ↦ xs ++ ys
  add_assoc := List.append_assoc
  zero_add := List.nil_append
  add_zero := List.append_nil
  nsmul    :=
    @nsmulRec (List α) (Zero.mk [])

```

```
(Add.mk (fun xs ys ↦ xs ++ ys)))}
```

13.2 具有两个二元运算的类型类

在数学上，__ 域 __ 是一个集合 F ，使得

- F 在一个称为加法的运算 $+$ 下构成交换群，其单位元为 0 。
- $F \setminus \{0\}$ 在一个称为乘法的运算 $*$ 下构成交换群。
- 乘法对加法满足分配律，即对所有 $a, b, c \in F$ ， $a * (b + c) = a * b + a * c$ 。

在 Lean 中，域同样是一个更大层级的一部分：

类型类	性质	例子
Semiring	带分配律的 Monoid 和 AddCommMonoid	$\mathbb{R}, \mathbb{Q}, \mathbb{Z}, \mathbb{N}$
CommSemiring	* 交换的 Semiring	$\mathbb{R}, \mathbb{Q}, \mathbb{Z}, \mathbb{N}$
Ring	带分配律的 Monoid 和 AddCommGroup	$\mathbb{R}, \mathbb{Q}, \mathbb{Z}$
CommRing	* 交换的 Ring	$\mathbb{R}, \mathbb{Q}, \mathbb{Z}$
DivisionRing	带乘法逆元 $^{-1}$ 的 Ring	$\mathbb{R}, \mathbb{Q}$
Field	* 交换的 DivisionRing	$\mathbb{R}, \mathbb{Q}$

```
#print Field
```

我们继续前面的例子：

```
def Int2.mul : Int2 → Int2 → Int2
| Int2.one, a => a
| Int2.zero, _ => Int2.zero
```

```
instance Int2.Field : Field Int2 :=
{ Int2.AddGroup with
  one      := Int2.one
  mul      := Int2.mul
  inv      := fun a ↦ a
  add_comm :=
  by
    intro a b
    cases a <.>
    cases b <.>
```



```

    rfl
exists_pair_ne :=
  by
    apply Exists.intro Int2.zero
    apply Exists.intro Int2.one
    simp
zero_mul      :=
  by
    intro a
    rfl
mul_zero      :=
  by
    intro a
    cases a <;>
    rfl
one_mul       :=
  by
    intro a
    rfl
mul_one       :=
  by
    intro a
    cases a <;>
    rfl
mul_inv_cancel :=
  by
    intro a h
    cases a
    · apply False.elim
      apply h
      rfl
    · rfl
inv_zero      := by rfl
mul_assoc     :=
  by
    intro a b c
    cases a <;>
    cases b <;>
    cases c <;>

```

```

    rfl
mul_comm      :=
  by
    intro a b
    cases a <;>
    cases b <;>
    rfl
left_distrib  :=
  by
    intro a b c
    cases a <;>
    cases b <;>
    rfl
right_distrib :=
  by
    intro a b c
    cases a <;>
    cases b <;>
    cases c <;>
    rfl
nnqsmul      := _
nnqsmul_def  :=
  by
    intro a b
    rfl
qsmul        := _
qsmul_def    :=
  by
    intro a b
    rfl
nnratCast_def :=
  by
    intro q
    rfl }

#reduce (1 : Int2) * 0 / (0 - 1)

#reduce (3 : Int2)

```

```
theorem ring_example (a b : Int2) :
  (a + b) ^ 3 = a ^ 3 + 3 * a ^ 2 * b + 3 * a * b ^ 2 + b ^ 3
  :=
  by ring
```

ring 通过把表达式规范化，来证明交换环与半环上的等式。

13.3 强制转换

当组合来自 $\mathbb{N}$ 、 $\mathbb{Z}$ 、 $\mathbb{Q}$ 和 $\mathbb{R}$ 的数时，我们可能希望从一种类型转换到另一种类型。Lean 有一种机制，能够自动引入强制转换；它由 `Coe.coe` 表示（语法糖为 `↑`）。可以设置 `Coe.coe`，使其在任意类型之间提供隐式强制转换。

许多强制转换已经就位，包括：

- `Coe.coe :  $\mathbb{N} \rightarrow \alpha$` 把 $\mathbb{N}$ 转换为另一个半环 α ；
- `Coe.coe :  $\mathbb{Z} \rightarrow \alpha$` 把 $\mathbb{Z}$ 转换为另一个环 α ；
- `Coe.coe :  $\mathbb{Q} \rightarrow \alpha$` 把 $\mathbb{Q}$ 转换为另一个除环 α 。

例如，下面能够通过，即便自然数上并未定义取负 `- n`：

```
theorem neg_mul_neg_Nat (n :  $\mathbb{N}$ ) (z :  $\mathbb{Z}$ ) :
  (- z) * (- n) = z * n :=
  by simp
```

注意 Lean 如何引入了一个 `↑` 强制转换：

```
#check neg_mul_neg_Nat
```

类型标注可以记录我们的意图：

```
theorem neg_Nat_mul_neg (n :  $\mathbb{N}$ ) (z :  $\mathbb{Z}$ ) :
  (- n :  $\mathbb{Z}$ ) * (- z) = n * z :=
  by simp
```

```
#print neg_Nat_mul_neg
```

在涉及强制转换的证明中，策略 `norm_cast` 会很方便。

```
theorem Eq_coe_int_imp_Eq_Nat (m n :  $\mathbb{N}$ )
```

```

(h : (m : ℤ) = (n : ℤ)) :
m = n :=
by norm_cast at h

```

```

theorem Nat_coe_Int_add_eq_add_Nat_coe_Int (m n : ℕ) :
  (m : ℤ) + (n : ℤ) = ((m + n : ℕ) : ℤ) :=
by norm_cast

```

`norm_cast` 会把强制转换向表达式内部移动，这是一种简化形式。与 `simp` 一样，它常常会产生一个子目标。

`norm_cast` 依赖如下定理：

```

#check Nat.cast_add
#check Int.cast_add
#check Rat.cast_add

```

13.3.1 列表、多重集与有限集

对于由元素组成的有限集合状对象，可以使用不同的结构：

- 列表：次序和重复次数都重要；
- 多重集：只有重复次数重要；
- 有限集：次序和重复次数都不重要。

```

def List.elems : Tree ℕ → List ℕ
| Tree.nil      => []
| Tree.node a l r => a :: List.elems l ++ List.elems r

```

```

def Multiset.elems : Tree ℕ → Multiset ℕ
| Tree.nil      => ∅
| Tree.node a l r =>
  {a} ∪ Multiset.elems l ∪ Multiset.elems r

```

```

def Finset.elems : Tree ℕ → Finset ℕ
| Tree.nil      => ∅
| Tree.node a l r => {a} ∪ Finset.elems l ∪ Finset.elems r

```

```

#eval List.sum [2, 3, 4]

```

```
#eval Multiset.sum ({2, 3, 4} : Multiset ℕ)
```

```
#eval List.prod [2, 3, 4]
```

```
#eval Multiset.prod ({2, 3, 4} : Multiset ℕ)
```

13.4 序类型类

上面引入的许多结构都可以带序。例如，自然数上众所周知的序可以如下定义：

```
inductive Nat.le : ℕ → ℕ → Prop where
```

```
| refl : ∀ a : ℕ, Nat.le a a
```

```
| step : ∀ a b : ℕ, Nat.le a b → Nat.le a (b + 1)
```

```
#print Preorder
```

这是一个线性序的例子。线性序（或 __ 全序 __）是一个二元关系 $\leq$ ，使得对所有 a 、 b 、 c ，下列性质成立：

- 自反性： $a \leq a$ ；
- 传递性：若 $a \leq b$ 且 $b \leq c$ ，则 $a \leq c$ ；
- 反对称性：若 $a \leq b$ 且 $b \leq a$ ，则 $a = b$ ；
- 完全性： $a \leq b$ 或 $b \leq a$ 。

若一个关系具有前三个性质，则它是 __ 偏序 __。一个例子是集合、有限集或多重集上的 $\subseteq$ 。若一个关系具有前两个性质，则它是 __ 预序 __。一个例子是按照长度比较列表。

在 Lean 中，这些不同种类的序对应类型类：LinearOrder、PartialOrder 与 Preorder。

```
#print Preorder
```

```
#print PartialOrder
```

```
#print LinearOrder
```

我们可以如下声明列表上的预序，它通过列表长度来比较列表：

```
instance List.length.Preorder {α : Type} : Preorder (List α) :=
```

```
{ le := fun xs ys ↦ List.length xs ≤ List.length ys
```

```
lt := fun xs ys ↦ List.length xs < List.length ys
```

```
le_refl :=
```

```
by
```

```

    intro xs
    apply Nat.le_refl
le_trans :=
  by
    intro xs ys zs
    exact Nat.le_trans
lt_iff_le_not_ge :=
  by
    intro a b
    exact Nat.lt_iff_le_not_le }

```

完全格（第 11 讲）也被形式化为另一个类型类 `CompleteLattice`；它继承自 `PartialOrder`。

也存在把序与代数结构相结合的类型类：

```

`OrderedCancelCommMonoid`
`OrderedCommGroup`
`OrderedSemiring`
`LinearOrderedSemiring`
`LinearOrderedCommRing`
`LinearOrderedField`

```

所有这些数学结构都通过单调性规则（例如 $a \leq b \rightarrow c \leq d \rightarrow a + c \leq b + d$ ）和消去规则（例如 $c + a \leq c + b \rightarrow a \leq b$ ），把 $\leq$ 与 $<$ 同 0 、 1 、 $+$ 、 $*$ 关联起来。# LoVe 演示 14: 有理数与实数

我们回顾如何把 $\mathbb{Q}$ 与 $\mathbb{R}$ 构造为商类型。

我们用来构造具有特定性质的类型的一般步骤如下：

1. 创建一个能够表示所有元素的新类型，但这种表示不一定唯一。
2. 对该表示取商，把应当相等的元素等同起来。
3. 通过从基类型提升函数，在商类型上定义运算；并证明这些运算与商关系相容。

我们在第 12 讲中曾用这种方法构造 $\mathbb{Z}$ 。它同样可用于 $\mathbb{Q}$ 与 $\mathbb{R}$ 。

13.5 有理数

第 1 步：有理数是能够表示为整数 n 与 $d \neq 0$ 的分数 n/d 的数：

```

structure Fraction where
  num      :  $\mathbb{Z}$ 

```

```
denom      :  $\mathbb{Z}$ 
denom_Neq_zero : denom  $\neq$  0
```

数 n 称为分子, 数 d 称为分母。

把有理数表示为分数并不是唯一的; 例如 $1/2 = 2/4 = -1/-2$ 。

第 2 步: 两个分数 n_1/d_1 与 n_2/d_2 表示同一个有理数, 当且仅当分子与分母之间的比值相同, 即 $n_1 * d_2 = n_2 * d_1$ 。这将作为分数上的等价关系 $\approx$ 。

```
instance Setoid : Setoid Fraction :=
{ r :=
  fun a b : Fraction  $\mapsto$  num a * denom b = num b * denom a
  iseqv :=
  { refl := by aesop
    symm := by aesop
    trans :=
      by
        intro a b c heq_ab heq_bc
        apply Int.eq_of_mul_eq_mul_right (denom_Neq_zero b)
        calc
          num a * denom c * denom b
          = num a * denom b * denom c :=
            by ac_rfl
          _ = num b * denom a * denom c :=
            by rw [heq_ab]
          _ = num b * denom c * denom a :=
            by ac_rfl
          _ = num c * denom b * denom a :=
            by rw [heq_bc]
          _ = num c * denom a * denom b :=
            by ac_rfl } }
```

```
theorem Setoid_iff (a b : Fraction) :
  a  $\approx$  b  $\leftrightarrow$  num a * denom b = num b * denom a :=
  by rfl
```

第 3 步: 定义 $0 := 0/1$ 、 $1 := 1/1$ 、加法、乘法, 等等。

```
`n1 / d1 + n2 / d2` := `(n1 * d2 + n2 * d1) / (d1 * d2)`
`(n1 / d1) * (n2 / d2)` := `(n1 * n2) / (d1 * d2)`
```

然后证明它们与 $\approx$ 相容。

```

def of_int (i :  $\mathbb{Z}$ ) : Fraction :=
  { num      := i
    denom    := 1
    denom_Neq_zero := by simp }

instance Zero : Zero Fraction :=
  { zero := of_int 0 }

instance One : One Fraction :=
  { one := of_int 1 }

instance Add : Add Fraction :=
  { add := fun a b : Fraction  $\mapsto$ 
    { num      := num a * denom b + num b * denom a
      denom    := denom a * denom b
      denom_Neq_zero := by simp [denom_Neq_zero] } }

@[simp] theorem add_num (a b : Fraction) :
  num (a + b) = num a * denom b + num b * denom a :=
  by rfl

@[simp] theorem add_denom (a b : Fraction) :
  denom (a + b) = denom a * denom b :=
  by rfl

theorem Setoid_add {a a' b b' : Fraction} (ha : a  $\approx$  a')
  (hb : b  $\approx$  b') :
  a + b  $\approx$  a' + b' :=
  by
    simp [Setoid_Iff, add_denom, add_num] at *
    calc
      (num a * denom b + num b * denom a)
        * (denom a' * denom b')
      = num a * denom a' * denom b * denom b'
        + num b * denom b' * denom a * denom a' :=
        by grind
      _ = num a' * denom a * denom b * denom b'

```



```

      + num b' * denom b * denom a * denom a' :=
    by simp [*]
  _ = (num a' * denom b' + num b' * denom a')
      * (denom a * denom b) :=
    by grind

```

```

instance Neg : Neg Fraction :=
{ neg := fun a : Fraction ↦
  { a with
    num := - num a } }

```

```

@[simp] theorem neg_num (a : Fraction) :
  num (- a) = - num a :=
  by rfl

```

```

@[simp] theorem neg_denom (a : Fraction) :
  denom (- a) = denom a :=
  by rfl

```

```

theorem Setoid_neg {a a' : Fraction} (hab : a ≈ a') :
  - a ≈ - a' :=
  by
    simp [Setoid_Iff] at *
    exact hab

```

```

instance Mul : Mul Fraction :=
{ mul := fun a b : Fraction ↦
  { num      := num a * num b
    denom    := denom a * denom b
    denom_Neq_zero := by simp [denom_Neq_zero] } }

```

```

@[simp] theorem mul_num (a b : Fraction) :
  num (a * b) = num a * num b :=
  by rfl

```

```

@[simp] theorem mul_denom (a b : Fraction) :
  denom (a * b) = denom a * denom b :=
  by rfl

```

```

theorem Setoid_mul {a a' b b' : Fraction} (ha : a ≈ a')
  (hb : b ≈ b') :
  a * b ≈ a' * b' :=
by
  simp [Setoid_iff] at *
  calc
    num a * num b * (denom a' * denom b')
    = (num a * denom a') * (num b * denom b') :=
      by ac_rfl
    _ = (num a' * denom a) * (num b' * denom b) :=
      by simp [*]
    _ = num a' * num b' * (denom a * denom b) :=
      by ac_rfl

instance Inv : Inv Fraction :=
{ inv := fun a : Fraction ↦
  if ha : num a = 0 then
    0
  else
    { num      := denom a
      denom    := num a
      denom_Neq_zero := ha } }

theorem inv_def (a : Fraction) (ha : num a ≠ 0) :
  a-1 =
  { num      := denom a
    denom    := num a
    denom_Neq_zero := ha } :=
dif_neg ha

theorem inv_zero (a : Fraction) (ha : num a = 0) :
  a-1 = 0 :=
dif_pos ha

@[simp] theorem inv_num (a : Fraction) (ha : num a ≠ 0) :
  num (a-1) = denom a :=
by rw [inv_def a ha]

@[simp] theorem inv_denom (a : Fraction) (ha : num a ≠ 0) :

```

```
denom (a-1) = num a :=
by rw [inv_def a ha]
```

```
theorem Setoid_inv {a a' : Fraction} (ha : a ≈ a') :
  a-1 ≈ a'-1 :=
by
  cases Classical.em (num a = 0) with
  | inl ha0 =>
    cases Classical.em (num a' = 0) with
    | inl ha'0 =>
      simp [ha0, ha'0, inv_zero]
    | inr ha'0 =>
      simp [ha0, ha'0, Setoid_Iff, denom_Neq_zero] at ha
  | inr ha0 =>
    cases Classical.em (num a' = 0) with
    | inl ha'0 =>
      simp [ha0, ha'0, Setoid_Iff, denom_Neq_zero] at ha
    | inr ha'0 =>
      simp [Setoid_Iff, ha0, ha'0] at *
      linarith
```

```
def Rat : Type :=
  Quotient Fraction.Setoid
```

```
def mk : Fraction → Rat :=
  Quotient.mk Fraction.Setoid
```

```
instance Zero : Zero Rat :=
  { zero := mk 0 }
```

```
instance One : One Rat :=
  { one := mk 1 }
```

```
instance Add : Add Rat :=
  { add := Quotient.lift₂ (fun a b : Fraction ↦ mk (a + b))
    (by
      intro a b a' b' ha hb
```

```

    apply Quotient.sound
    exact Fraction.Setoid_add ha hb) }

```

```

instance Neg : Neg Rat :=
{ neg := Quotient.lift (fun a : Fraction ↦ mk (- a))
  (by
    intro a a' ha
    apply Quotient.sound
    exact Fraction.Setoid_neg ha) }

```

```

instance Mul : Mul Rat :=
{ mul := Quotient.lift₂ (fun a b : Fraction ↦ mk (a * b))
  (by
    intro a b a' b' ha hb
    apply Quotient.sound
    exact Fraction.Setoid_mul ha hb) }

```

```

instance Inv : Inv Rat :=
{ inv := Quotient.lift (fun a : Fraction ↦ mk (a⁻¹))
  (by
    intro a a' ha
    apply Quotient.sound
    exact Fraction.Setoid_inv ha) }

```

13.5.1 $\mathbb{Q}$ 的替代定义

把 $\mathbb{Q}$ 定义为 `fraction` 的一个子类型, 要求分母为正, 并且分子与分母除了 **1** 和 **-1** 之外没有公共因子:

```

def Rat.IsCanonical (a : Fraction) : Prop :=
  Fraction.denom a > 0
  ∧ Nat.Coprime (Int.natAbs (Fraction.num a))
    (Int.natAbs (Fraction.denom a))

```

```

def Rat : Type :=
{a : Fraction // Rat.IsCanonical a}

```

这大致就是 `mathlib` 中的定义。

优点:

- 不需要商;
- 计算更高效;
- 更多性质在计算意义下就是句法等式。

缺点:

- 函数定义更复杂。

13.5.2 实数

某些有理数序列看起来会收敛, 因为序列中的数彼此越来越接近; 但它们并不收敛到一个有理数。

例子:

```
`a0 = 1`
`a1 = 1.4`
`a2 = 1.41`
`a3 = 1.414`
`a4 = 1.4142`
`a5 = 1.41421`
`a6 = 1.414213`
`a7 = 1.4142135`
⋮
```

这个序列看起来会收敛, 因为每个 a_n 与后面任意一个数的距离至多为 10^{-n} 。但它的极限是 $\sqrt{2}$, 而这不是有理数。

有理数是不完备的, 而实数是有理数的 __ 完备化 __。

为了构造实数, 我们需要填补那些由“看起来会收敛但实际上没有有理极限”的序列所揭示出来的空隙。

在数学上, 一个有理数序列 $a_0, a_1, \dots$ 称为 __Cauchy 序列 __, 如果对任意 $\varepsilon > 0$, 存在一个 $N \in \mathbb{N}$, 使得对所有 $m \geq N$, 有 $|a_N - a_m| < \varepsilon$ 。

换言之, 无论我们把 ε 取得多么小, 总能在序列中找到一个位置, 使得从该位置起所有后续项与它的偏差都小于 ε 。

```
def IsCauchySeq (f :  $\mathbb{N} \rightarrow \mathbb{Q}$ ) : Prop :=
   $\forall \varepsilon > 0, \exists N, \forall m \geq N, \text{abs } (f\ N - f\ m) < \varepsilon$ 
```

并非每个序列都是 Cauchy 序列:

```

theorem id_Not_CauchySeq :
  ¬ IsCauchySeq (fun n : ℕ ↦ (n : ℚ)) :=
by
  rw [IsCauchySeq]
  intro h
  cases h 1 zero_lt_one with
  | intro i hi =>
    have hi_succ :=
      hi (i + 1) (by simp)
    simp at hi_succ

```

我们把 Cauchy 序列的类型定义为一个子类型:

```

def CauchySeq : Type :=
  {f : ℕ → ℚ // IsCauchySeq f}

def seqOf (f : CauchySeq) : ℕ → ℚ :=
  Subtype.val f

```

Cauchy 序列表示实数:

- $a_n = 1/n$ 表示实数 0;
- $1, 1.4, 1.41, \dots$ 表示实数 $\sqrt{2}$;
- $a_n = 0$ 也表示实数 0。

由于不同的 Cauchy 序列可以表示同一个实数, 我们需要取商。形式化地说, 两个序列表示同一个实数, 当且仅当它们的差收敛到零:

```

instance Setoid : Setoid CauchySeq :=
{ r :=
  fun f g : CauchySeq ↦
    ∀ε > 0, ∃N, ∀m ≥ N, abs (seqOf f m - seqOf g m) < ε
  iseqv :=
    { refl :=
      by
        intro f ε hε
        apply Exists.intro 0
        aesop
      symm :=

```

```

by
  intro f g hfg  $\varepsilon$  h $\varepsilon$ 
  cases hfg  $\varepsilon$  h $\varepsilon$  with
  | intro N hN =>
    apply Exists.intro N
    intro m hm
    rw [abs_sub_comm]
    apply hN m hm
trans :=
by
  intro f g h hfg hgh  $\varepsilon$  h $\varepsilon$ 
  cases hfg ( $\varepsilon / 2$ ) (by linarith) with
  | intro N1 hN1 =>
    cases hgh ( $\varepsilon / 2$ ) (by linarith) with
    | intro N2 hN2 =>
      apply Exists.intro (max N1 N2)
      intro m hm
      calc
        abs (seqOf f m - seqOf h m)
        ≤ abs (seqOf f m - seqOf g m)
          + abs (seqOf g m - seqOf h m) :=
          by apply abs_sub_le
      _ <  $\varepsilon / 2$  +  $\varepsilon / 2$  :=
        add_lt_add (hN1 m (le_of_max_le_left hm))
          (hN2 m (le_of_max_le_right hm))
      _ =  $\varepsilon$  :=
        by simp } }

```

```

theorem Setoid_iff (f g : CauchySeq) :
  f ≈ g ↔
  ∀  $\varepsilon > 0$ , ∃ N, ∀ m ≥ N, abs (seqOf f m - seqOf g m) <  $\varepsilon$  :=
  by rfl

```

我们可以把 0 和 1 这样的常量定义为常值序列。任意常值序列都是 **Cauchy** 序列：

```

def const (q :  $\mathbb{Q}$ ) : CauchySeq :=
  Subtype.mk (fun _ :  $\mathbb{N}$  ↦ q)
  (by
    rw [IsCauchySeq]

```

```
intro  $\epsilon$  h $\epsilon$ 
aesop)
```

定义实数加法需要更多工作。我们把 **Cauchy** 序列上的加法定义为逐项加法:

```
instance Add : Add CauchySeq :=
{ add := fun f g : CauchySeq  $\mapsto$ 
  Subtype.mk (fun n :  $\mathbb{N}$   $\mapsto$  seqOf f n + seqOf g n)
  (by
    intro  $\epsilon$  h $\epsilon$ 
    obtain ⟨N1, hN1⟩ :=
      Subtype.property f ( $\epsilon$  / 4) (by linarith)
    obtain ⟨N2, hN2⟩ :=
      Subtype.property g ( $\epsilon$  / 4) (by linarith)
    let N := N1 + N2
    use N
    intro m m_geq_N
    have m_geq_N1 : m  $\geq$  N1 :=
      by simp[N] at m_geq_N; linarith
    have m_geq_N2 : m  $\geq$  N2 :=
      by simp[N] at m_geq_N; linarith

    have h_fN_fm : |seqOf f N - seqOf f m| <  $\epsilon$  / 2 :=
      by
        have : |seqOf f N1 - seqOf f N| <  $\epsilon$  / 4 :=
          hN1 N (by aesop)
        have : |seqOf f N1 - seqOf f m| <  $\epsilon$  / 4 :=
          hN1 m m_geq_N1
        calc
          |seqOf f N - seqOf f m|
            = |(seqOf f N - seqOf f N1) + (seqOf f N1 - seqOf f m)| :=
              by aesop
          _  $\leq$  |seqOf f N - seqOf f N1| + |seqOf f N1 - seqOf f m| :=
              by apply abs_add_le
          _ = |seqOf f N1 - seqOf f N| + |seqOf f N1 - seqOf f m| :=
              by simp[abs_sub_comm]
          _ <  $\epsilon$  / 4 +  $\epsilon$  / 4 :=
              by linarith
          _ =  $\epsilon$  / 2 :=
```



```

    by linarith

have h_gN_gm: |seqOf g N - seqOf g m| < ε / 2 :=
by
  have : |seqOf g N2 - seqOf g N| < ε / 4 :=
    hN2 N (by aesop)
  have : |seqOf g N2 - seqOf g m| < ε / 4 :=
    hN2 m m_geq_N2
  calc
    |seqOf g N - seqOf g m|
    = |(seqOf g N - seqOf g N2) + (seqOf g N2 - seqOf g m)| :=
      by aesop
    _ ≤ |seqOf g N - seqOf g N2| + |seqOf g N2 - seqOf g m| :=
      by apply abs_add_le
    _ = |seqOf g N2 - seqOf g N| + |seqOf g N2 - seqOf g m| :=
      by simp [abs_sub_comm]
    _ < ε / 4 + ε / 4 :=
      by linarith
    _ = ε / 2 :=
      by linarith

calc
  |seqOf f N + seqOf g N - (seqOf f m + seqOf g m)|
  = |(seqOf f N - seqOf f m) + (seqOf g N - seqOf g m)| :=
    by rw [add_sub_add_comm]
  _ ≤ |(seqOf f N - seqOf f m)| + |(seqOf g N - seqOf g m)| :=
    by apply abs_add_le
  _ < ε / 2 + ε / 2 :=
    by linarith
  _ = ε :=
    by linarith) }

```

接着，我们需要证明这个加法与 $\approx$ 相容：

```

theorem Setoid_add {f f' g g' : CauchySeq} (hf : f ≈ f')
  (hg : g ≈ g') :
  f + g ≈ f' + g' :=
by
  intro ε₀ hε₀

```

```

simp
cases hf ( $\epsilon_0 / 2$ ) (by linarith) with
| intro Nf hNf =>
cases hg ( $\epsilon_0 / 2$ ) (by linarith) with
| intro Ng hNg =>
apply Exists.intro (max Nf Ng)
intro m hm
calc
abs (seqOf (f + g) m - seqOf (f' + g') m)
= abs ((seqOf f m + seqOf g m)
- (seqOf f' m + seqOf g' m)) :=
by rfl
_ = abs ((seqOf f m - seqOf f' m)
+ (seqOf g m - seqOf g' m)) :=
by
have arg_eq :
seqOf f m + seqOf g m
- (seqOf f' m + seqOf g' m) =
seqOf f m - seqOf f' m
+ (seqOf g m - seqOf g' m) :=
by linarith
rw [arg_eq]
_ ≤ abs (seqOf f m - seqOf f' m)
+ abs (seqOf g m - seqOf g' m) :=
by apply abs_add_le
_ <  $\epsilon_0 / 2 + \epsilon_0 / 2$  :=
add_lt_add (hNf m (le_of_max_le_left hm))
(hNg m (le_of_max_le_right hm))
_ =  $\epsilon_0$  :=
by simp

```

实数就是该商:

```

def Real : Type :=
  Quotient CauchySeq.Setoid

```

```

instance Zero : Zero Real :=
  { zero := ⌊CauchySeq.const 0⌋ }

```

```

instance One : One Real :=
  { one := [[CauchySeq.const 1]] }

instance Add : Add Real :=
  { add := Quotient.lift₂ (fun a b : CauchySeq ↦ [[a + b]])
    (by
      intro a b a' b' ha hb
      apply Quotient.sound
      exact CauchySeq.Setoid_add ha hb) }

```

13.5.3 $\mathbb{R}$ 的替代定义

- Dedekind 分割: $r : \mathbb{R}$ 本质上表示为 $\{x : \mathbb{Q} \mid x < r\}$ 。
- 二进制序列 $\mathbb{N} \rightarrow \mathbf{Bool}$ 可以表示区间 $[0, 1]$ 。这可用于构造 $\mathbb{R}$ 。